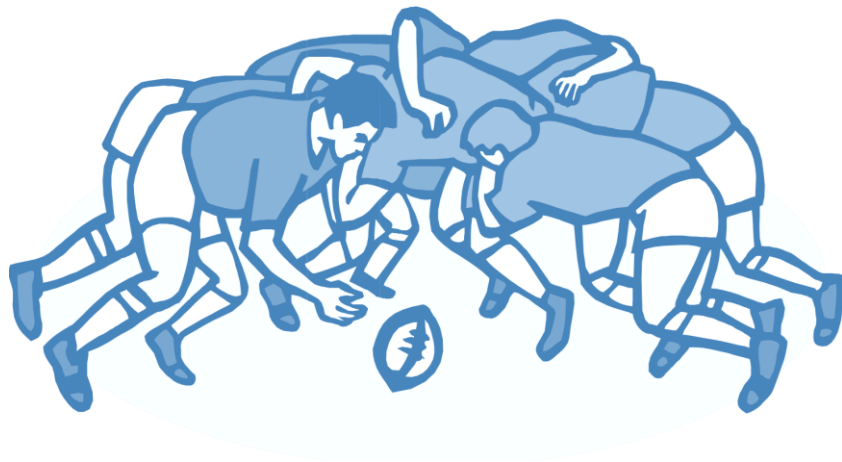


CÔNG NGHỆ PHẦN MỀM CHUYÊN SÂU
OVERVIEW OF AGILE DEVELOPMENT



Reference Materials

1. Agile Retrospectives: Make Good Teams Great – Esther Derby, Diana Larsen, Ken Schwaber
2. Agile Estimating and Planning – Mike Cohn
3. User Stories Applied: For Agile Software Development: Mike Cohn
4. Agile Project Management with Scrum: Ken Schwaber
5. Scrum.Inc
6. Scrum Org
7. Scrum Alliance
8. Scrum Guide

1: AGILE

Content

- Agile Introduction
- Agile vs. Waterfall
- Agile Methodologies
- Agile Practices
- Agile Myths

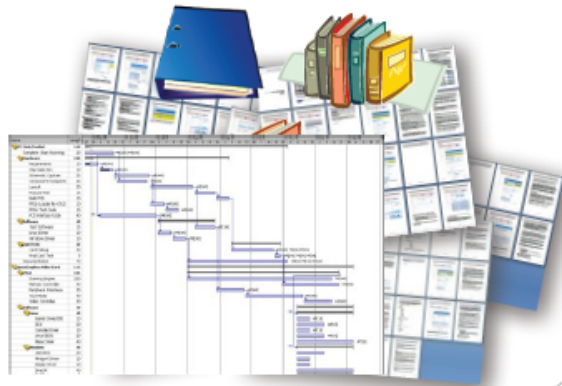
→ Section 2. SCRUM Framework

Traditional Approach

- **Waterfall** – Predictive approach
 - 3 things **we wish were true IN WATERFALL**
 - The customer **knows what he/she wants**
 - The developers **know how to build it**
 - **Nothing will change** along the way
 - 3 things **we have to LIVE WITH**
 - The customer **discovers what he/she wants**
 - The developers **discover how to build it**
 - **Many things change** along the way

Traditional Approach (cont.)

WATERFALL IS LIKE A CANNONBALL



ASSUMPTIONS:

- THE CUSTOMER KNOWS WHAT HE WANTS
- THE TEAM KNOWS HOW TO BUILD IT
- NOTHING WILL CHANGE ALONG THE WAY

Agile Approach

AGILE IS LIKE A HOMING MISSILE



ASSUMPTIONS:

- THE CUSTOMER DISCOVERS WHAT HE WANTS
- THE TEAM DISCOVERS HOW TO BUILD IT
- MANY THINGS CHANGE ALONG THE WAY

Why Are Projects Late?

Third party projects are almost always late...

- Things change. Over **65% of the requirements change** during the average development project.
- **Change is good for business.**
Competitive bidding means an initial bid has low profit margin. The money is made on changes billed at time and materials.
- The project has to **be late for vendors** to make a lot of money.

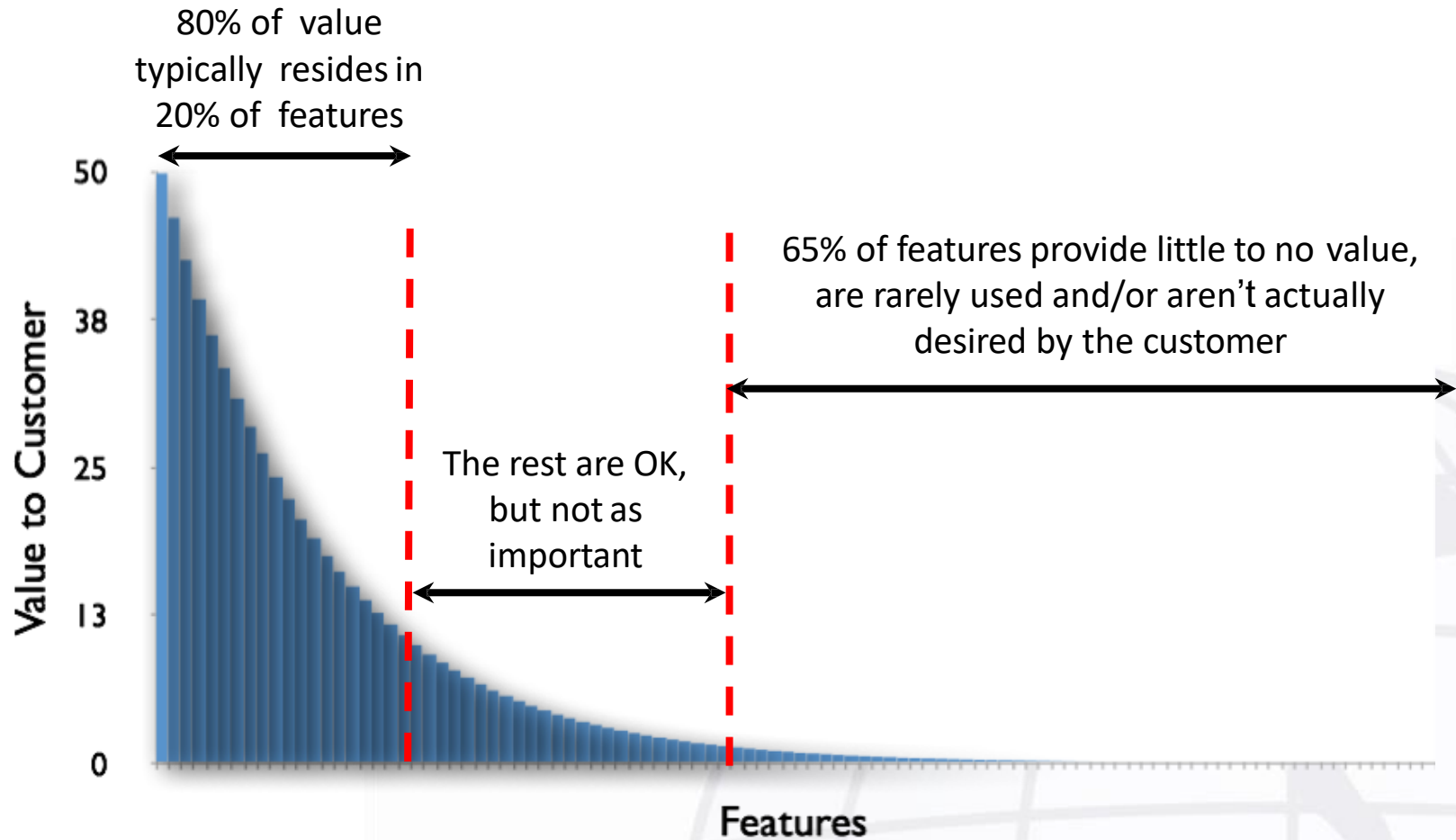
...But **internal projects** are usually worse

- On average, **80% of the cost** of a project is spent up front in the decision process, project management, and requirements development.
- **Implementation starts building assuming nothing will change.**
- **Change control boards** try to fend off change.
- By the time it becomes clear that 65% of the requirements are changing, **most of the time and budget has been spent.**

Why Do Things Change?

- Users **don't know what they want** until they see working product (Humphrey's Law).
- Since users have to **specify all requirements up front**, they put **everything but the kitchen sink** in the requirements documents. **65% of features are never or rarely used.**
- Users discover what they want later, particularly when the **business changes**. By that time they have spent their budget on a lot of **unnecessary features**.

Not All Features Are Created Equal!



The Solution - Get Your Change for Free

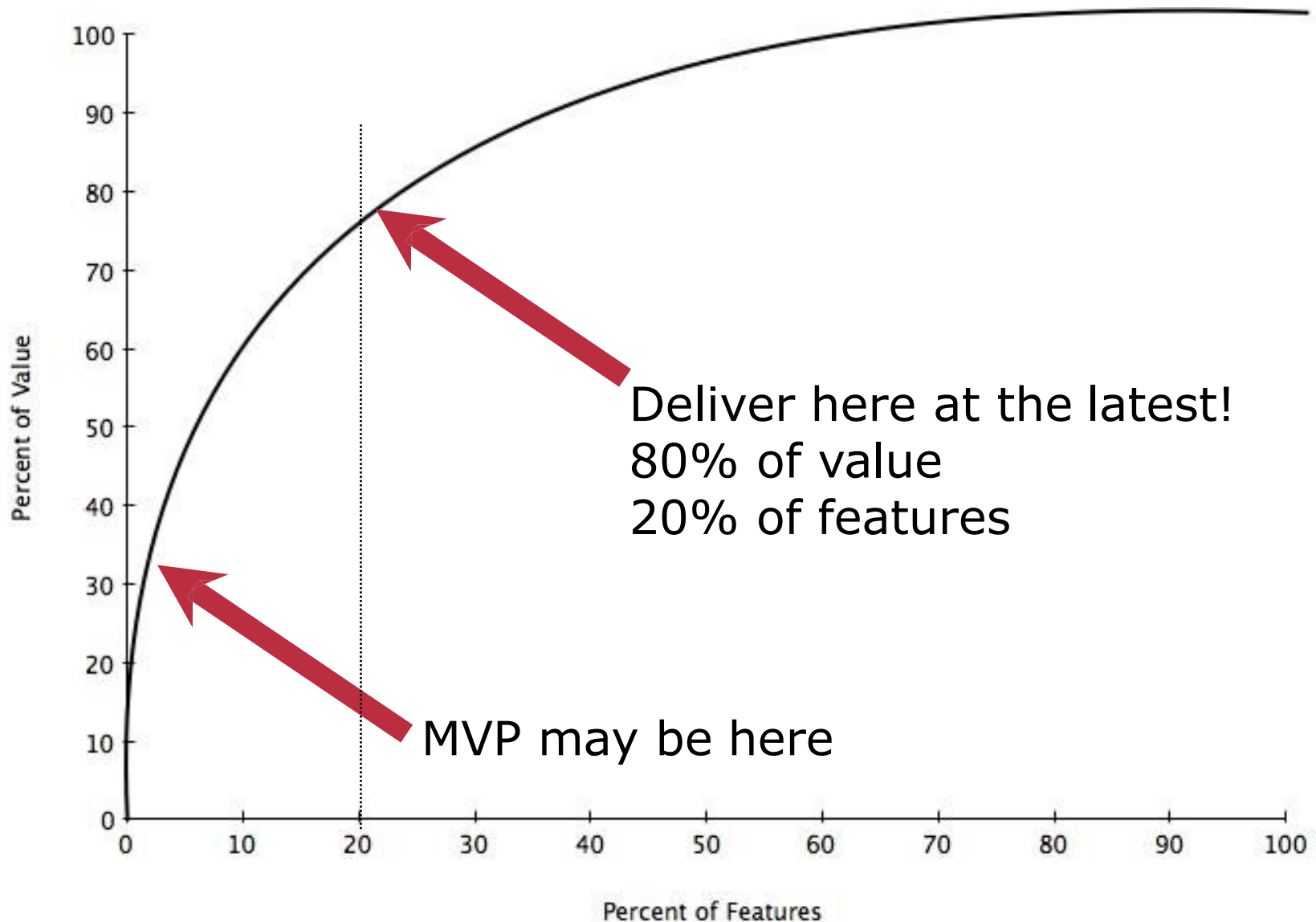
- Create a prioritized **backlog** of work to be done with **highest business value** items first.
- Implement in **short sprints**, always less than a month.
- When **higher priority requirements emerge**, put them in the **next sprint**.
- **Cut lowest priority** items out of the project equal to the amount of work added. These features are unlikely to be used anyway.
- **Change for free** allows you to meet your budget and deliver on time.

Money for Nothing:

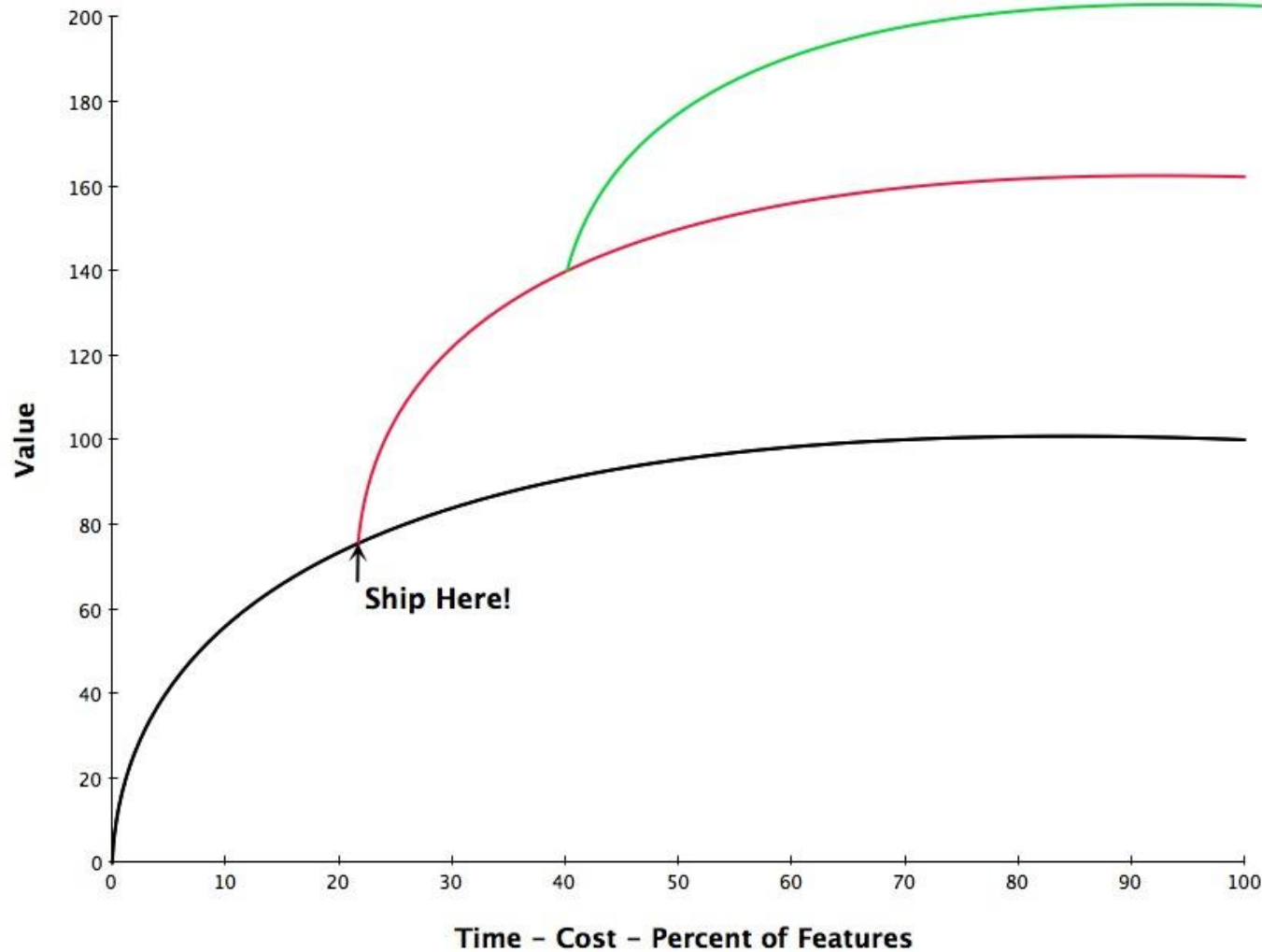
Even Better Than Change for Free

- Projects are usually prioritized by **return on investment**.
- Ordering your **Product Backlog** allows you to prioritize features by return on investment.
- Since **65% are never or rarely used**, during the project it will become evident that the next low priority feature costs more than the value it delivers.
- **Stop the project at that point and deploy the valuable features.**
- All projects should deliver early and save money.

Value Curve - Radically Faster Delivery

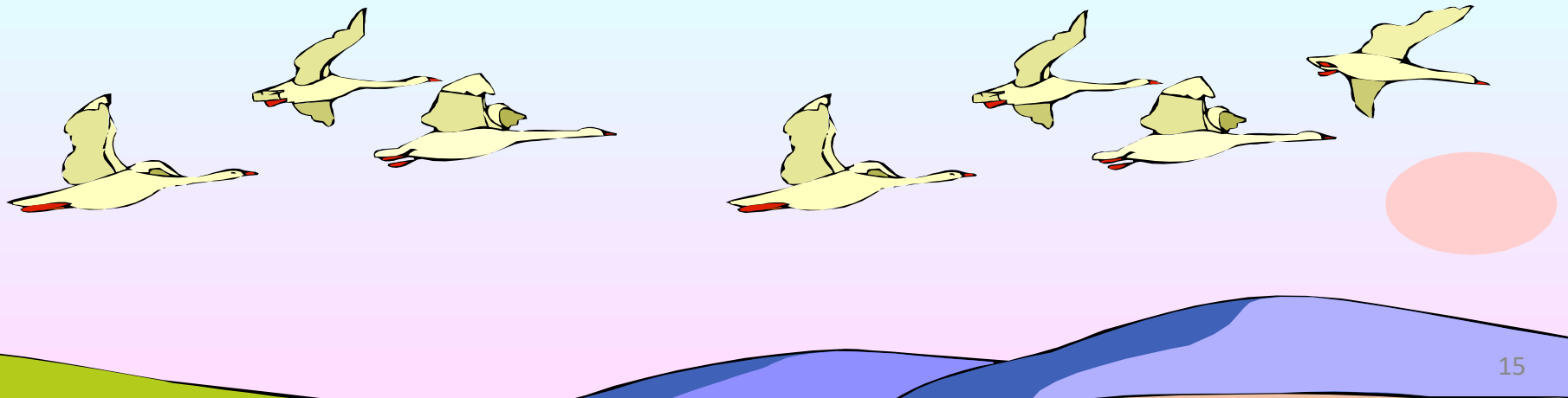


Compounding Value - Radically Better Delivery



Waterfall (Defined process) vs. Agile (Empirical Process)

- “It is typical to adopt the defined (theoretical) modeling approach when the underlying mechanisms by which a process operates are reasonably well understood.
- When the process is too complicated for the defined approach, the empirical approach is the appropriate choice.” → SCRUM is Empirical Process.



Agile vs. Waterfall

WATERFALL



AGILE



Deliverable Product



Chance of failure



Project Run Rate

Agile development uses an iterative approach to reduce risk and create products that are "launchable" throughout development.

The Agile Paradigm Shift

Waterfall

Agile

Fixed

Requirements

Resources

Time

**PLAN
driven**

**VALUE
driven**

Estimated

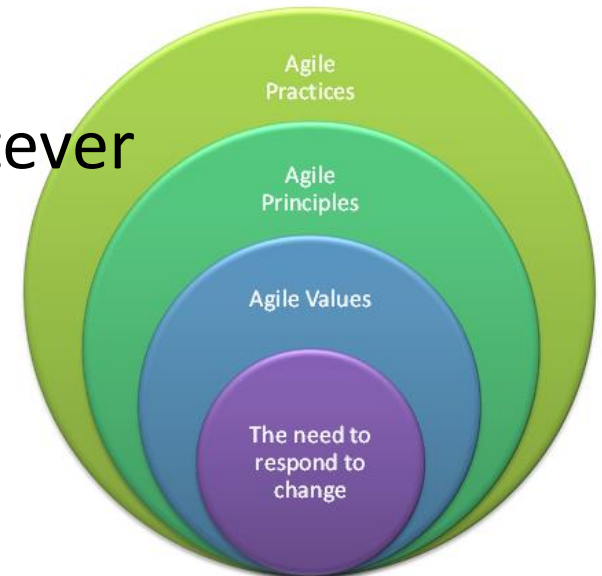
Resources

Time

Features

What is Agile?

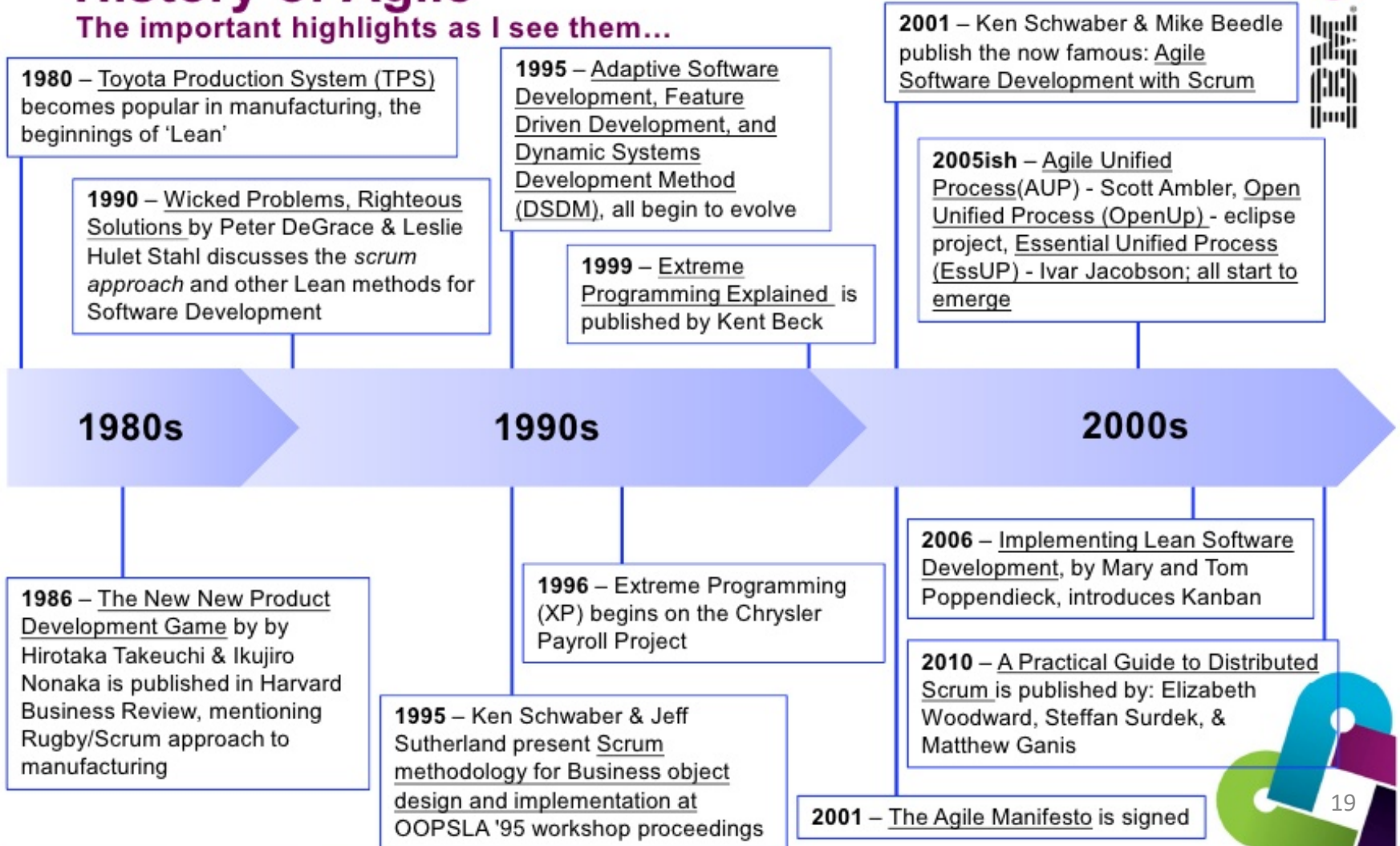
- Agile is about Values and Principles
 - People and organizations **can not** do Agile – they **can be** Agile
- Working practices can be Agile and fit into Values and Principles
 - Teams can do Scrum or XP or whatever



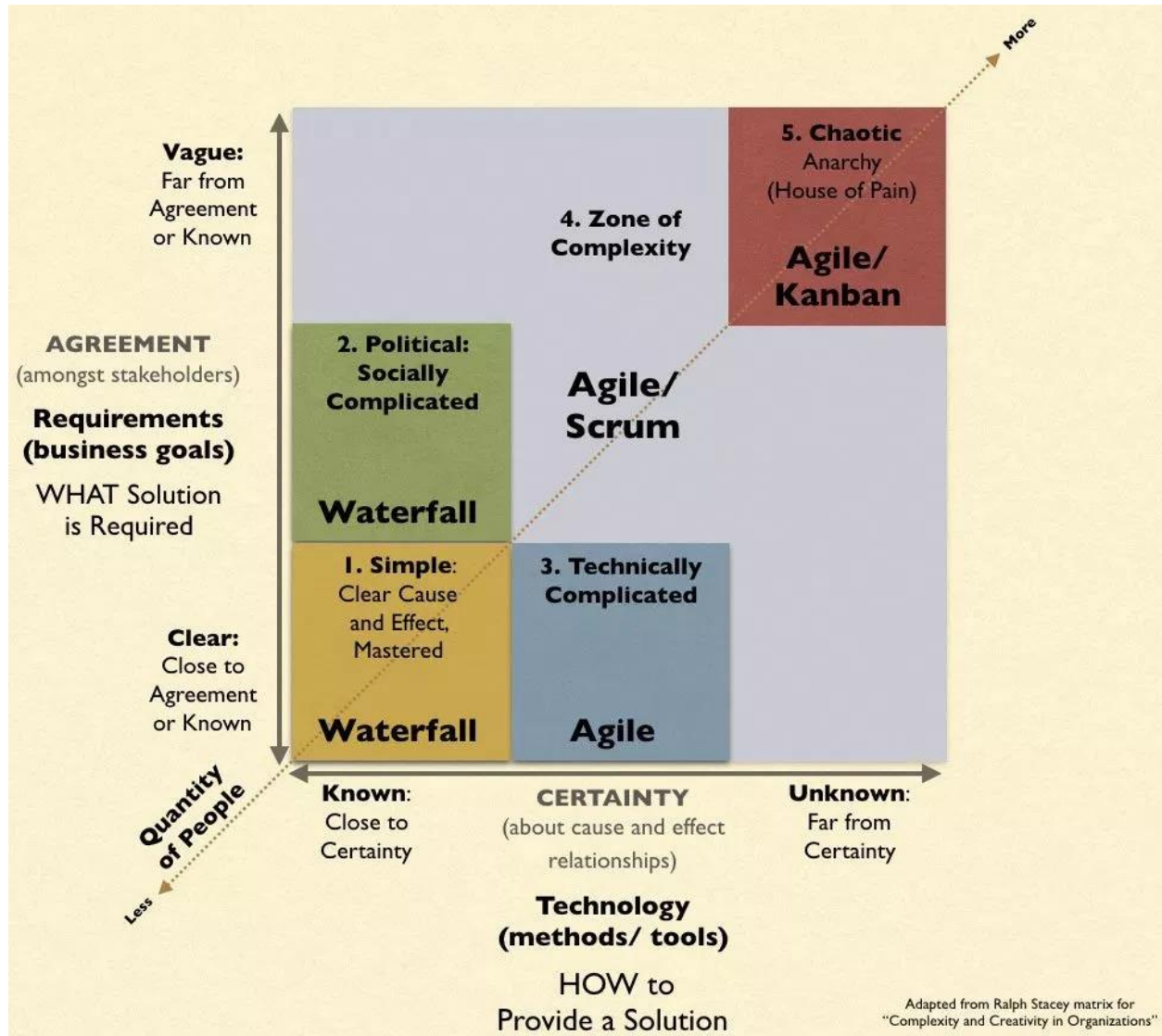
What is Agile?

History of Agile

The important highlights as I see them...



What is Agile?





The Agile Manifesto* (**Agile Values**)

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right,
we value the items on the **left** more.”

(2001, **Kent Beck**, Mike Beedle, Arie van Bennekum, Alistair **Cockburn**, Ward **Cunningham**,
Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern,
Brian Marick, Robert C. Martin, Steve Mellor, **Ken Schwaber**, **Jeff Sutherland**, Dave Thomas)

* www.agilemanifesto.org

12 Agile Principles

1. Our **highest** priority is to **satisfy the customer** through early and continuous delivery of valuable software.
2. **Welcome changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage.
3. **Deliver** working software **frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must **work together daily** throughout the project.

12 Agile Principles (cont.)

5. Build projects around **motivated individuals**. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is **face-to-face conversation**.
7. Working software is the **primary measure of progress**.
8. Agile processes promote **sustainable development**. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

12 Agile Principles (cont.)

9. Continuous attention to **technical excellence and good design** enhances agility.
10. **Simplicity**--the art of maximizing the amount of work not done--is essential.
11. The best architectures, requirements, and designs **emerge from self-organizing teams**.
12. At regular intervals, **the team reflects** on how to become more effective, then tunes and **adjusts** its behavior accordingly.

How the manifesto impacts business through its 12 Agile Principles – **Working Software**

1. Our highest priority is to satisfy the customer through **early and continuous delivery** of valuable software
3. **Deliver working software frequently**, from a couple of weeks to a month, with a preference to the shorter timescale
7. **Working software is the primary measure of progress**
9. Continuous attention to **technical excellence and good design** enhances agility
10. Simplicity - the art of **maximizing the amount of work not done (not need at current)** - is essential

Agile Principles (cont.) – Responding to Change / Customer Collaboration

2. We welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage

4. Business people and developers must work together daily throughout the project

Agile Principles (cont.) – People & Interactions

6. The most efficient and effective method of conveying information to and within a development team is **face-to-face conversation**

5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done

11. The **best architectures**, requirements, and designs emerge from **self-organizing teams**

8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a **constant pace indefinitely**

12. At regular intervals, the team **reflects on how to become more effective**, then **tunes and adjusts its behavior accordingly**

12 Agile Principles ?

1 Satisfy the **customer**



2 Welcome **change**



3 Deliver **frequently**

Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5
story	story	story	story	story
story	story	story	story	story
story	story	story	story	story

4 Work **together**



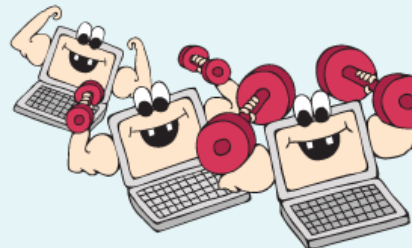
5 Trust and **support**



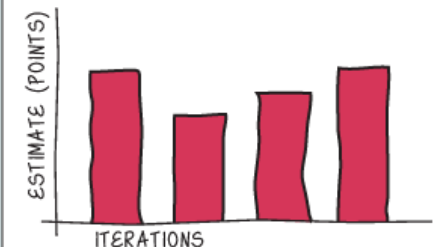
6 Face-to-face **conversation**



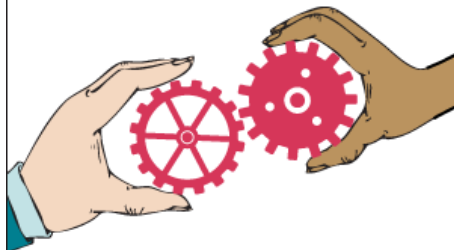
7 Working **software**



8 Sustainable **development**



9 Continuous **attention**



10 Maintain **simplicity**



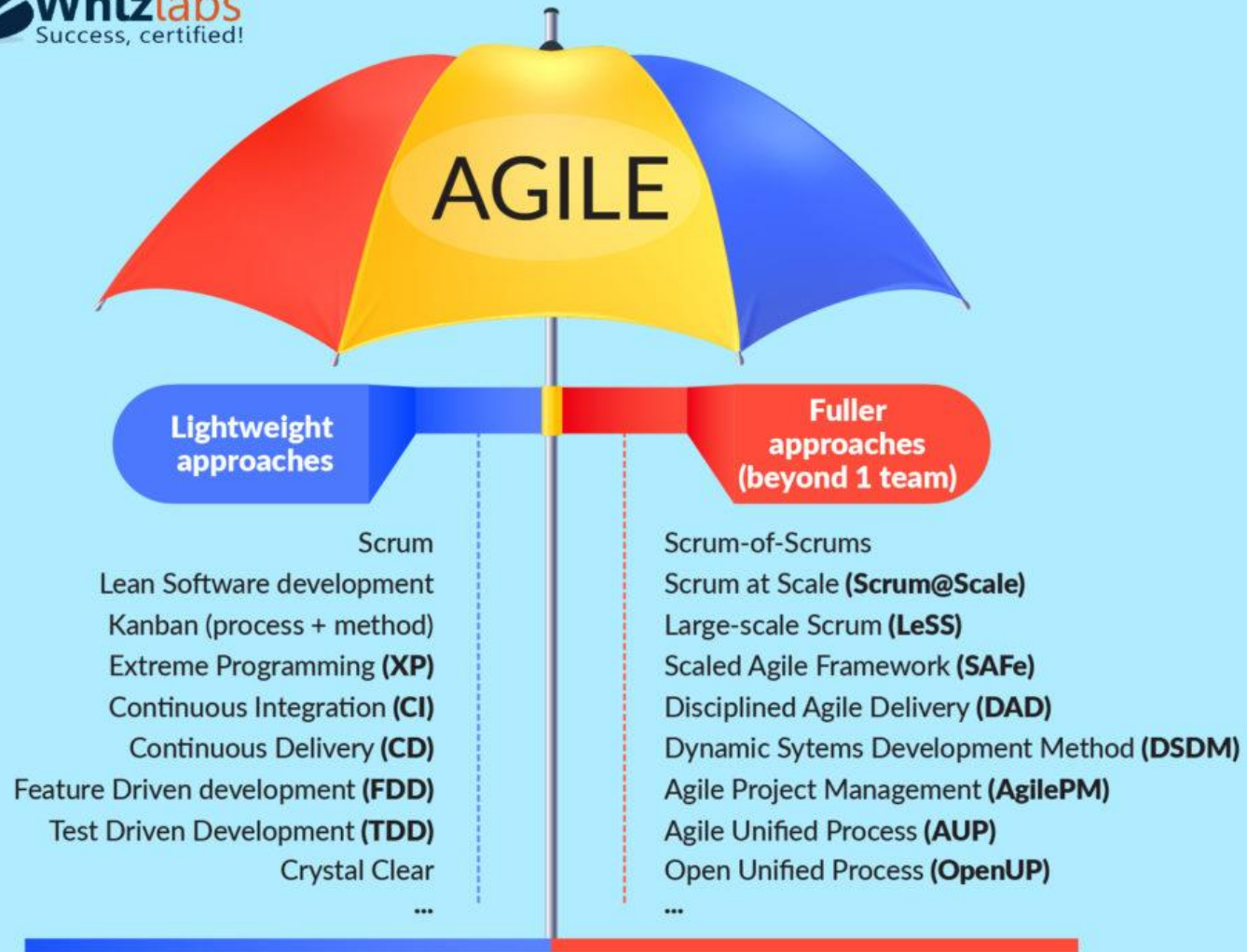
11 Self-organizing **teams**



12 Reflect and **adjust**

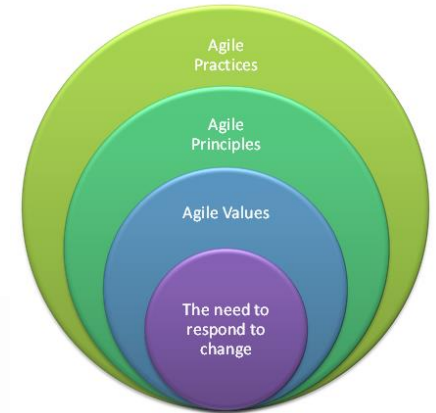


Agile Methodologies



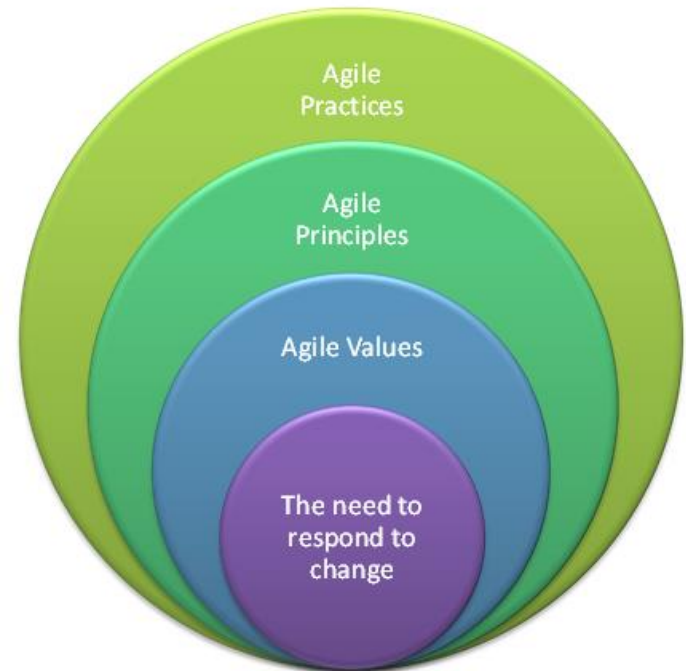
Agile Practices

- Practices for **Effective Teamwork**
 - Model with Others
 - Active Stakeholder Participation
 - Collective ownership
 - Display Models Publicity
- Practices that enable Simplicity
 - Create simple content
 - Use the simplest tool



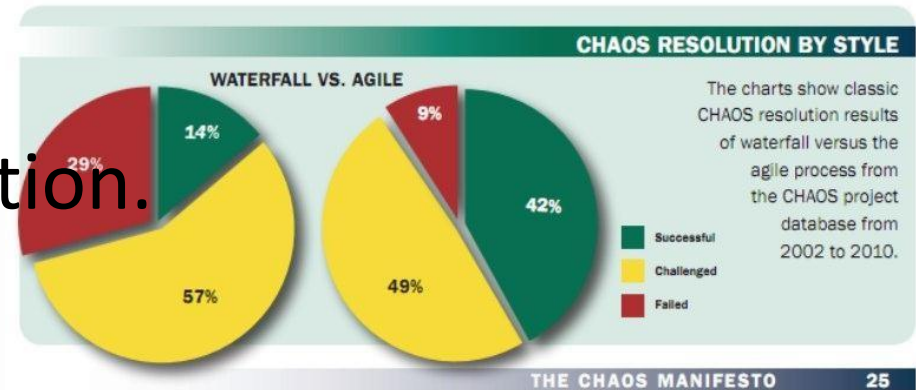
Agile Practices (cont.)

- Practices for Validation Your Work
 - Consider testability
 - Prove It with Code
- Pair programming
- Continuous integration
- Feedback loop
- Daily meeting
- Code Review
- Test-Design Driven (TDD)

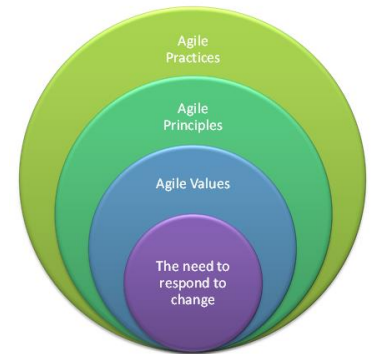


Agile Myths

- Agile is a silver bullet
- Agile is anti-documentation.
- Agile is anti-planning.
- Agile is undisciplined.
- Agile requires a lot of rework.
- Agile is anti-architecture.
- Agile doesn't scale.

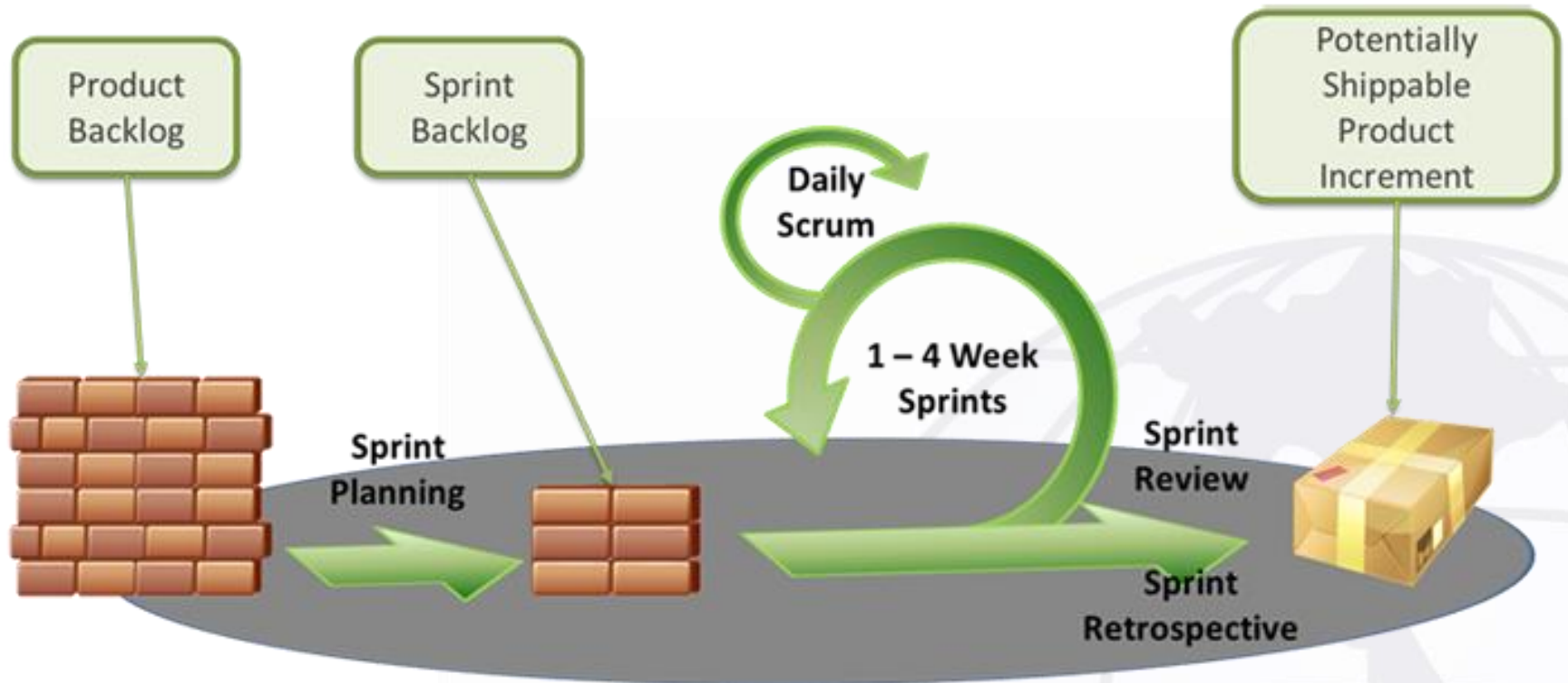


http://www.agilenutshell.com/agile_myths



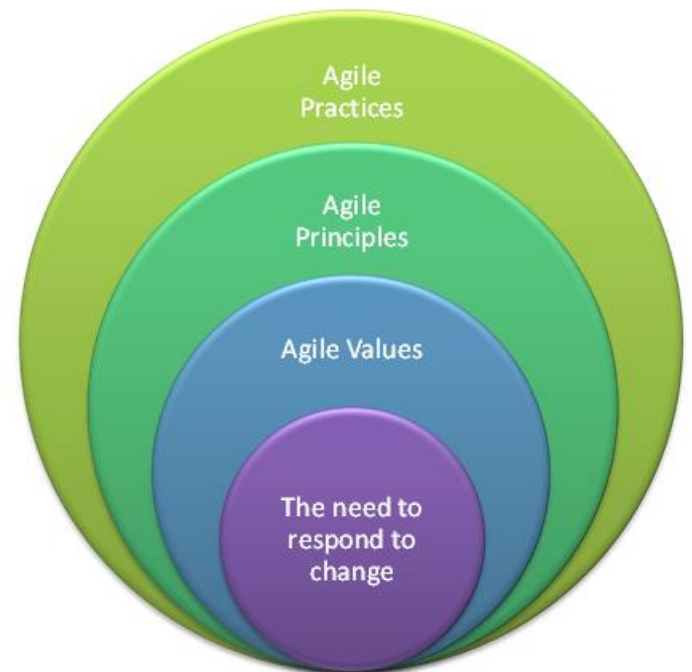
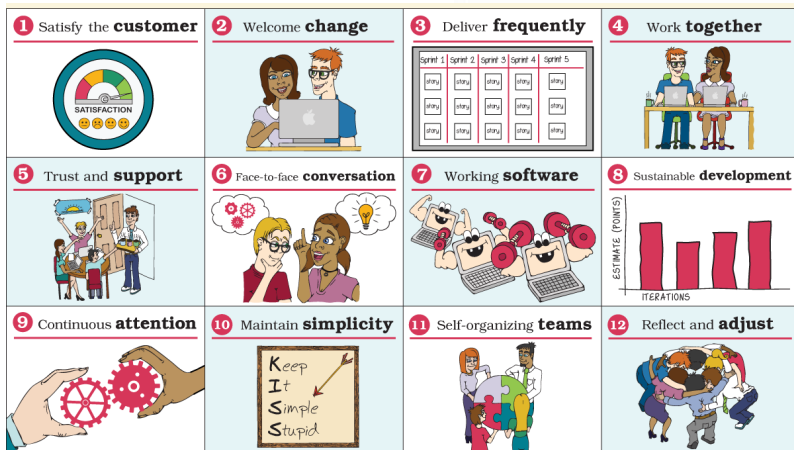
Be agile...
Don't just “do agile”
We help organizations and teams get there

Section 2: SCRUM Framework



Group Discussion

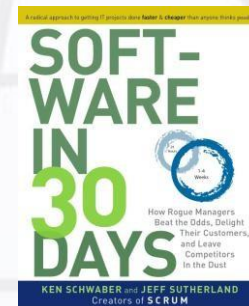
- What are the advantages and disadvantages when applying agile to project?
- What needs to be changed in order to apply agile to project?
- **Why managers need agile?**



Why managers need agile?

Agile is Faster, Better

- Projects can be delivered **at 50% of the cost** with **40% fewer defects**.
 - J. Sutherland, C. Jacobson, and K. Johnson, "Scrum and CMMI Level 5: A Magic Potion for Code Warriors!," in Agile 2007, Washington, D.C., 2007.
- Customers are happier and developers have more fun.
- The agile process is the universal **remedy** for software development project failure.
- Software applications developed through the agile process have three times the success rate of traditional waterfall method and a much lower percentage of time and cost overruns.





**Thank you for
your attention!**

**& See you in
2.SCRUM
FRAMEWORK**

2: SCRUM FRAMEWORK

Content

- SCRUM Introduction
- SCRUM Framework
- SCRUM Roles & Responsibility
- SCRUM Events

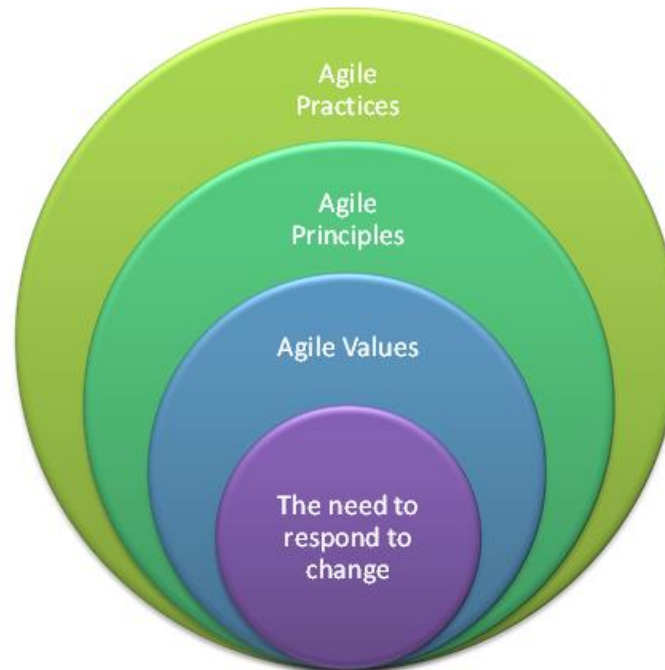
➔ Section 3. Artifacts, Events, and More in SCRUM

History of SCRUM

- Formalize in 1993 by Ken Schwaber and **Dr. Jeff Sutherland**
- Ken Schwaber **leads Scrum.org**
- Jeff Sutherland **leads ScrumAlliance.org (CEO of Scrum.inc)**
- Both organizations can provide Scrum certification
 - ScrumAlliance.org – CSM
 - Scrum.org – PSM
- In Sep 2014, Scrum.org and the Scrum Alliance agreed to release a common version of the Scrum Guide
<http://scrumguides.org/>

What is SCRUM?

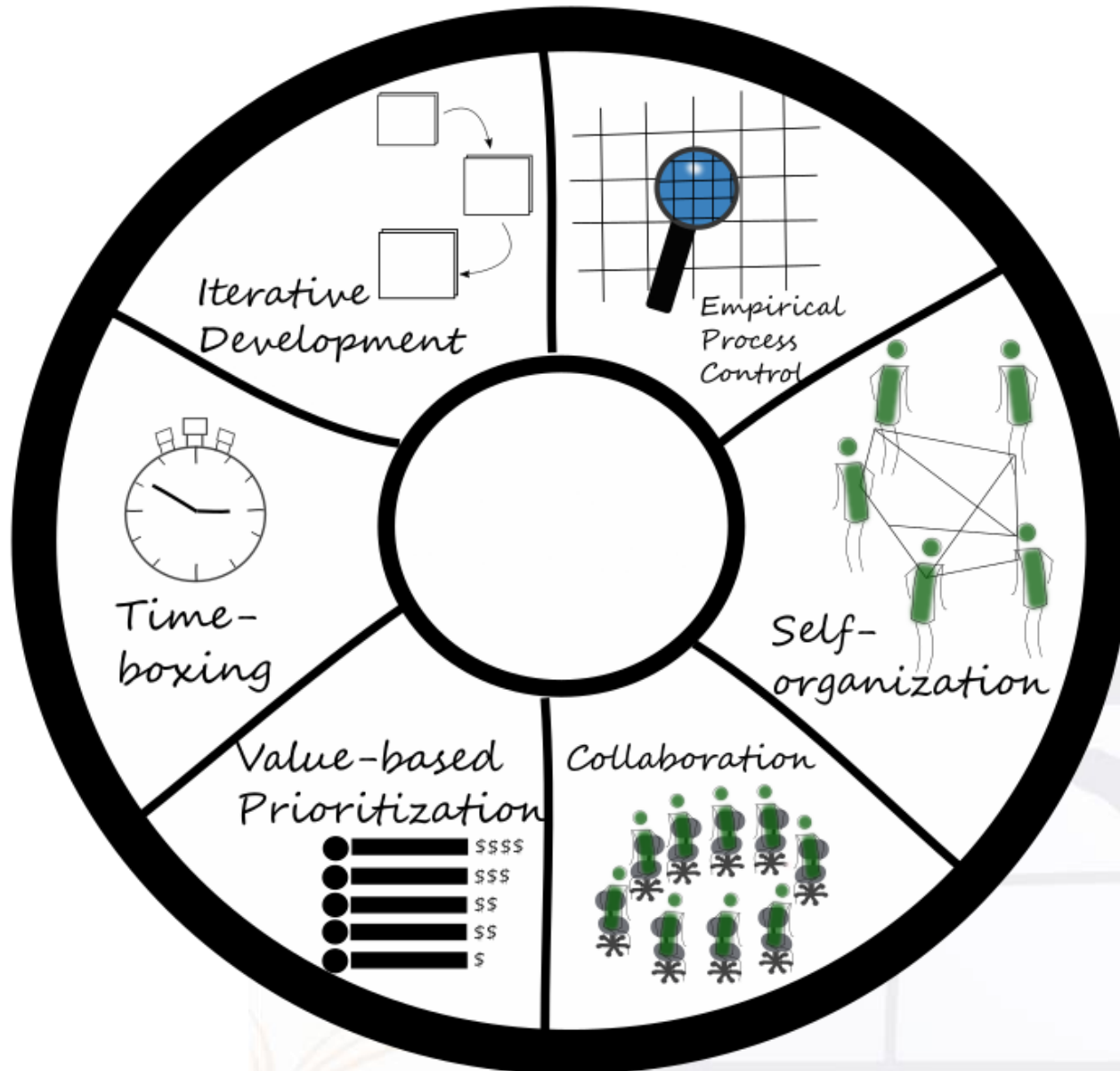
- Scrum is **not a process** or a **technique** for **building products**
- It is a framework within which **you can employ various processes and techniques**



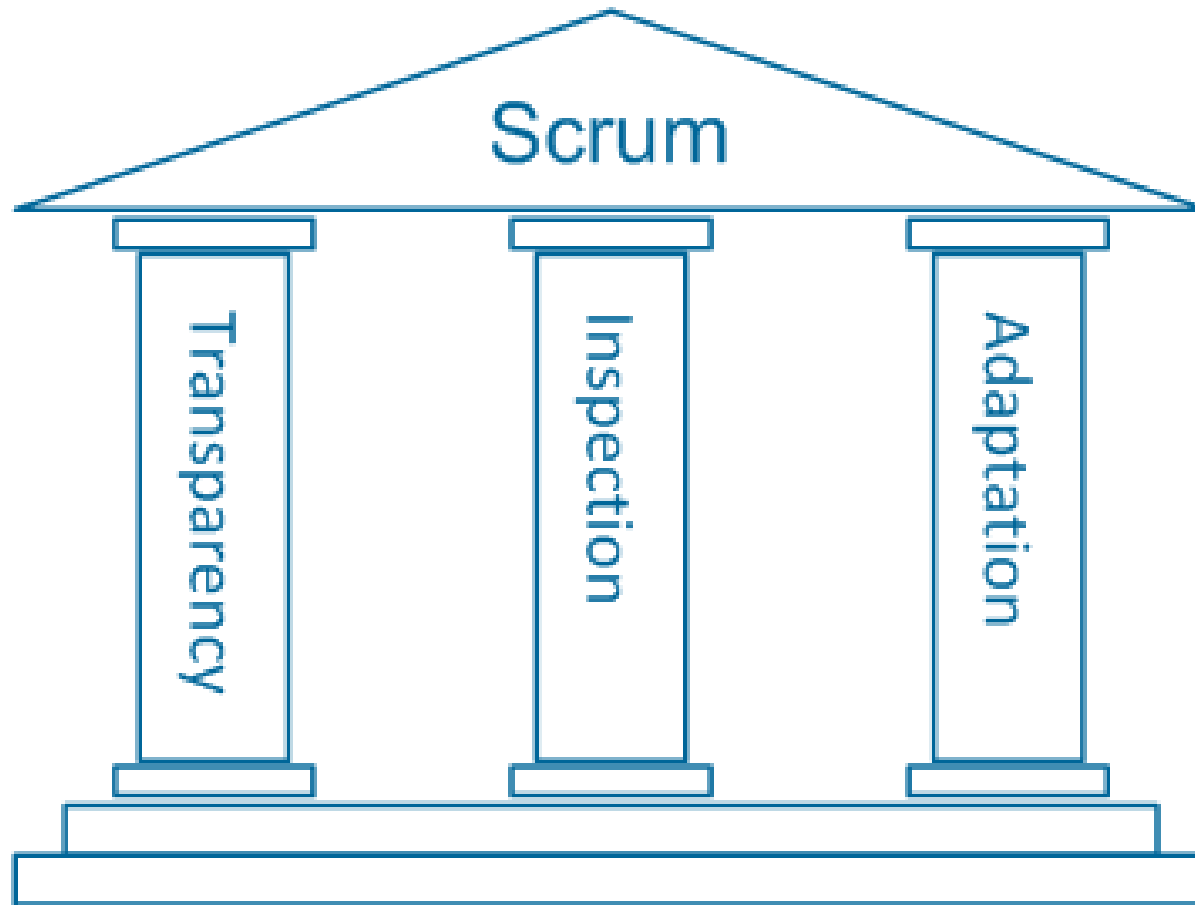
SCRUM Values



Principles of Scrum

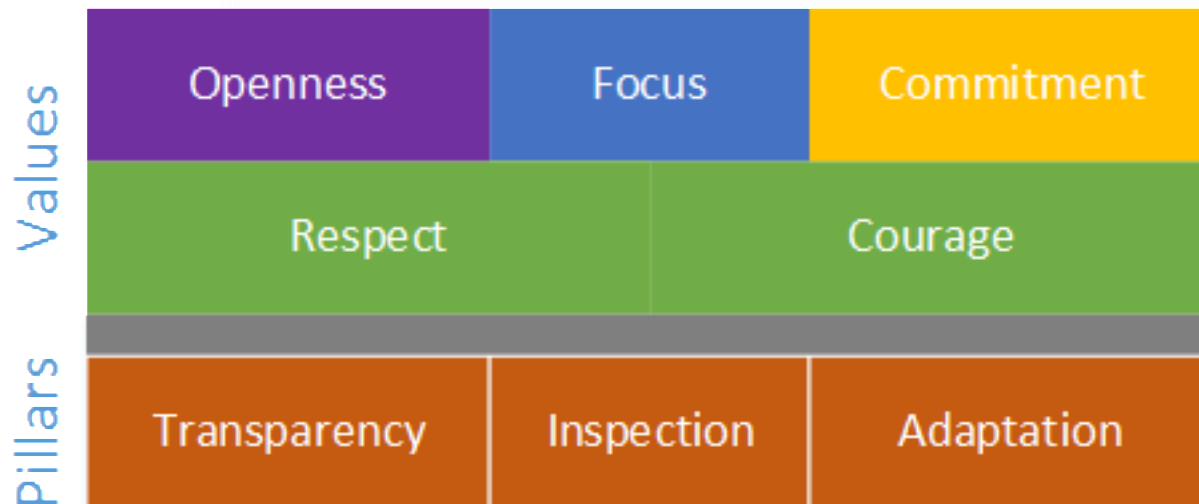


SCRUM Pillars



SCRUM Values & Pillars

The Five Scrum Values & Three pillars of empiricism

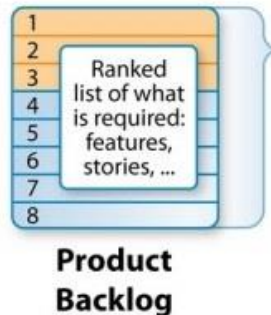


www.khurrambhatti.com

SCRUM Framework

The Agile: Scrum Framework at a glance

Inputs from Executives,
Team, Stakeholders,
Customers, Users



Burndown/up
Charts

Every
24 Hours

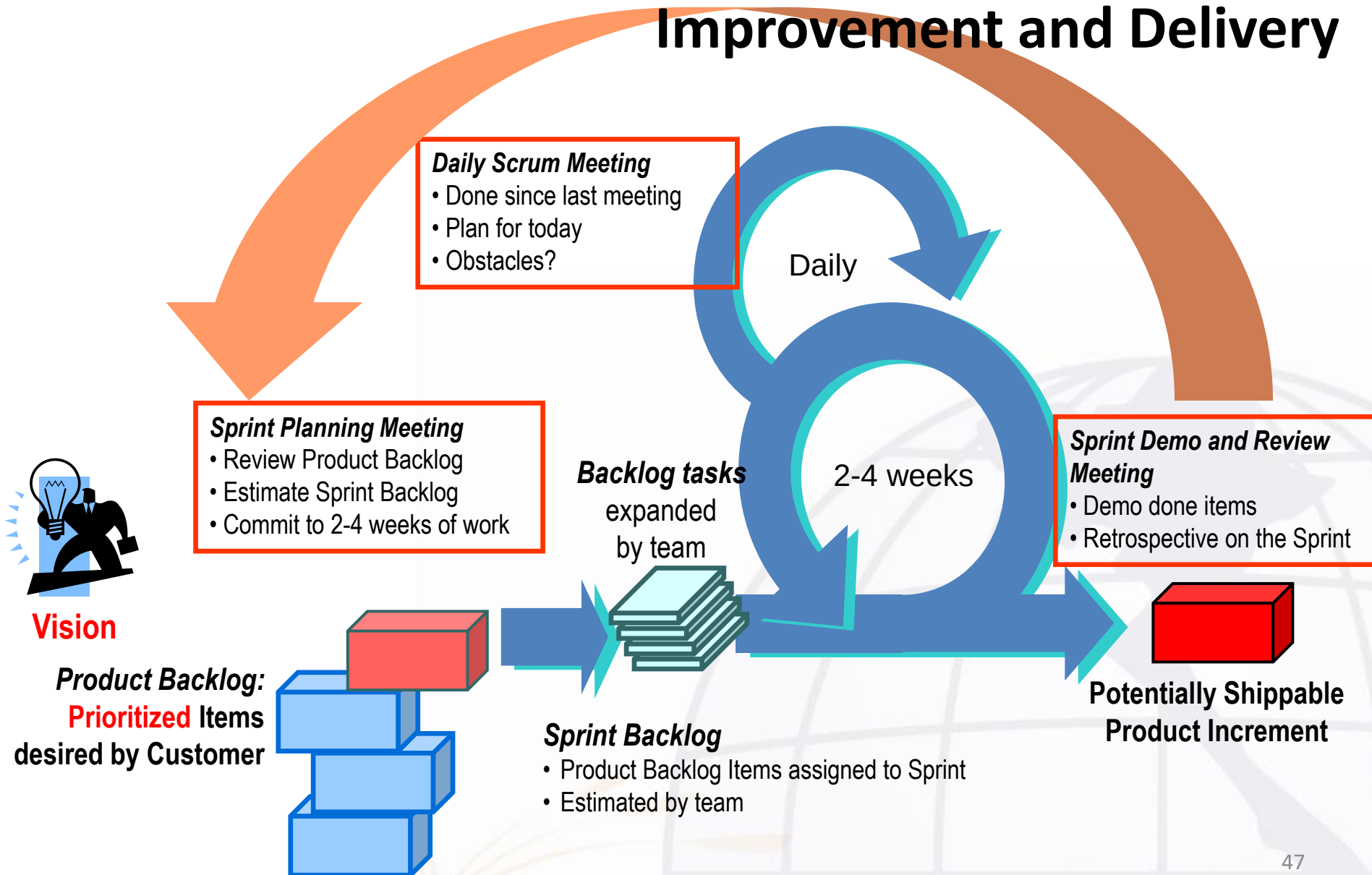


**1-4 Week
Sprint**



Sprint end date and
team deliverable
do not change

Now There is a Framework of Continuous Improvement and Delivery



SCRUM Roles

- Product Owner
- Scrum Master
- The Development Team



Product Owner

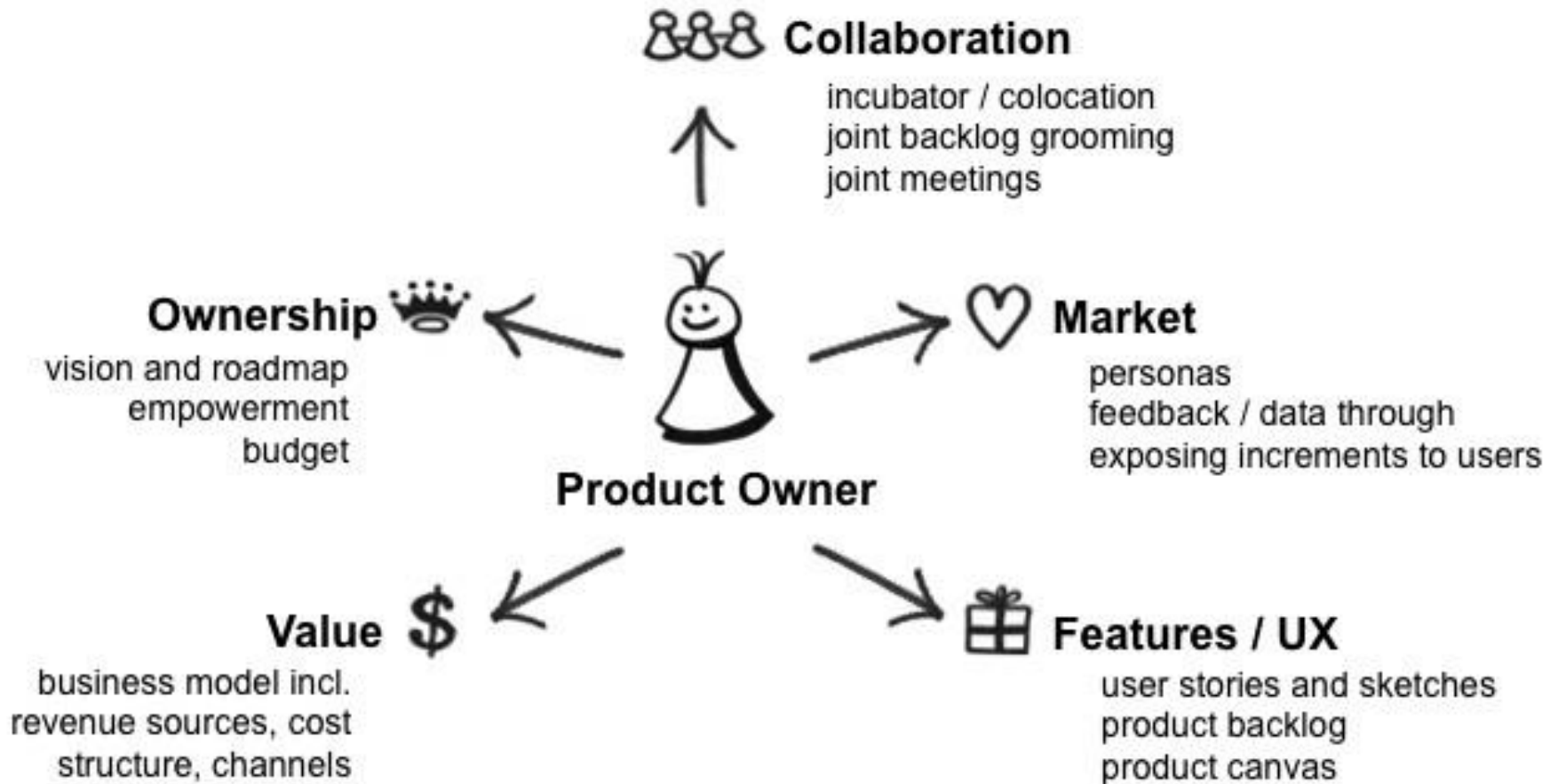
- Represents the stakeholders and is the **voice of the customer**
- **Owns the vision** of what would be produced to achieve business success
- Get input from **customer, manager, stakeholders**
- Turn this into a single list (**Product Backlog**) of what would be produced, prioritized based on business values and risks

Product Owner

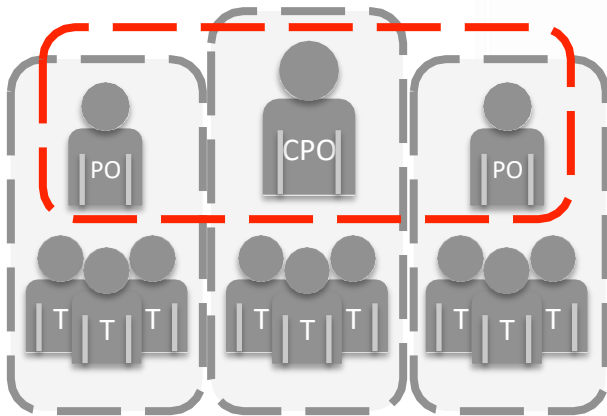
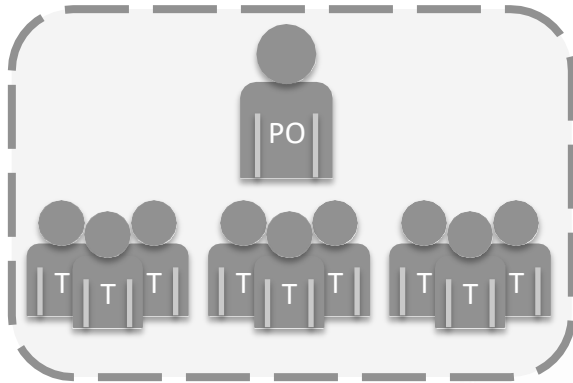
- **Responsibility:**

- Maximizing the value of the product and the work of the Development Team
- Managing the Product Backlog
 - Clearly expressing PBI (Product Backlog Item)
 - Ordering the items in the Product Backlog to best achieve goals and missions
 - Optimizing the value of the work the Development Team performs
 - Ensuring that the Product Backlog is visible, transparent, and clear to all, and shows what the Scrum Team will work on next
 - Ensuring the Development Team understands item in the Product Backlog to the level needed

Product Owner



Product Owner is a **Big Job**

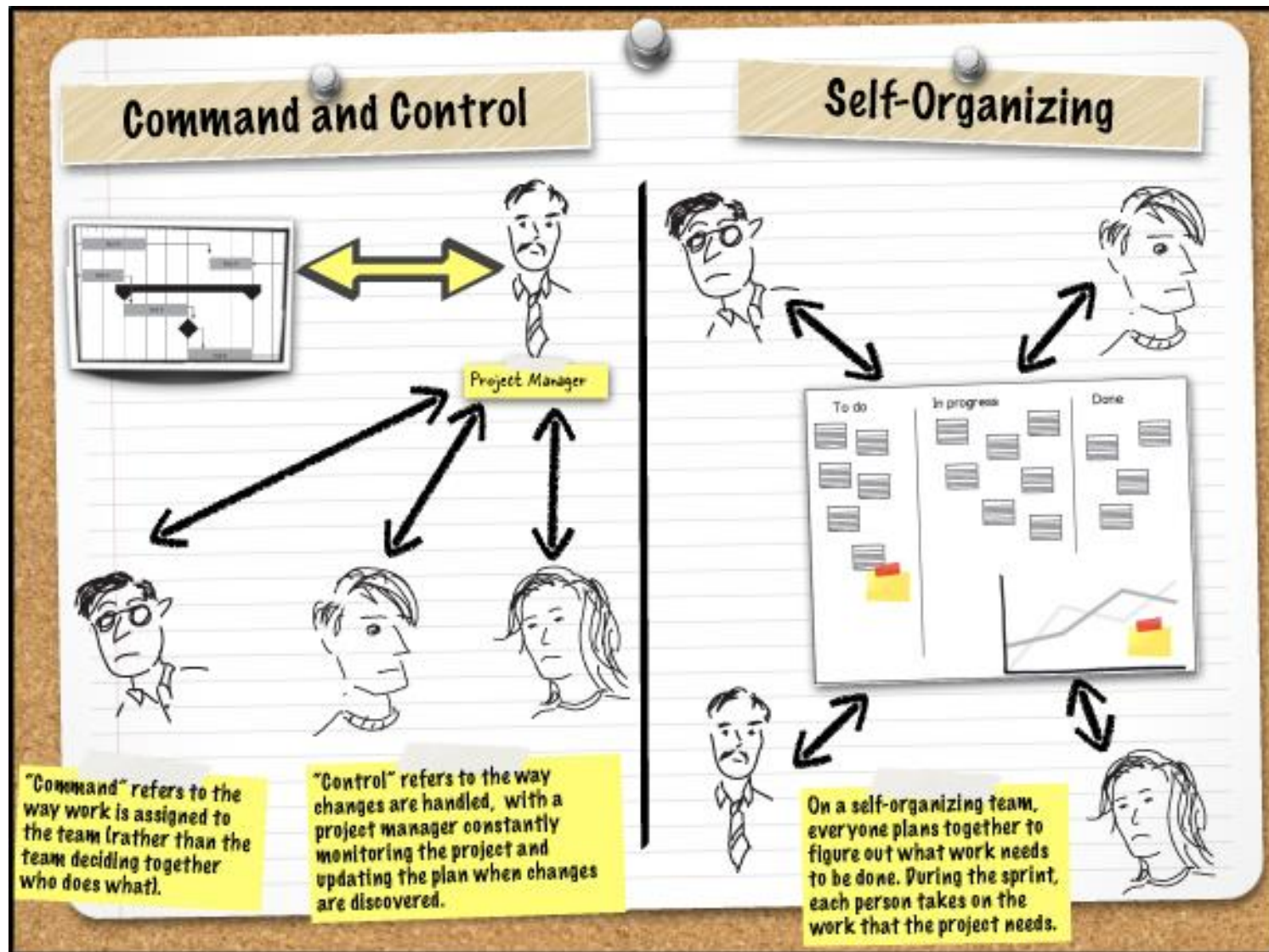


- Initially, one Product Owner may be able to generate ready backlog for several teams
- As team velocity increases, a Product Owner team, led by a Chief Product Owner, will be needed
- The Product Owner team are domain experts that describe the user experience, the screen shots, the workflow, the data requirements, the look and feel.

Development Team

- Is responsible for **delivering potentially shippable** product increments at the end of each Sprint.
- The **ideal team size** in SCRUM is 3 to 9 (in scrum guide)
- The team is **cross-functional**. It has all skills to produce finished product: designer, coder, tester, etc...
- Everyone contributes based on competency, rather than job title
- The team is **self-organizing** and **self-managing**. It is responsible for making a commitment and managing itself to hit the goal. SCRUM provides tool to help team to do it.

Development Team (cont.)



The Development Team (cont.)

- Developer Team:
 - No title other than Developer; No sub team
 - Accountability belongs the Development Team as a whole
 - Developers, Testers,
 - Who else do you need to deliver value for the increment?
 - *What about the Product Owner?*
- Activities:
 - Commit to the Sprint
 - Plan their work (tasks, dependencies)
 - Determine the estimates



SCRUM Master

- Is responsible for **ensuring Scrum (theory, practices, and rules)** is understood.
- Facilitates creativity and empowerment
- Improves productivity of development team in any way possible
- Improves engineering practices and tools so each sprint is ready to deploy
- Is a **servant-leader** for the Scrum Team
- Is not a project manager
- Does not have authority over the team

9 BEHAVIOURS of a servant leader

1. Serve first
2. Build trust
3. Live your values
4. Listen to understand
5. Think about your thinking
6. Add value to others
7. Demonstrate courage
8. Increase your influence
9. Live your transformation

Servant Leadership Institute

SCRUM Master (cont.)

- **Service to Product Owner**
 - Finding techniques for effective **Product Backlog management**
 - Helping the Scrum Team **understand the need for clear and concise PBIs**
 - **Understanding product planning** in an empirical environment
 - Ensuring the PO knows how to arrange the **Product Backlog to maximize value**

SCRUM Master (cont.)

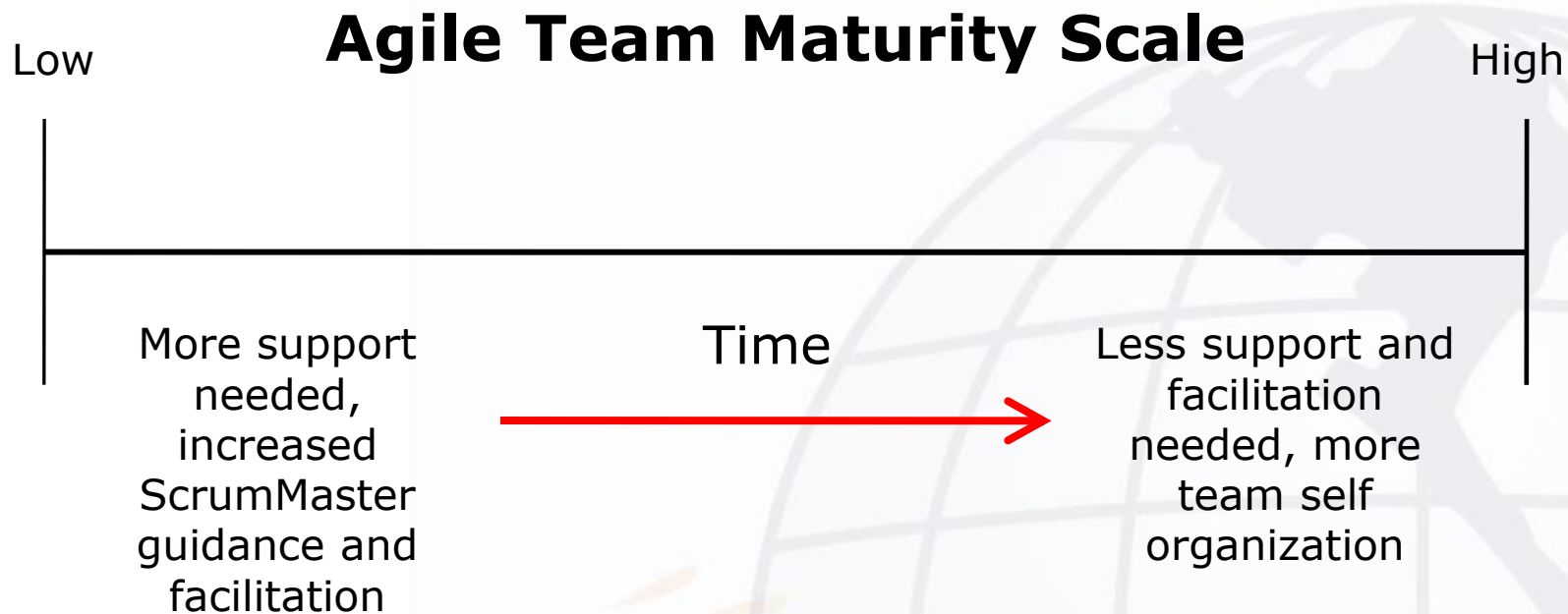
- **Service to Development Team:**
 - Coaching the Development Team in self-organization and cross-functionality
 - Helping the Development Team to create high-value products
 - Removing **impediments** to the Development Team's progress
 - Coaching the Development Team in organizational environments

SCRUM Master (cont.)

- **Service to Organization**
 - Leading and coaching the organization in its Scrum adoption
 - Planning Scrum implementations within the organization
 - Helping employees and stakeholders understand and enact Scrum and empirical product development
 - Causing change that increases the productivity of the Scrum Team
 - Working with other Scrum Masters to increase the effectiveness of the application of Scrum in the organization.

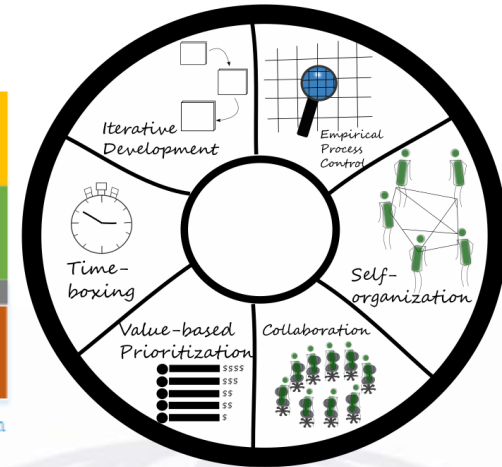
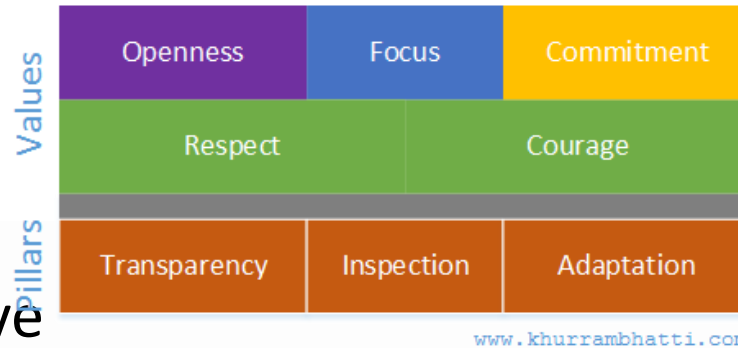
ScrumMaster Facilitates Team Organization and Maturity

- Self-organizing teams evolve and grow over time
- A ScrumMaster's involvement evolves with the team on the Agile Team Maturity Scale



SCRUM Events

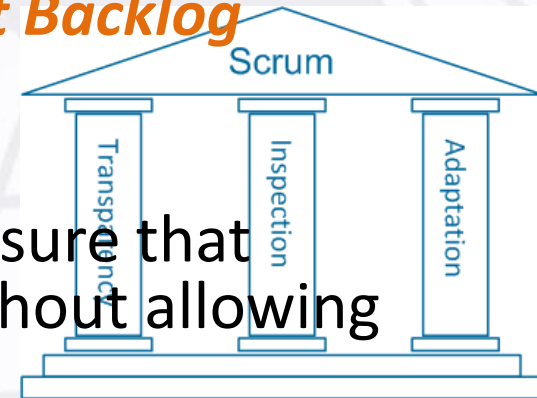
- Sprint
- Sprint Planning
- Daily Meeting
- Sprint Review
- Sprint Retrospective



- *And more such as Release Planning, Product Backlog Grooming, Scrum of Scrum, ...*

Events

- All events are **time-boxed** in order to ensure that appropriate amount of time is spent without allowing waste in the process
- All events are to **enable transparency** and a change for **inspection** and **adaptation**.



Sprint

- A sprint is a constant duration identified for delivering a product increment.
- Sprints are typically **between 1 and 4 weeks** length.
- For a project, **all sprints have the same duration**.
- Sprints have a **start date and an end date**.
- A sprint **cannot be closed early unless it is canceled** or is the last sprint for the product.

Sprint (cont.)

- Only the product owner can decide whether a sprint can be canceled when it is identified that the sprint goal is obsolete.
- Sprint occurs one after another without any down-time between them
- Each sprint is started by a planning meeting and ended by a sprint review an retrospective meeting

Sprint (cont.) - Abnormal Termination of Sprint

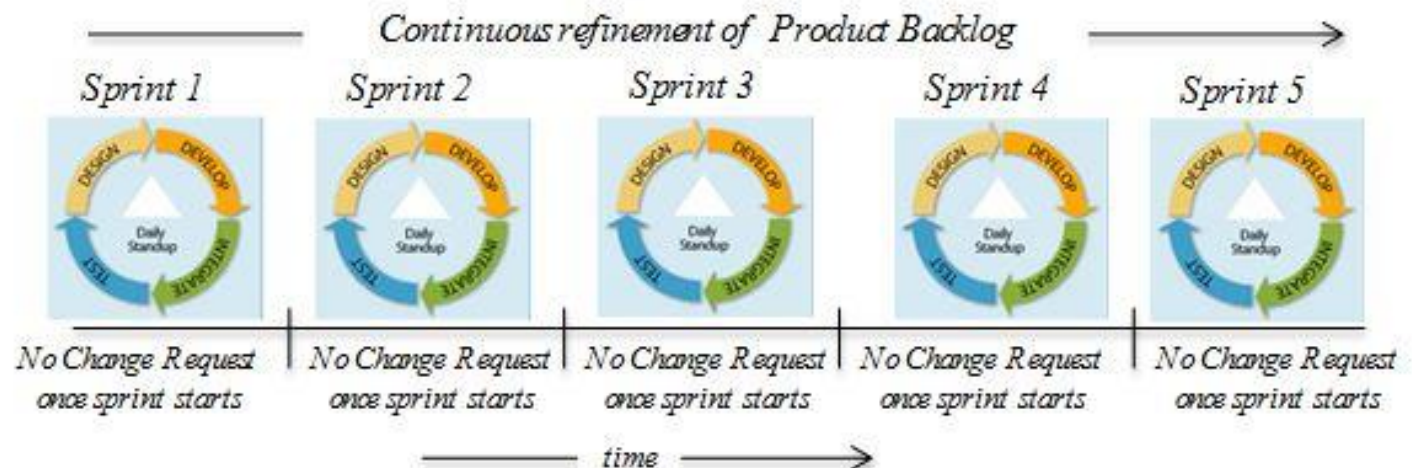
Sprints can be **cancelled** before the allotted **timebox** is

Management can cancel sprint if **external circumstances** **negate** the value of the **Sprint goal**

If a sprint is **abnormally terminated**, the next step is to **conduct a new sprint planning meeting**, where the reason for the termination is reviewed

Sprint (cont.) - No Change in Sprint

- No change keeps the team commitment
- During the sprint, what the team committed to deliver does not change, and the end-date of Sprint does not change.
- If something major comes up, Product Owner can terminate the Sprint prematurely, and start a new one



Sprint Planning

- Is a negotiation between the team and the product owner about what the team will do during the next sprint. ➔ make sure that the commitment is reasonable
- The product owner and all team members agree on **a set of sprint goals**, which is used to determine which product backlog items to commit from the uncommitted backlog to the sprint.
- No one should bring pressure on the team to over-commit;
- **Time-boxed: 8 hours**



Sprint Planning (cont.)

- Normally, will be **divided into 2 parts**:
 - **Part 1**: to decide what will be done in next sprint
 - **Input**: the latest product Increment, projected capacity of the Development Team during the Sprint, and past performance of the Development Team
 - **Output**: Sprint Goal and PBIs in sprint.

Sprint Planning (cont.)

- **Part 2:** to decide how will the chosen work get done?
 - **Enough work** is planned during Sprint Planning for the Development Team **to forecast what it believes** it can do in the upcoming Sprint
 - **Work planned** for the first days of the Sprint by the Development Team is decomposed by the end of this meeting
 - If the Development Team **determines it has too much or too little work**, it may renegotiate the selected Product Backlog items with the Product Owner

Daily Scrum (Standup Meeting)

- Daily Scrum is a meeting where the team has a short discussion to update each other progress and surface blocks.
- During the meeting, each team member answers three questions:
 - What have you done since **yesterday**?
 - What are you planning to do **today**?
 - Any **impediments**/stumbling blocks?
- Scrum Master notes the block and afterwards helps to resolve them.
- **Even without Scrum Master, the daily meeting still happens; Team (Required), SM (O) and ~~PO (O)~~**
- **Time-boxed: 15'**

Sprint Review

- At the end of Sprint, Product Owner, Scrum Master and stakeholders come together and see the demo of what team has produced.
- An informal meeting, not a status meeting to elicit feedback and foster collaboration
- The Product Owner gathers feedbacks from everyone on ways to improve what has been built.
- Inspect the Increment and adapt the Product Backlog if needed.
- Result is a revised Product Backlog
- **Time-boxed:** 4 hours; Team, SM, PO (R); Stakeholder (O)

Sprint Retrospective

- Retrospectives and **feedback loops** are at the heart of any successful Agile/Scrum implementation.
- The Retrospective is a chance for the team to **act like a team, hearing every voice, integrating their perspective and reaching consensus on how to move forward** in the next iteration.
- This is a mechanism for **continuous improvement**, and also where critical problems are identified and addressed or surfaced to management for assistance.
- Time-boxed: 3 hours, Team (R), SM(R) and PO (O)

SCRUM TEAM



Team



Product Owner



ScrumMaster



Stakeholders

CORE MEETINGS

1. Sprint planning
a. What
b. How
2. Daily Scrum
3. Sprint review
4. Sprint retrospective

Happens at regular intervals

ONE SPRINT: REPEAT N TIMES

CORE ARTIFACTS

1. Product backlog
2. Sprint backlog
3. Burn-down/burn-up charts

THE SPRINT

THE 3 QUESTIONS ARE ANSWERED
2 DAILY SCRUM



THE TEAM
The team completes the tasks in the sprint backlog while consulting with the PO and stakeholders on requirements and "doneness"

THE SCRUMMASTER

The SM facilitates the Scrum process as a servant-leader, radiates information to the stakeholders, and clears roadblocks out of the team's way

TRACK PROGRESS

With burn-down and/or burn-up charts

2-4 WEEKS

3 SPRINT REVIEW

INSPECT AND ADAPT THE PRODUCT
The TM's demo the new product increment to the PO and stakeholders; everyone gives feedback and discusses new ideas, PO makes necessary changes to the product backlog

4 SPRINT RETROSPECTIVE

INSPECT AND ADAPT THE PROCESS
The TM's evaluate their process and how well they work together; teams adapt to enable high performance

1 SPRINT PLANNING

"WHAT" MEETING



The PO presents the highest priority product backlog items and the TM's determine what they can accomplish and commit to in the sprint

TO DO	IN PROGRESS	READY FOR REVIEW	DONE
☐	☐	☐	☐
☐	☐	☐	☐
☐	☐	☐	☐
☐	☐	☐	☐

1b "HOW" MEETING

The TM's create the sprint backlog by decomposing the product backlog items they committed to into tasks and estimating the tasks

IT'S "DONE"

Potentially shippable product increment

HIGH PRIORITIES

PRODUCT BACKLOG

The PO works with the appropriate individuals to seed, prioritize, and groom the product backlog

LOW PRIORITIES

ONE SPRINT: REPEAT N TIMES



**Thank you for
your attention!
& See you in 3.
Advanced
SCRUM**

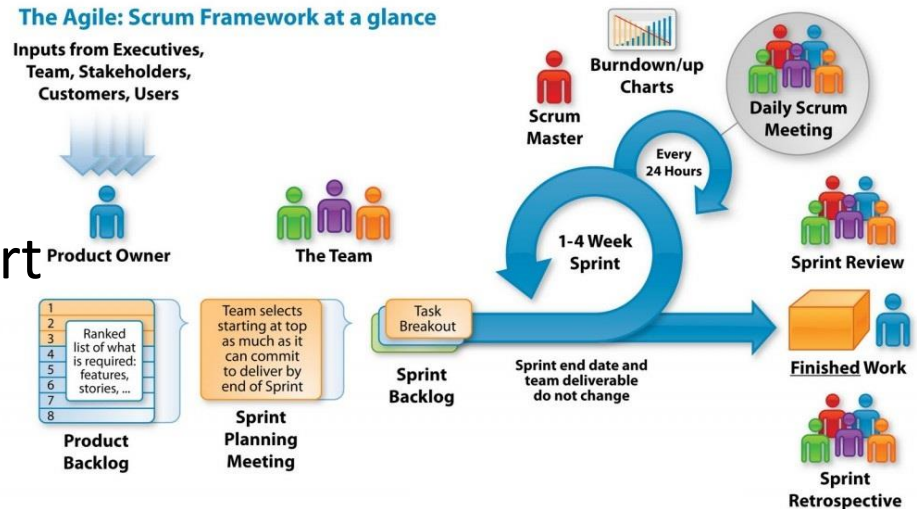
3: ADVANCED SCRUM ARTIFACTS, EVENTS And More

Content

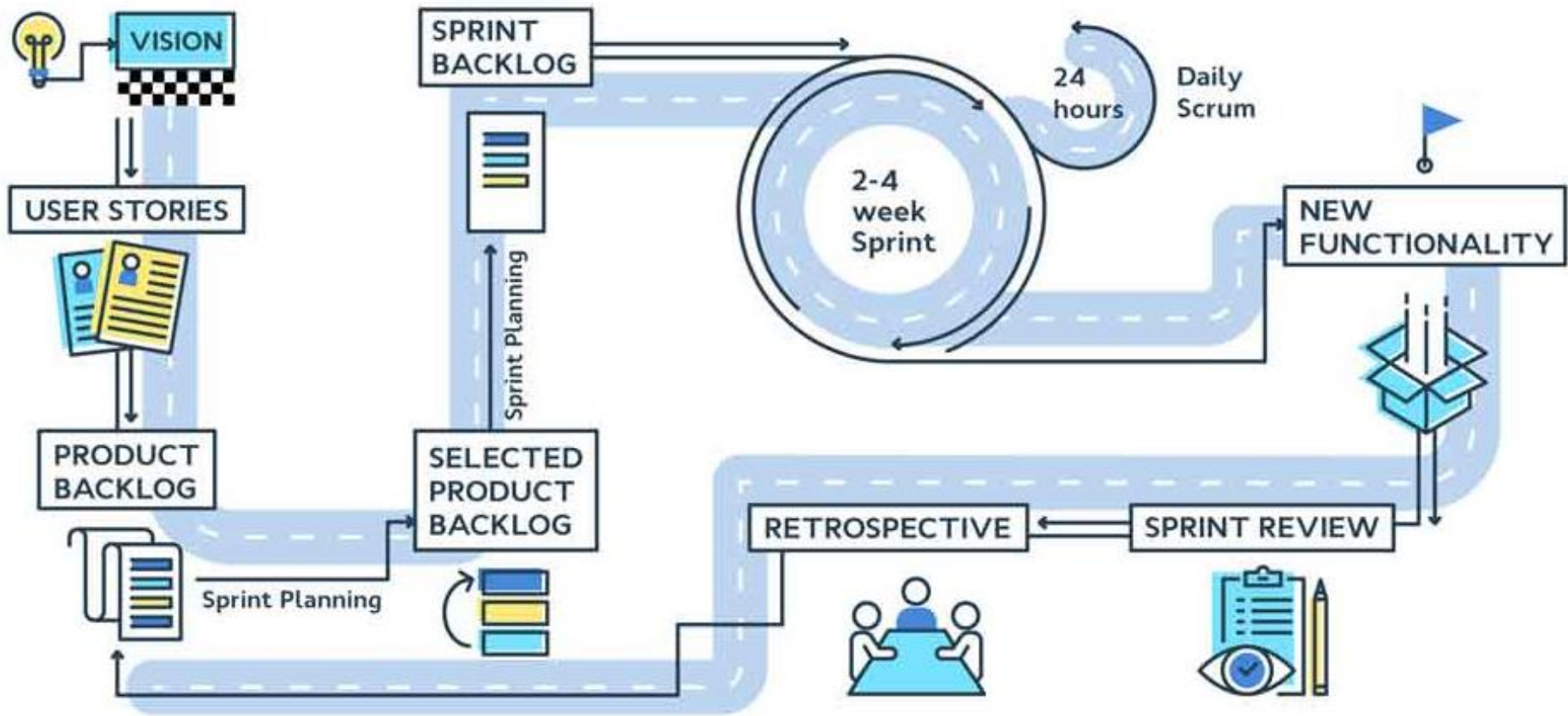
- Artifacts
 - Product Backlog
 - Sprint Backlog
 - Product Backlog Item (User Story)
 - INVEST
 - MOSCO
 - Definition of DONE
 - Definition of READY
 - Velocity and Burndown Chart

- Detailed Scrum Events

- Sprint Planning
 - Kanban board
 - Impediment Management (Risk Management)
- Daily Scrum
- Sprint Review
- Sprint Retrospective
- Product Backlog Grooming



Scrum Framework (remind)



As a PO, I need to understand the
Product Backlog so I can build a
better product and deliver the most
value quickly

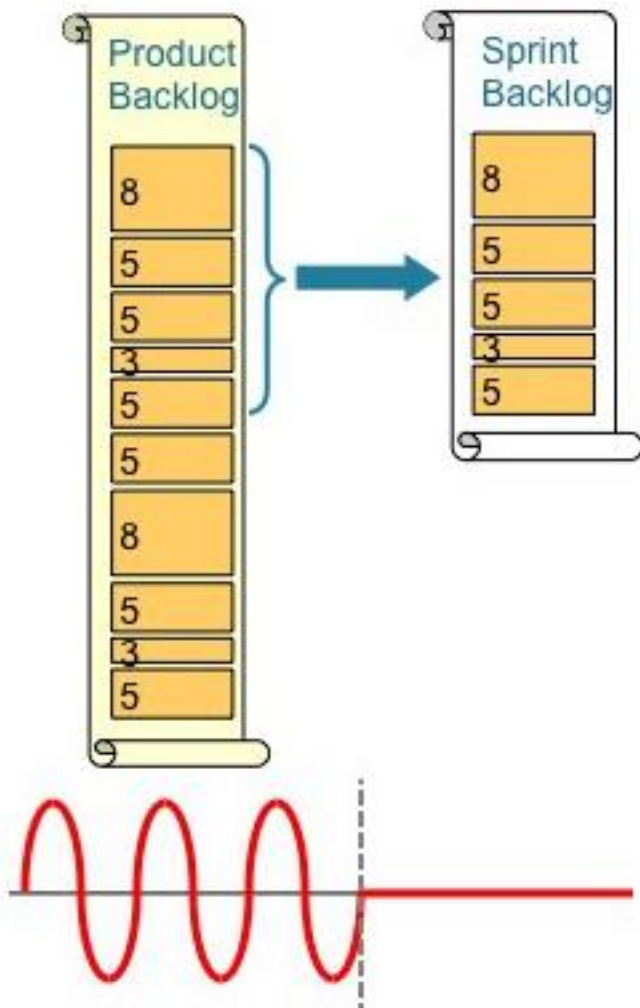
The Product Backlog

- An ordered list of everything that might be needed in the product
 - Composed of product “features” that describe the product
 - THE single source of requirements for any changes to be made to the product and focus of team discussions
- The Product Backlog is shared across teams working on the same product to drive coordination
- There is only one Chief Product Owner who ultimately owns the product backlog
 - For scaling we want one enterprise backlog which may have multiple products
- Ordered to provide maximum return on investment, accounting for:
 - Market demand
 - Team capability
 - Risk



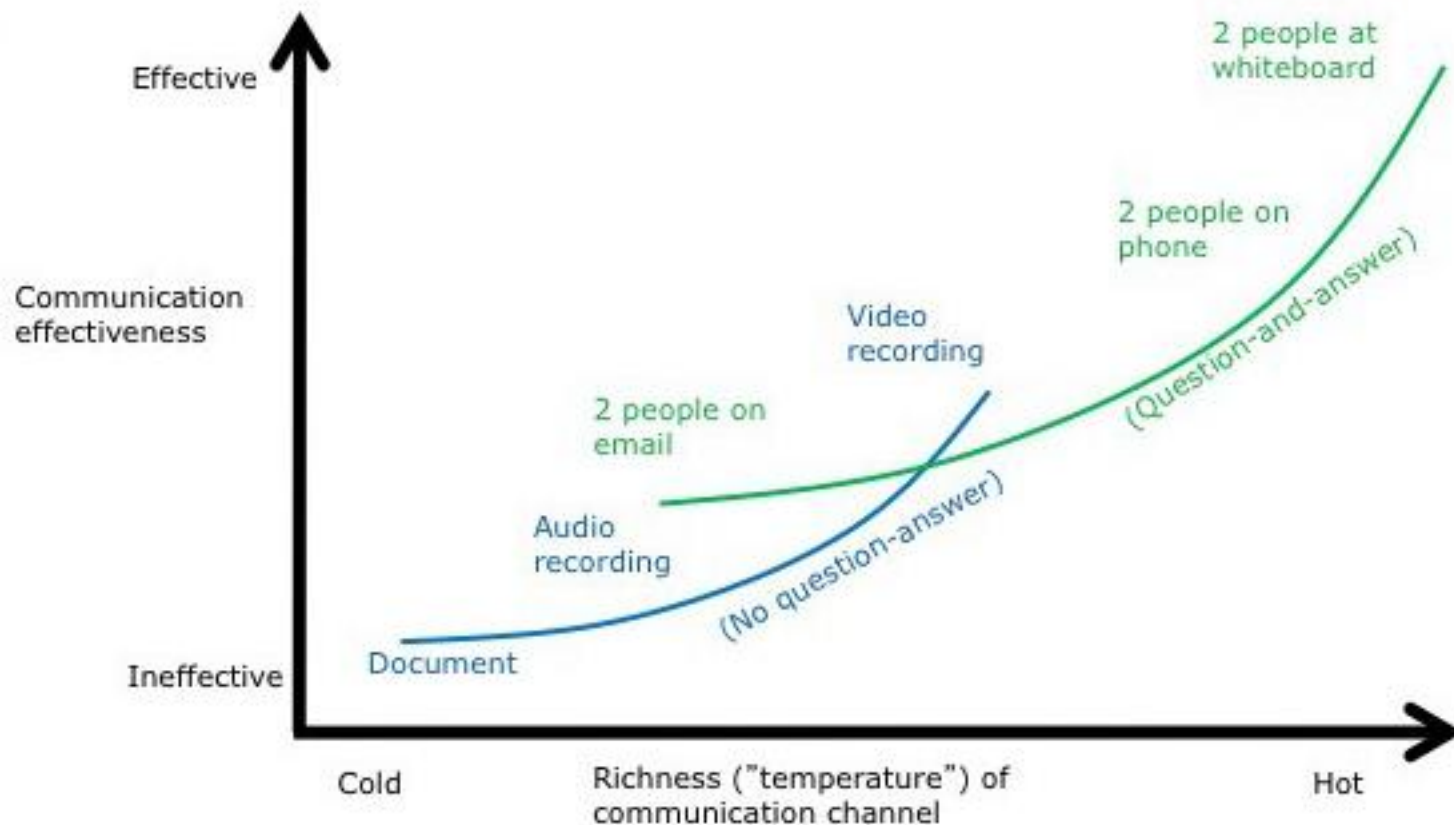
PB a Working Vision of the Product

Unlike the Sprint Backlog, it can change at any time



- Any stakeholder can add anything to the Product Backlog
- Product Owner orders the backlog
- Backlog items that are lower priority can be defined more roughly
- The PO and team should spend time each sprint to “groom” the backlog
 - Build in lessons learned from earlier work
 - Refine feature definitions so they are “ready” when the time comes

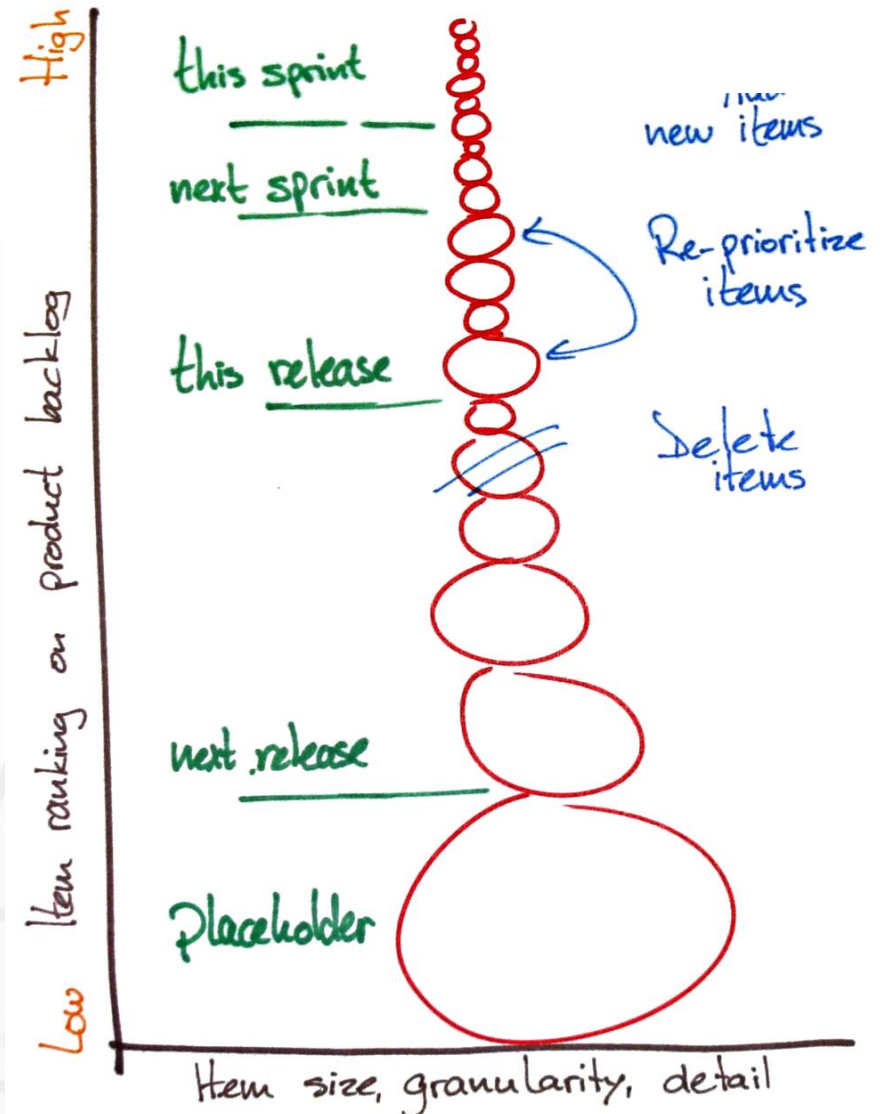
Product Backlog Improves Communication Effectiveness



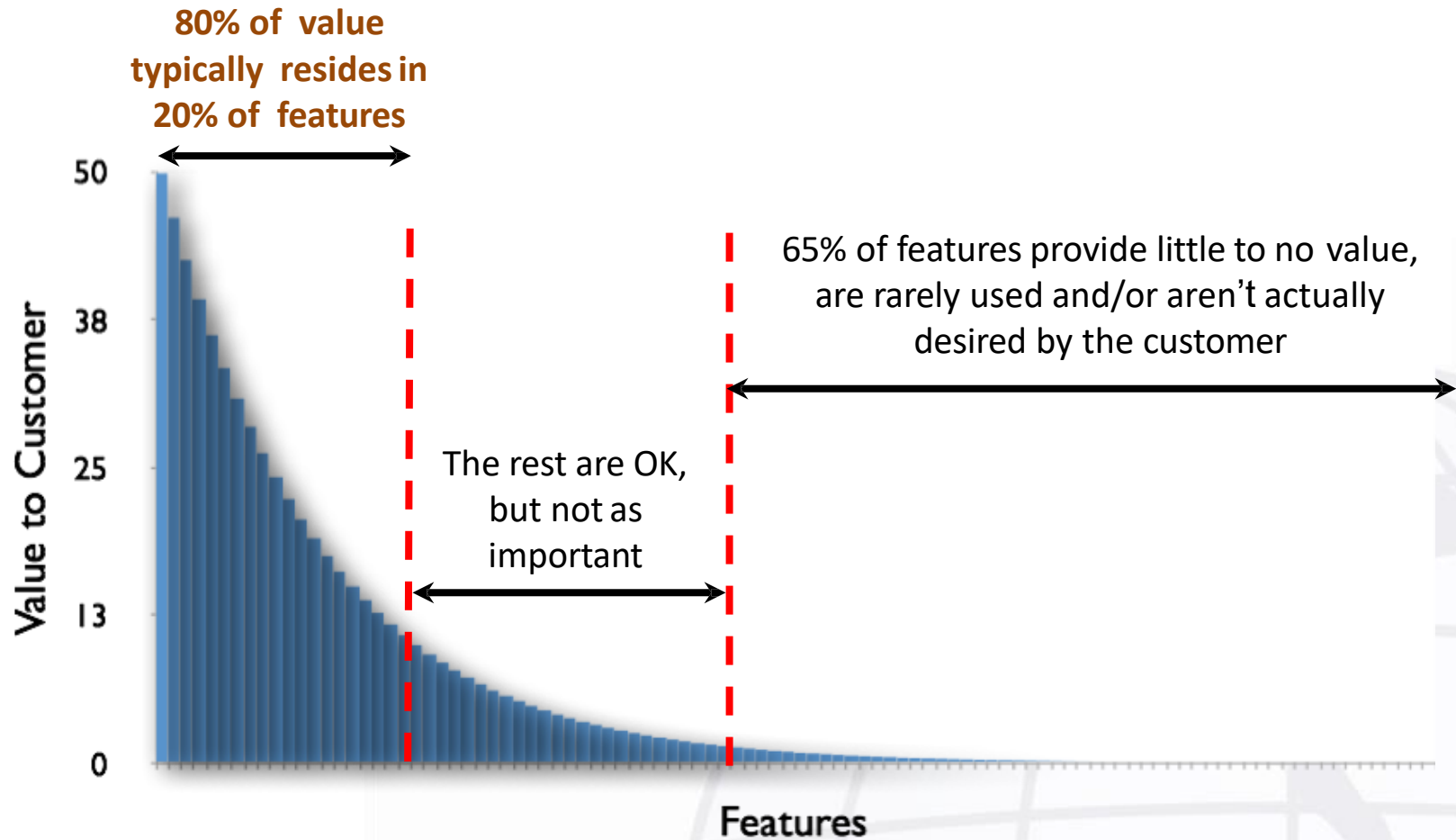
Source: research from McCarthy and Monk (1994)

The Product Backlog

- Items **vary** in size and detail
- Is always in **ranked** order
- **Priority** items are detailed in time for the **next Sprint Planning Meeting**
- Evolves over time and Is never done
- **Is always ready**

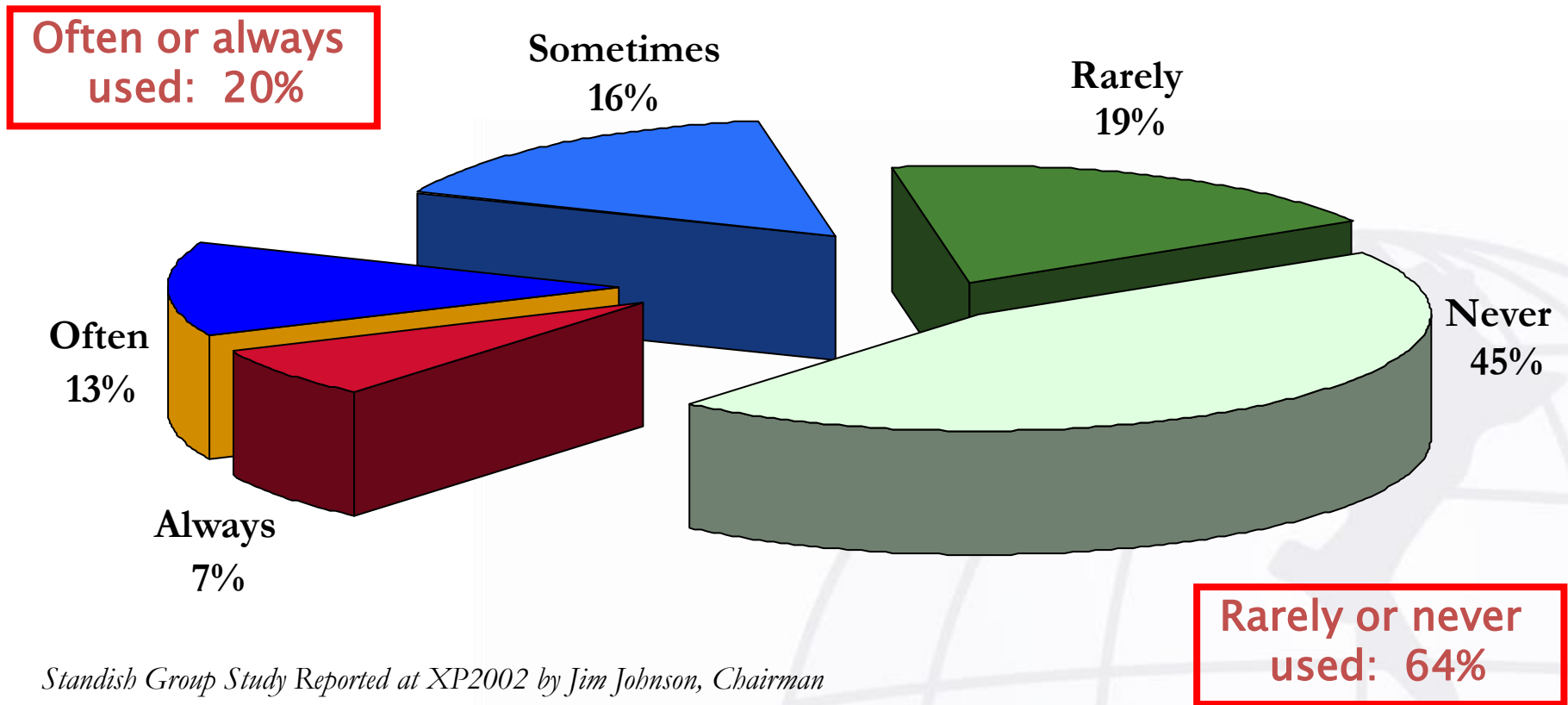


Not All Features Are Created Equal!



Why We Prioritize the Product Backlog

Delivered product function usage in a typical system



Standish Group Study Reported at XP2002 by Jim Johnson, Chairman

© 2004 Poppendieck LLC

Product Backlog Items

- aka PBI's
- Issues, features, user stories, use cases, functions, requirements, specifications, fixes
- Are in the customer space: what, not how
- Can start out very rough but evolve into enabling specifications if needed
- Each potentially shippable increment includes one or more PBIs

Specification Issues (cost 10 hrs/defect)

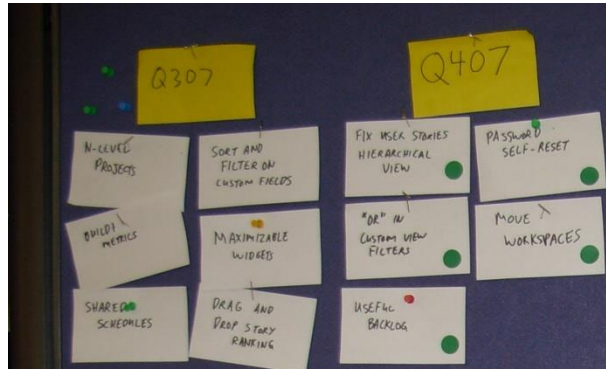
- Clear enough to test
- Unambiguous to intended readers
- No design options



Product Backlog Item

- Attributes
 - Description (**Spec in SRS**)
 - Rank/Priority
 - Complexity/Cost/Size (Story Points or T-Shirt)
 - Optional attributes:
 - **Business Value (\$, L/M/H),**
 - Owner,
 - Tests, Sample results
 - **Other options?**
- Can be defined via a “**Story Card**”, “**User Story**”, “**Use Case**”, what else?

Example: Product Backlog Item as a Story Card



Story cards

Acceptance Criteria & Tests

1. _____
2. u_____
3. -_____
4. u_____

Title _____

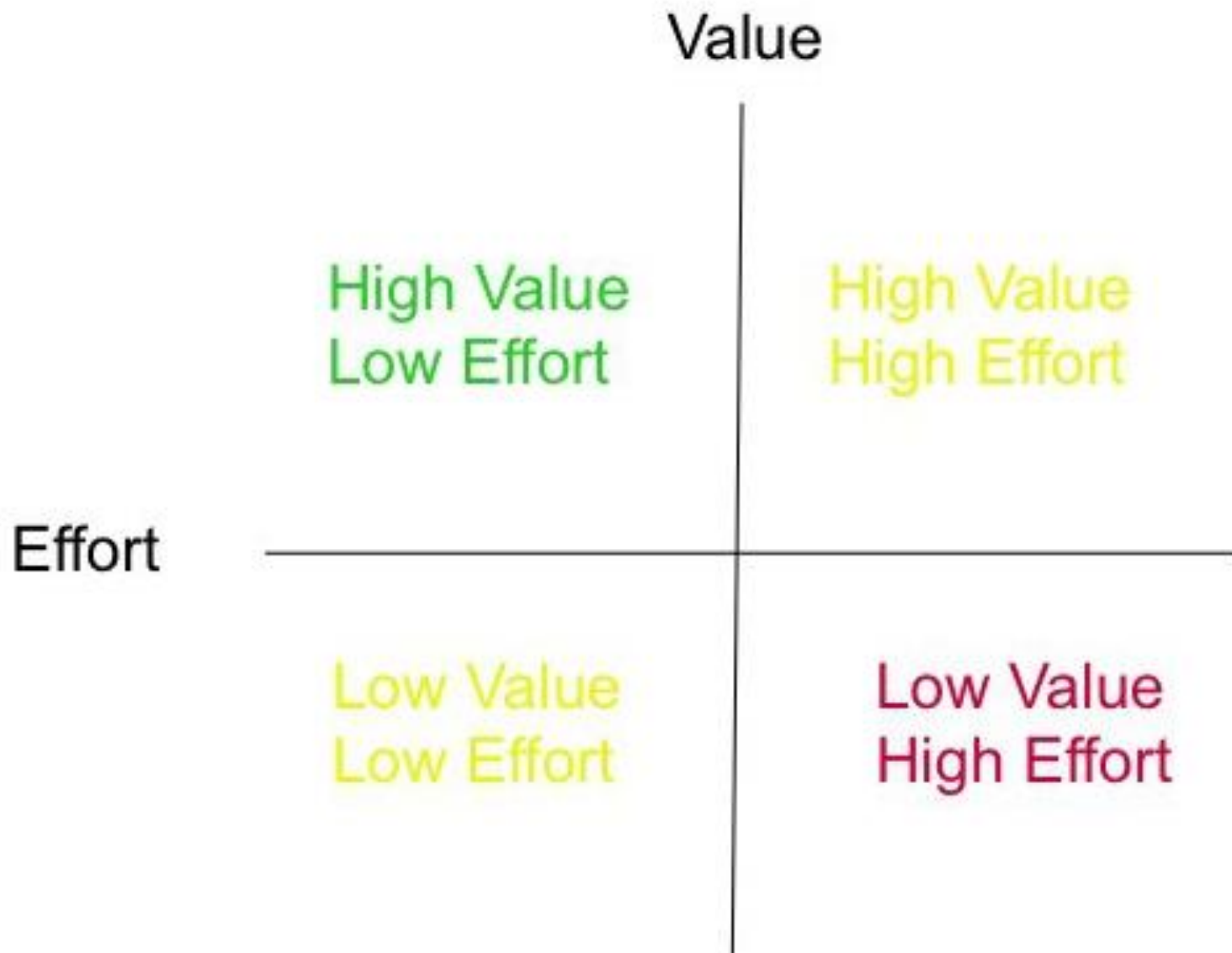
As a <roles>
I want to <actions>
so that <benefits>

Priority	Estimate
----------	----------

Steps to Creating a Product Backlog

1. What are high level stories? (epics)
2. What are epic sub-stories?
3. Which epics are most important?
 - MOSCOW, Kano, ROI, NPV, NPV/point
4. Prioritize the product personas
5. Walk the personas through the stories
6. This yields a single ordered list - the Product Backlog
7. Get the top of the Product Backlog READY for the first sprint

What Do We Build First?

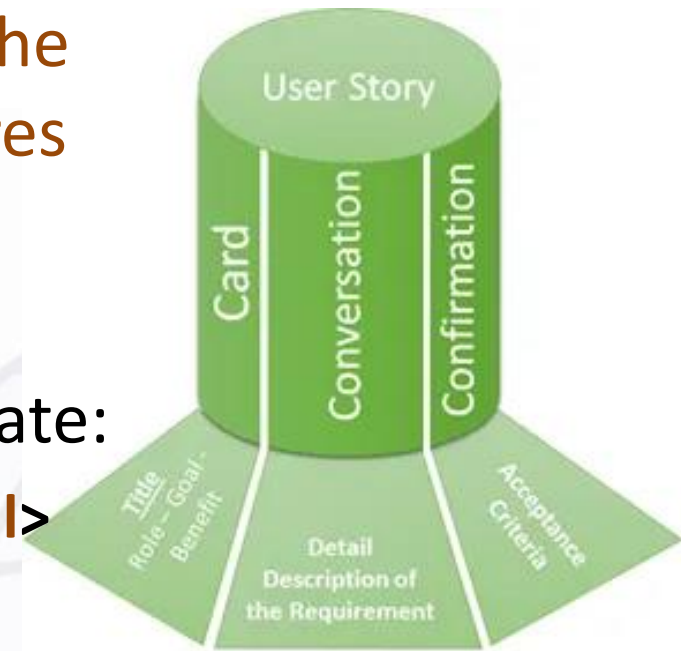


Simple Ordering of Product Backlog

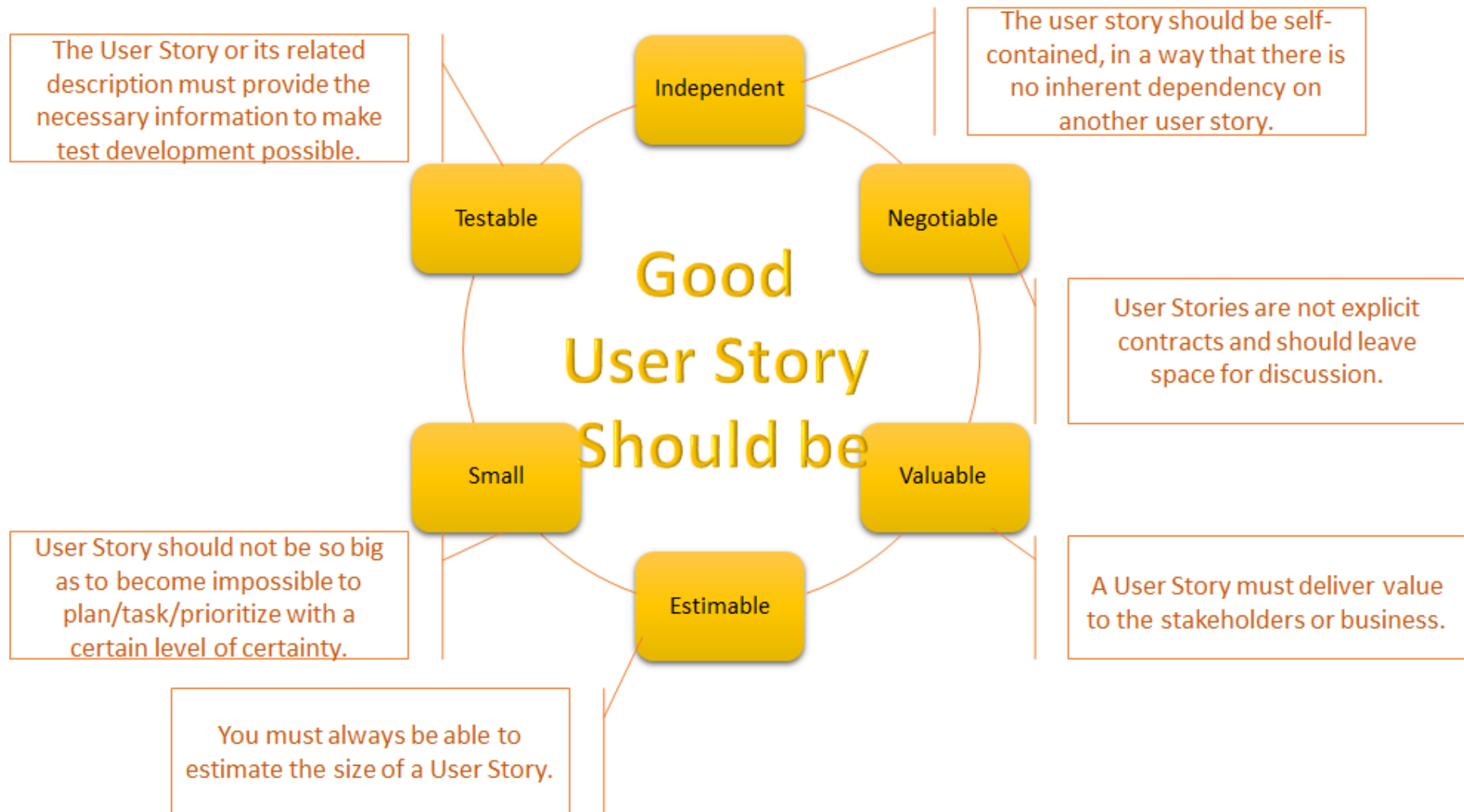
- Point Strategy
 - Allocate 1,000 arbitrary units of value among the Product Backlog
 - Sort by Value
 - Manually sort backlog items to fine tune
- Bubblesort Strategy
 - Take first two items – which is more important.
 - Take second and third – which is more important
 - Keep doing it until sort is complete
- Low priority first strategy
 - Assume project does not complete one item – which item is given up
 - Assume another is not complete – which one is given up
 - Keep doing this and back into a forced ranking

PBI as User Story

- **User stories** are short, simple descriptions of a feature told from the perspective of the person who desires the new capability, usually a user or customer of the system.
- They typically follow a simple template:
 - As a **<type of user>**, I want **<some goal>** so that **<some reason>**.
- A story should use the I.N.V.E.S.T acronym



I.N.V.E.S.T



PBI as Theme, Epic and Task

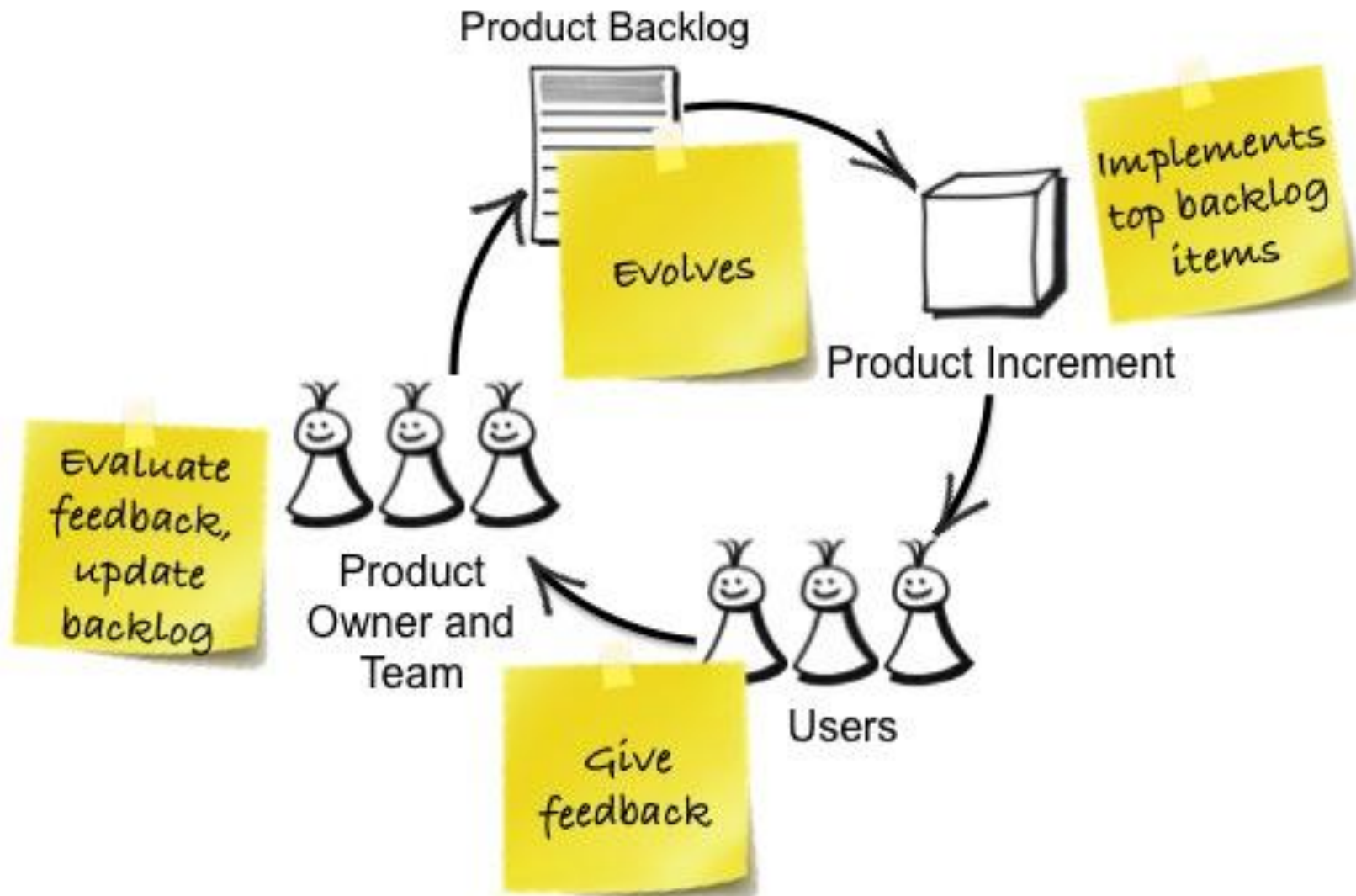
- **Epic: large story**, generally take more than one or two sprints to develop and test. They are usually broad in scope, short on details, and will commonly **need to be split into multiple, smaller stories** before the team can work on them.
- **Theme** is a collection of **related user stories**.
- **Task** is simply more granular versions of the work entailed to complete a PBI

Product Backlog Grooming

Activity in during the Sprint

- Look ahead to Product Backlog that's coming soon
- Split large Product Backlog items into smaller ones
- Start to get more detailed understanding of items
- Begin to think about how they'll be implemented
- PO, Team and SM do this together each Sprint, at a time of their choosing, and for a duration of their choosing Set a regular date and time to do this every Sprint
- Initially provide about 10% of the time to this activity and then Inspect and Adapt

Product Backlog Grooming (cont.)



Product Backlog Grooming (cont.)

- An **on-going process** in which the Product Owner and the Development Team collaborate on the details of Product Backlog items.
 - Removing stories that no longer appear relevant
 - Create new user stories
 - Re-assess the relative priority of stories
 - Request for the most detailed product backlog items (PBI)
 - Add acceptance criteria and define when this item is done
 - Light-estimate PBIs
 - Only focused on higher ordered PBIs
 - **[DOR] A GOOD “READY” PBI (user story) is I.N.V.E.S.T**

Product Backlog Grooming (cont.)

PBI Prioritization with MOSCO Technique

M Must have requirement

S Should have if at all possible

C Could have but not critical

W Would be good to have... *(Won't have the time to do it now, but maybe later)*

As a PO, the Team and I need clear definitions of **READY and DONE** so the Team can complete the backlog quickly and effectively



READY Stories that can reach DONE are Key to Team Success



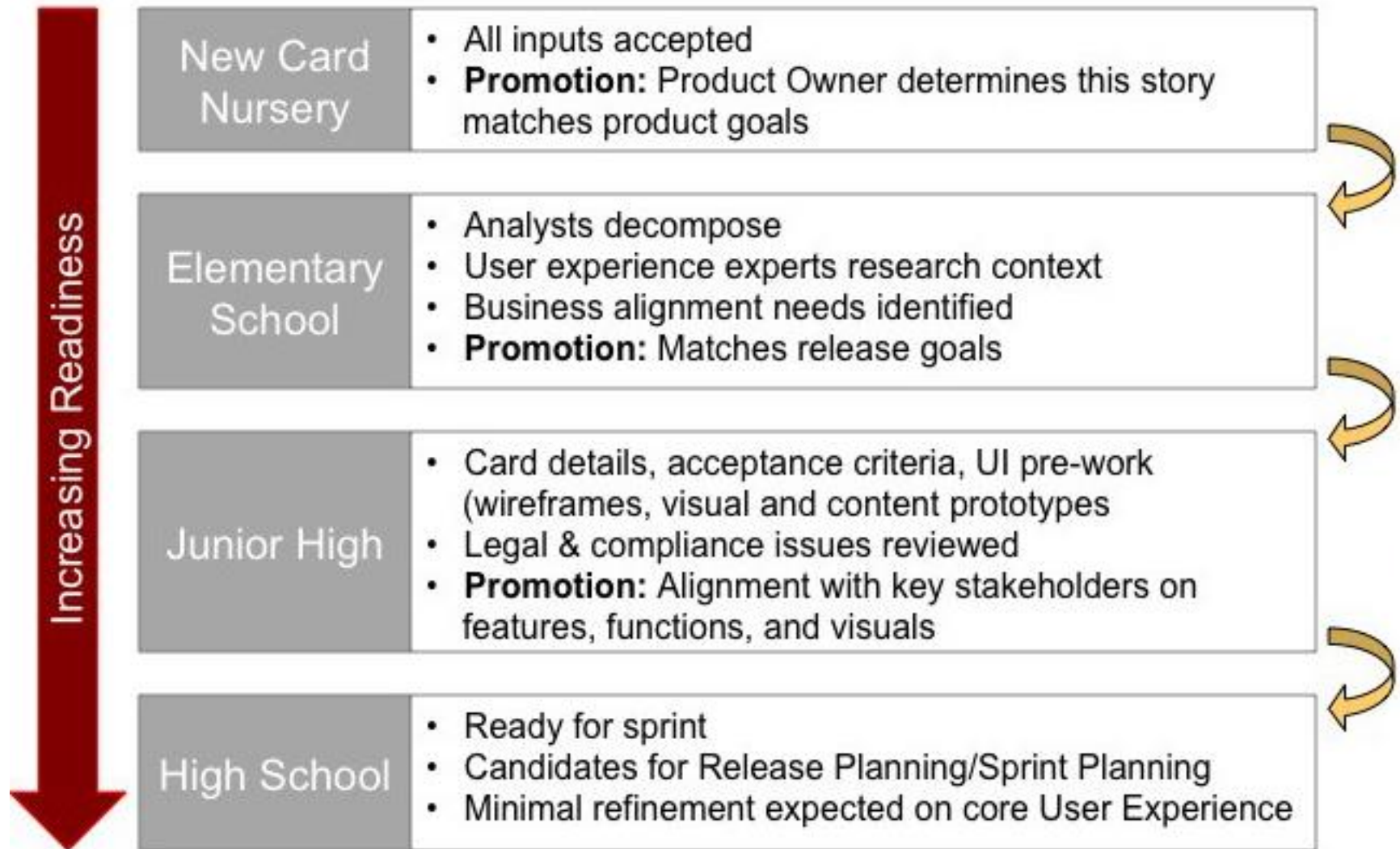
What does it mean to be **READY**?

1. Defined clearly enough that all members of the team understand what must be done
 - Includes team-developed tasking, if needed
 - Assume some ongoing discussion to refine, coordinate and clarify
2. Includes clear statement of resulting business value that allows the Product Owner to prioritize
3. Includes any required enabling specs, wire frames, etc.
4. Fully meet INVEST criteria for user stories
 - Estimated and sized to complete easily within one sprint
5. Free from external dependencies
 - I.e. there is nothing beyond the team's control that must be done first in order to complete the story

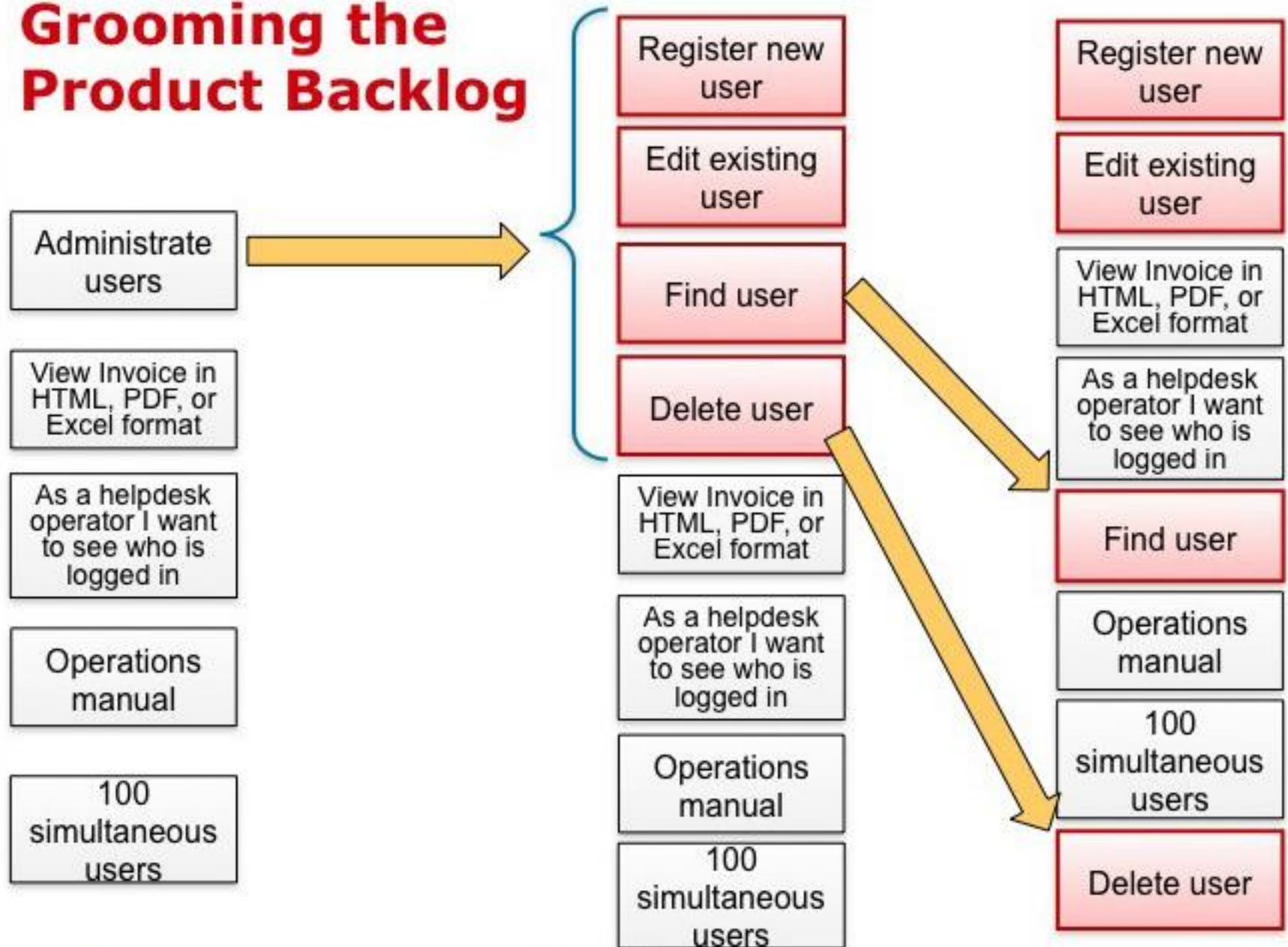
Moving toward **READY**

- The Team regularly estimates items as part of “Grooming” the backlog
 - Expect to spend ~10% of team time
 - Often once a week at "The Wednesday Afternoon Meeting"
 - Update at the Sprint planning meeting
- The team needs Product Owner support to clarify PBIs
- Product Owner supports and informs the team, but may direct them only through the product backlog
- The Scrum Master should enforce this!

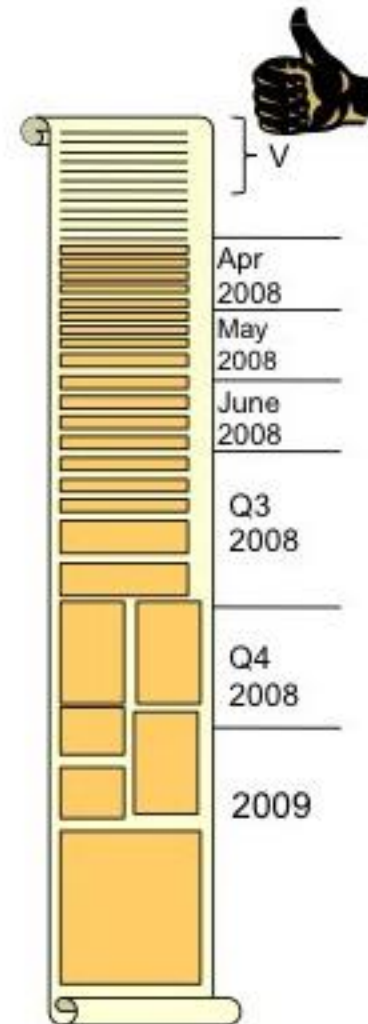
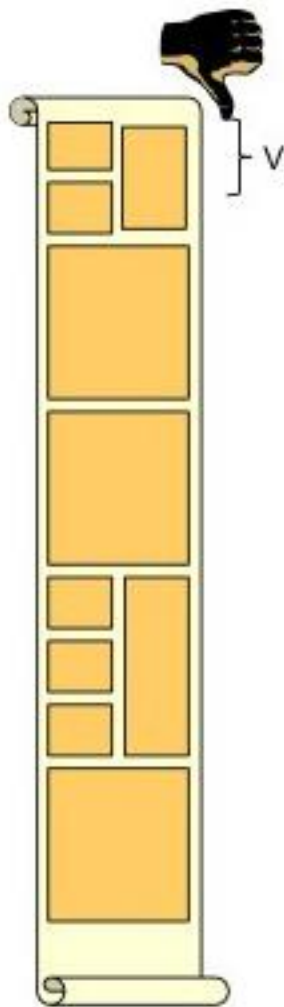
User Story Readiness Progression



Grooming the Product Backlog



Team Works to Maintain the Right Progression of Backlog Definition



What Does it Mean to be DONE?

1. "Definition of Done" (DoD) decided on beforehand – along with acceptance tests
 - DoD can be standard across a group of common stories, or defined specifically for unique ones
2. Done means the feature has been developed, tested AND meets all required acceptance tests
3. Ideally, Done means the feature could be shipped to a customer
4. Product Owner officially "Accepts" Done features back from Team at the Sprint Review meeting

Some Definitions of Done

Default Definition of Done

- **Releasable**

Default Definition of Done

- **Unit/Integration tested**
- **Ready for acceptance test**
- **Deployed on demo server**

Default Definition of Done

- **Acceptance tested**
- **Release notes written**
- **Releasable**
- **No increased technical debt**

= I haven't messed up
the codebase

What else must be done before shipping the code?

- For example "customer acceptance test + user documentation"

Why not? Who does it? When? What happens if a problem turns up?

Burn up this work in release burndown!

Some Definitions of Done

- DoD is **not static**
 - The DoD **changes over time.**
 - Organizational support and the team's ability to **remove impediments** may enable **the inclusion of additional activities into the DoD** for features or sprints.
 - Data from **retrospectives** are **added to DoD** as the **team learns** and **understands better**



7 TIPS FOR WRITING ACCEPTANCE CRITERIA

01

Each user story / PBI should have at least one acceptance criteria



02

Is identified before implementation



03

Is independently testable



04

Has a clear Pass / Fail result



05

Focuses on the end result: What
Not the solution: How



06

Includes functional and non-functional criteria



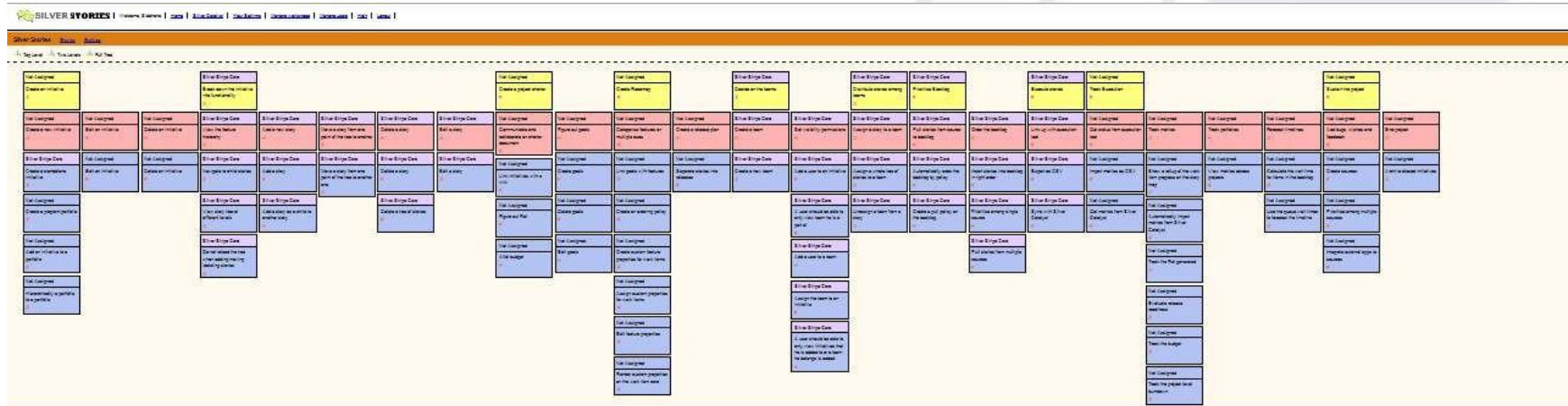
07

Team members may write it, Product Owner verifies it

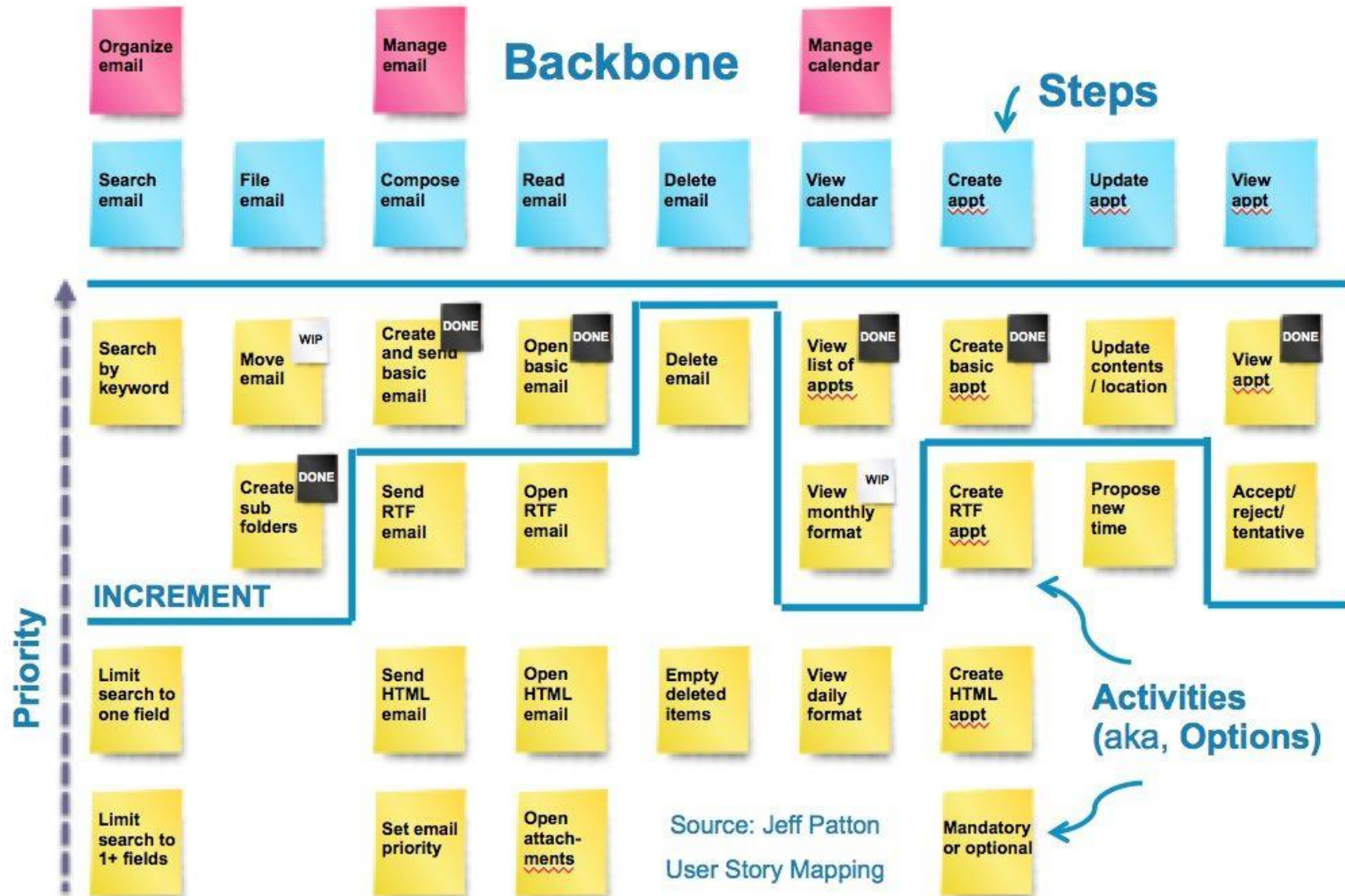


User Story Mapping

- Epics at top, stories underneath
- Shows workflow
- Can be large features, company initiatives
- Two dimension view easier to understand than linear ordering
- Tool for identifying MVP
- Allows the team to see the big picture



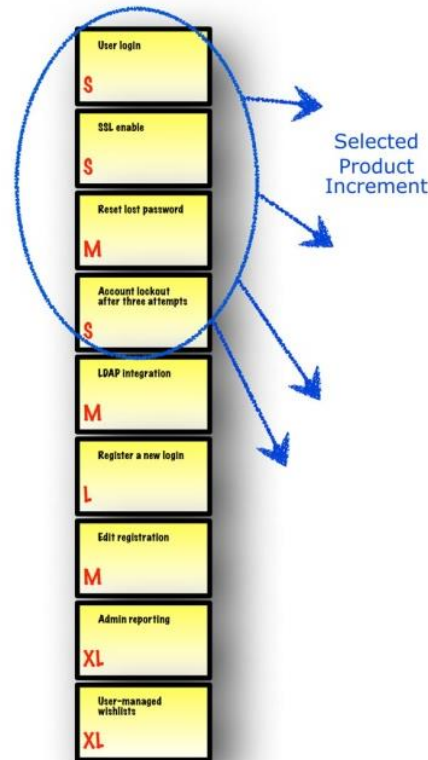
User Story Mapping (cont.)



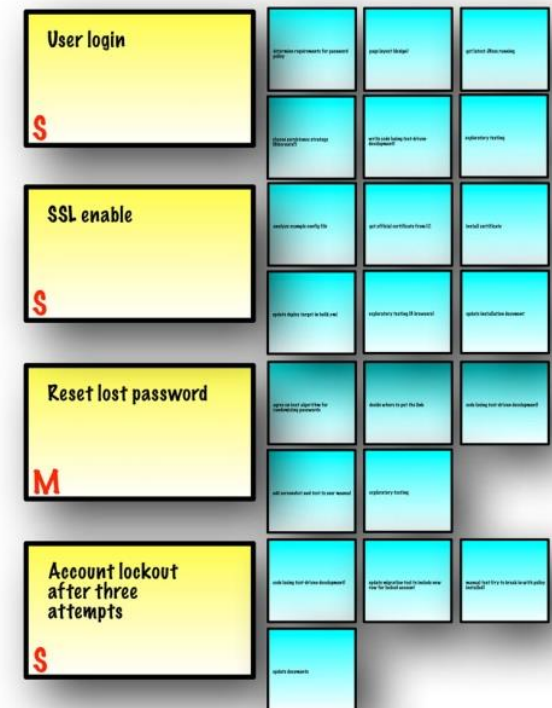
Sprint Backlog

- The **sprint backlog** is a **list of tasks** identified by the **Scrum team** to be completed during the **Scrum sprint**
- During the **sprint** planning meeting, the team selects some number of **product backlog** items, usually in the form of user stories, and identifies the tasks necessary to complete each user story.

Product Backlog



Sprint Backlog



Sprint Backlog (cont.)

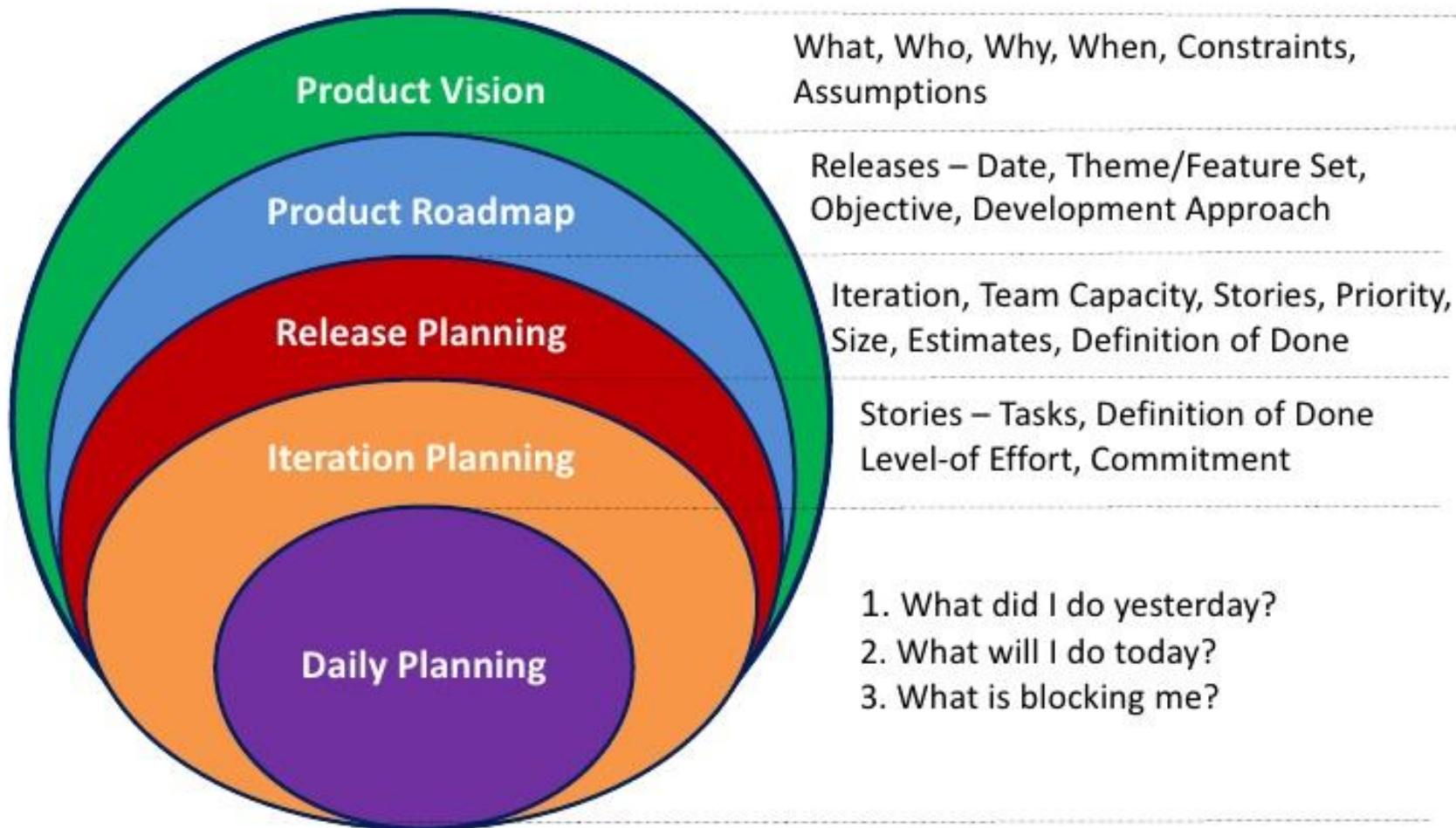
- Scrum team takes **the Sprint Goal** and decides what tasks are necessary to complete the selected features.
 - Tasks should be **no more than 16 hours** in length; larger tasks need additional breakdown
- Team **self-organizes** around how they'll meet the **Sprint Goal**.
- Scrum Masters **don't make decisions** for the team.
- Sprint Backlog (a list of tasks to be completed during the Sprint is created).



Sprint Planning



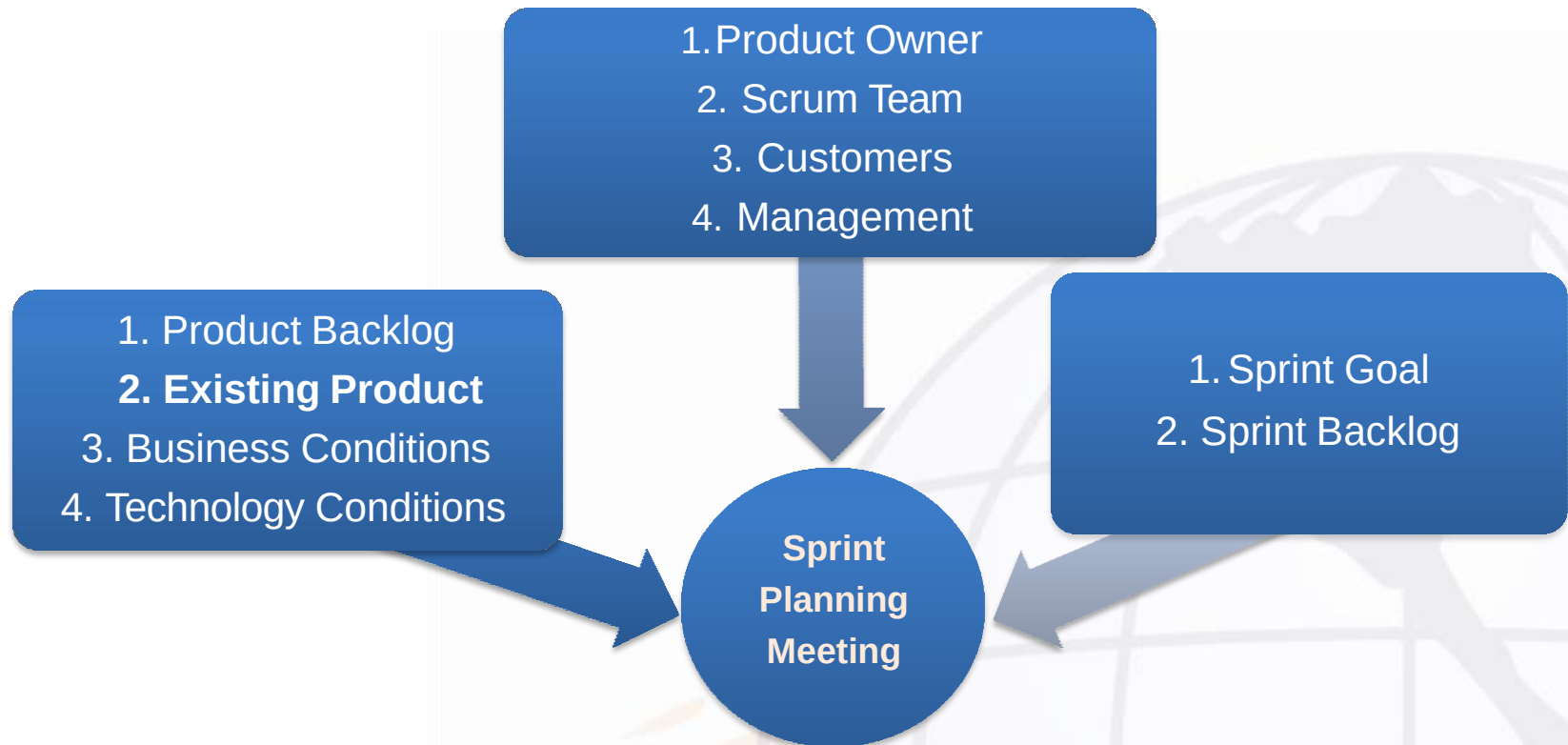
5 levels of Agile Planning



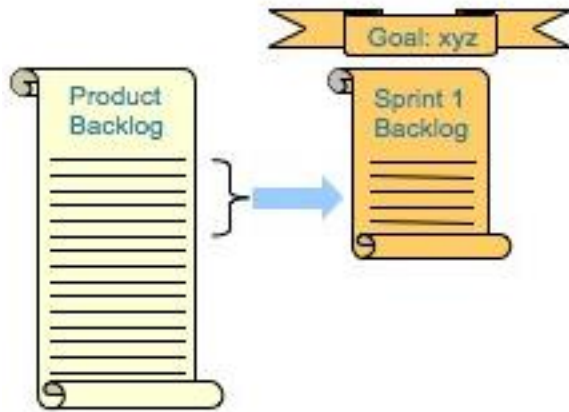
Sprint Planning

➤ Sprint Planning Meeting

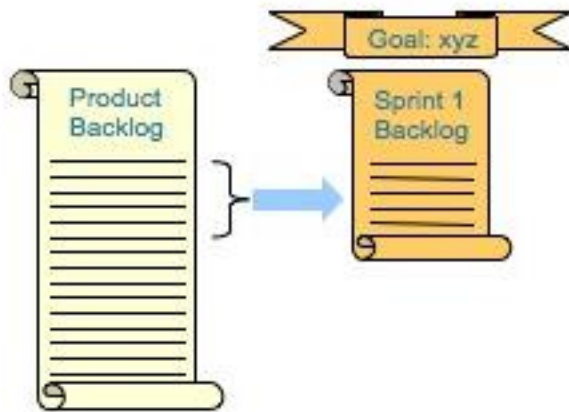
- Each sprint is preceded by a planning meeting, where the User Stories for the sprint are identified and an estimated commitment for **the sprint goal** is made



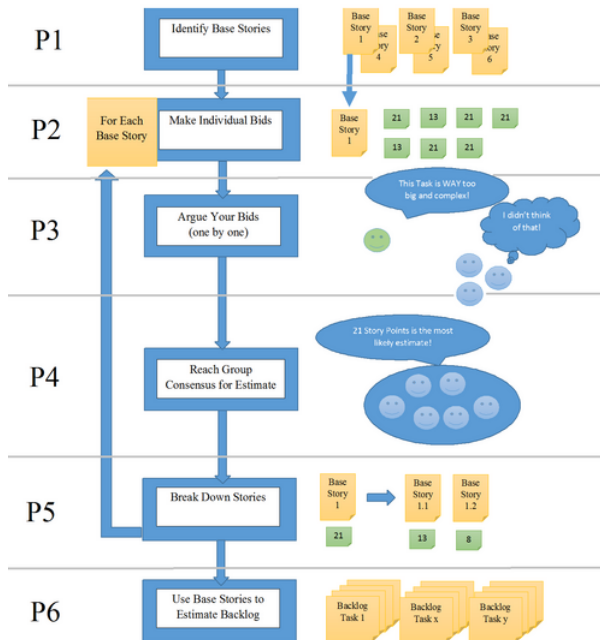
Sprint Planning (cont.) - INPUT



Sprint Planning (cont.) - OUTPUT



Six Phases of Story Point Estimation



Jackass team, sprint 15

Sprint goal

- Beta-ready release!

Sprint backlog

- Deposit (5)
 - Migration tool (13)
 - Backoffice login (3)
 - Backoffice user admin (5)
- (Estimated velocity = 26)

Schedule

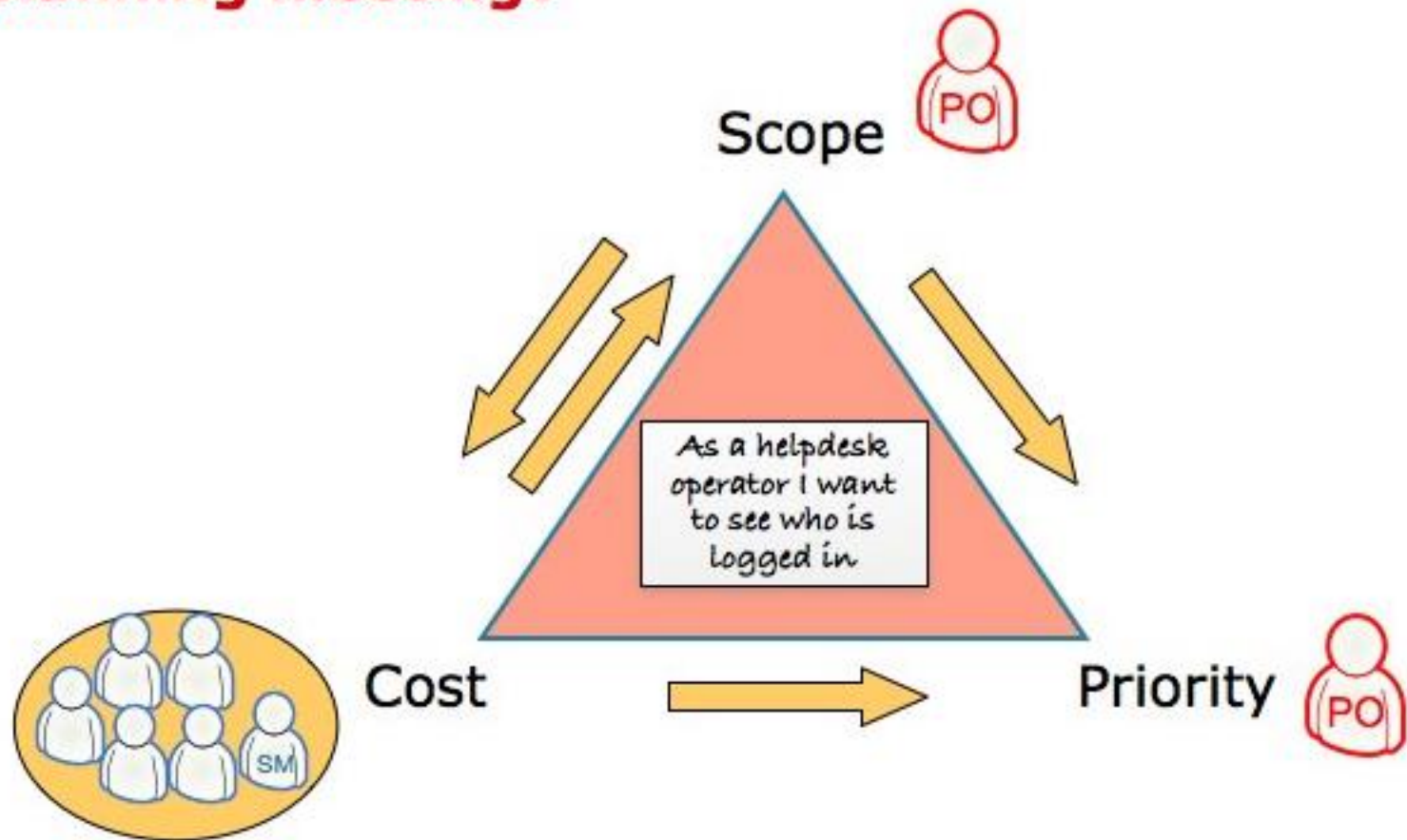
- Sprint period: 2006-11-06 to 2006-11-24
- Sprint demo: 2006-11-24, 13:00, in the cafeteria
- Daily scrum: 9:30 – 9:45, in conference room Jimbo

Team

- Jim
- Erica (scrum master)
- Tom (75%)
- Niklas
- Eva
- John

Sprint Planning (cont.)

Why is it important that the team AND product owner attend the sprint planning meeting?

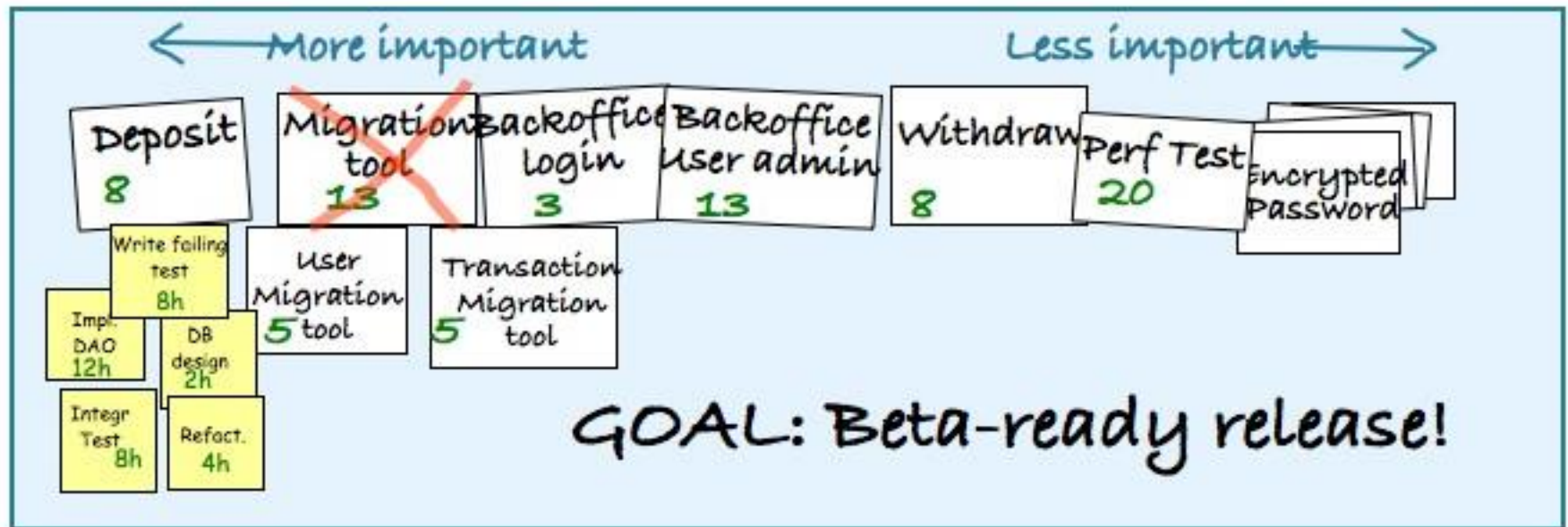


Sprint Planning Meeting - Example



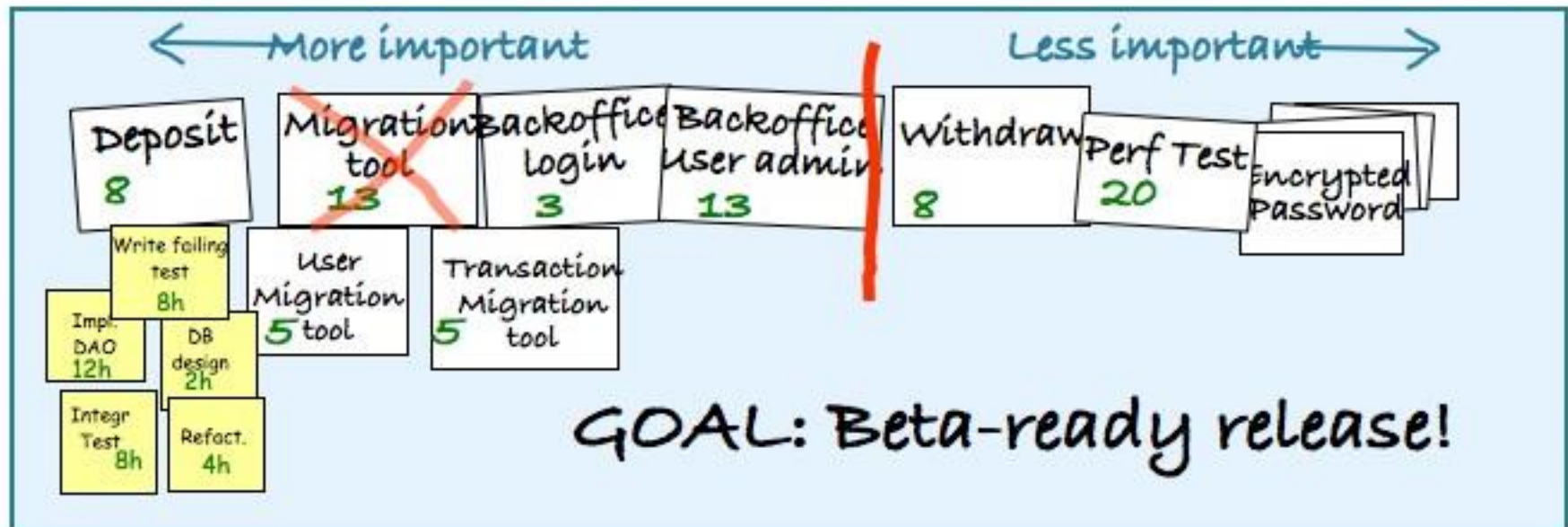
- Goal
- Present product backlog
- Reprioritize, re-estimate, split/combine stories
- Break out tasks
- Estimate velocity, draw the line

Sprint Planning Meeting - Example



- Goal
- Present product backlog
- Reprioritize, re-estimate, split/combine stories
- Break out tasks
- Estimate velocity, draw the line

Sprint Planning Meeting - Example



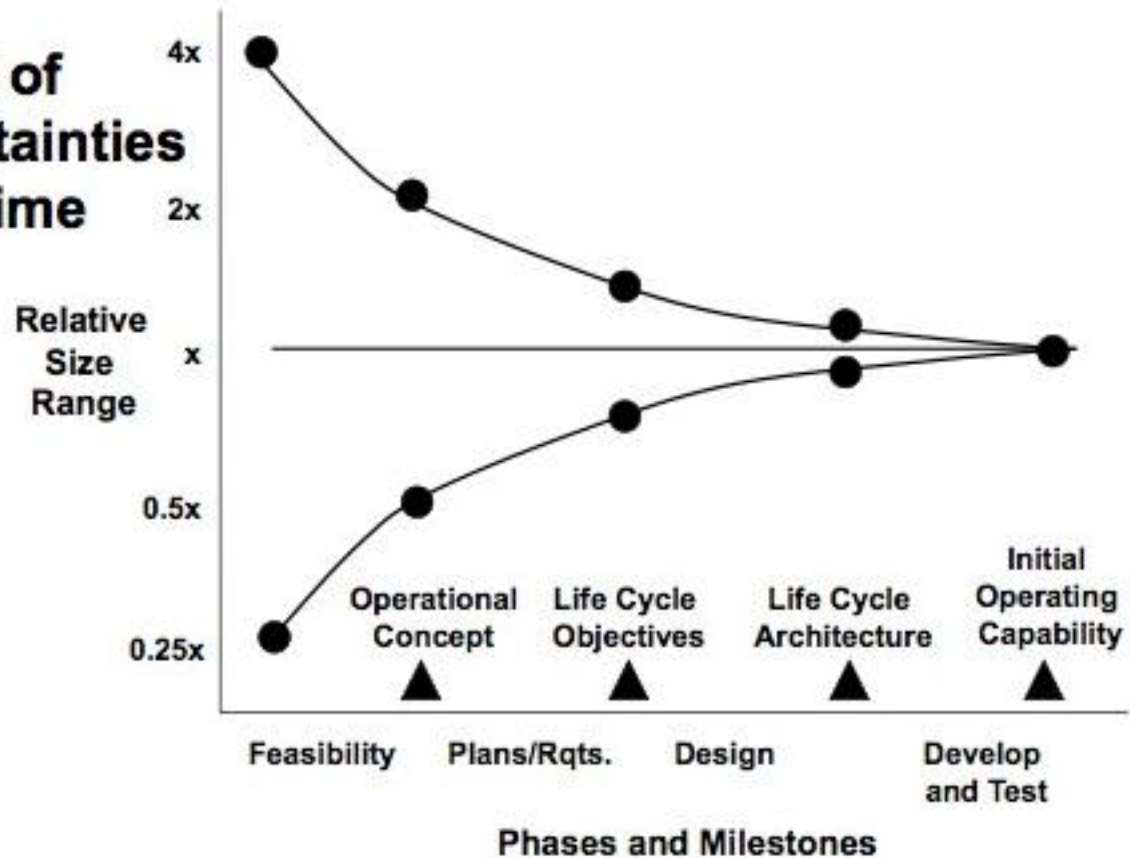
- Goal
- Present product backlog
- Reprioritize, re-estimate, split/combine stories
- Break out tasks
- Estimate velocity, draw the line

The Sprint Commitment

- Team's commitment to the Product Owner:
- "We promise that ..."
 - We believe we can reach the sprint goal.
 - We will do everything in our power to reach the goal and will inform you immediately if we have problems.
 - Code will be potentially shippable at the end of the sprint.
 - If we fall behind schedule we will remove the lowest priority stories first.
 - If we get ahead of schedule we will add stories from the product backlog in priority order.
 - We will display our progress and status on a daily basis.
 - Every story we do is complete.
- Caveat
 - Estimates are estimates. We will be early some times and late other times. We will document this normal variation with our sprint velocity.

Software Estimation Accuracy

- **Effect of uncertainties over time**

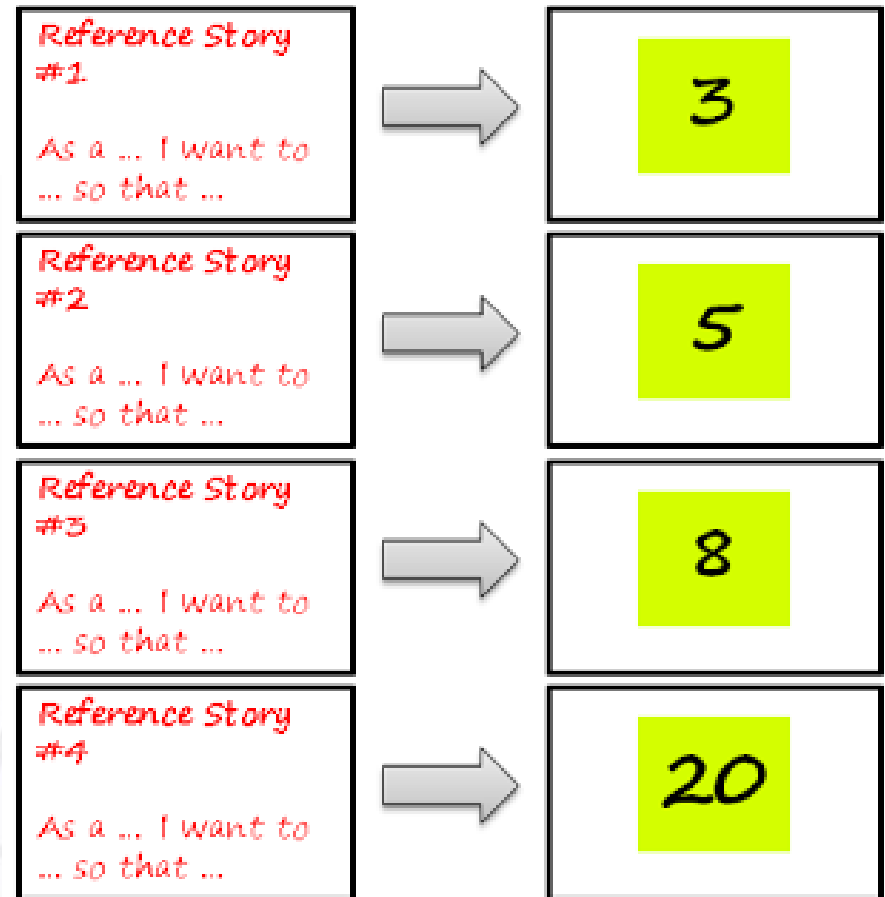


Relative Estimation

- It is hard to estimate in absolute hours
- The effort in hours could vary from developer to developer.
- Relative Estimation consists of estimating tasks or user stories, not separately and in absolute units of time, but by comparison or by grouping of items of equivalent difficulty.
- Methods:
 - T-shirt Size
 - Poker Planning

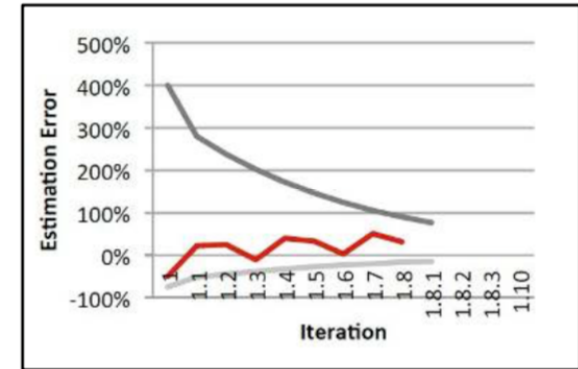
Story Point Estimation

- A story point is an arbitrary measure of effort required to implement a user story
- A reference story is an example of a story that we can fairly well relate to
- A new story can be compared to the reference stories and we can tell whether it is larger or smaller than each one of them



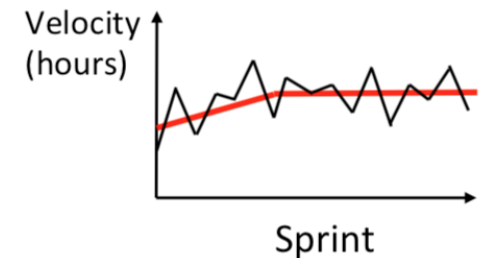
Why Points are Better Than Hours

1. Repeated studies have shown estimates in points are more accurate than estimates in hours



2. Estimating in hours undermines the fundamental purpose of velocity...
Measuring Output!

3. Time is finite. There are a fixed number of hours in a day, so a velocity based on hours can't increase!



Estimation: Story Point

- The “bigness” of a task
- Influenced by
 - How hard it is
 - How much of it there is
- Relative values are what is important
- A login screen is a 2. A search feature is a 8.
- Points are unit-less

Estimation: Story Point Scale



Based on Fibonacci sequence, a recurring organizational pattern

The story point scale has no statistically reliable relationship to man hours

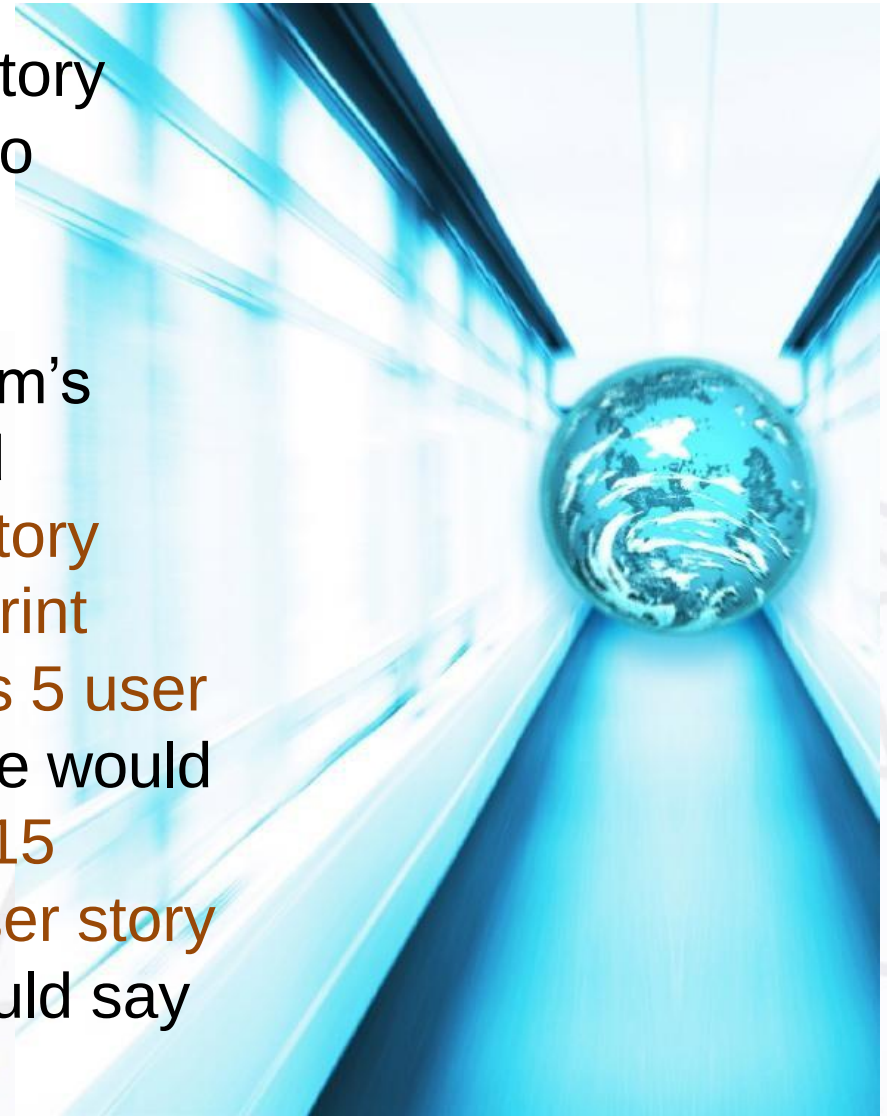
Value	Meaning
0	No effort required
1	No. problem, We could do this in few hours
2	
3	
5	Most common use
8	
13	
20	
40	
100	Impossible, this is very large
? ...	Need more information

As a PO or SM, I need to use
Velocity & the Burndown Chart
to monitor the Team's progress and ensure
a successful sprint/release

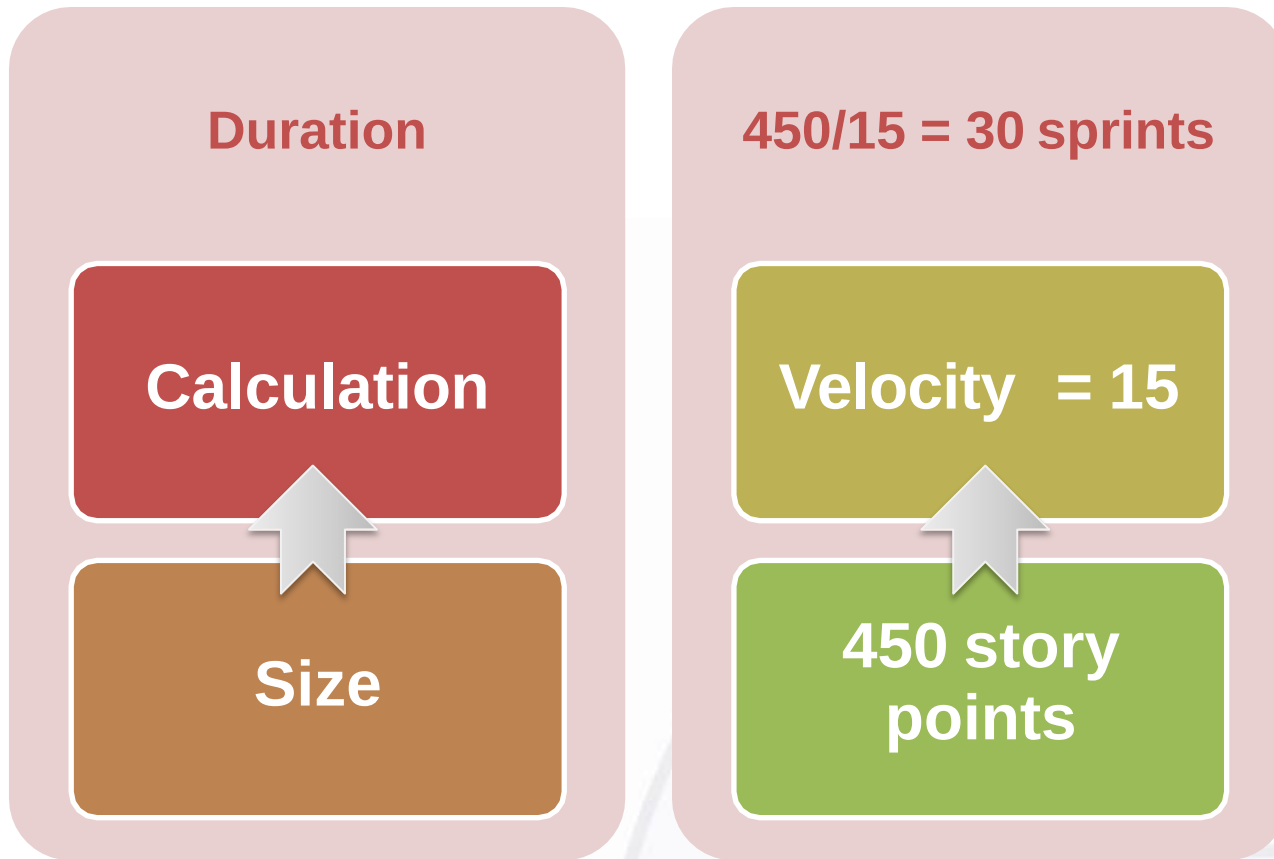
- Velocity is a capacity planning tool
- $\text{Velocity} = \sum \text{Story Points completed in a Sprint}$
- Estimated Velocity for next Sprint:
 - Estimated Velocity = Averages over Last N Sprints
- Velocity normally becomes stable after five or six sprints
- For the first sprint, just pick up number of stories that you think the team can finish at the end of sprint.

Velocity (cont.)

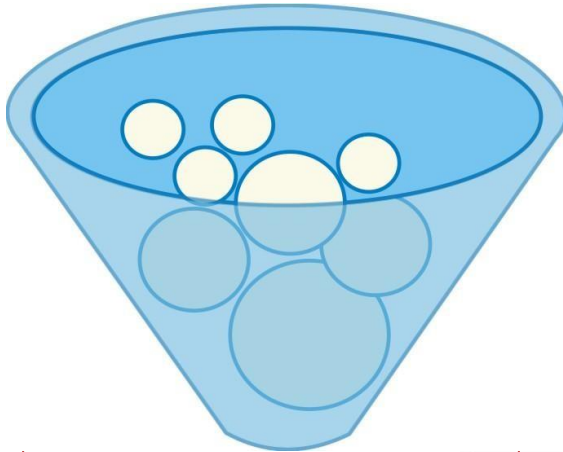
- To understand how unit less story points would work, we need to introduce a new concept – VELOCITY
- Velocity is a measure of a team's rate of progress. It calculated summing up the number of story points completed during a sprint
- Therefore if a team completes 5 user story of 3 points each, then we would say that the team velocity is 15
- Now if a team completes 4 user story of 5 points each, then we would say that the team velocity is 20



Velocity as Productivity



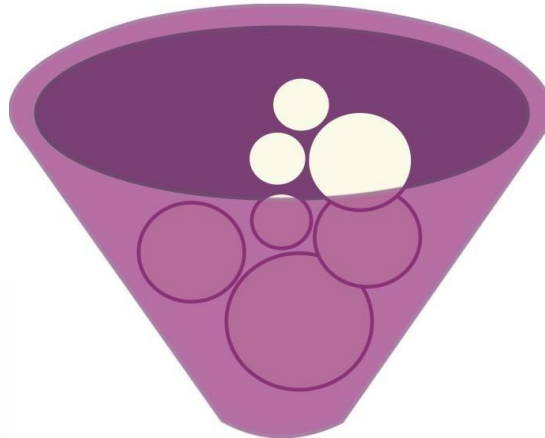
Velocity of Team A



Sprint 1

24 Points

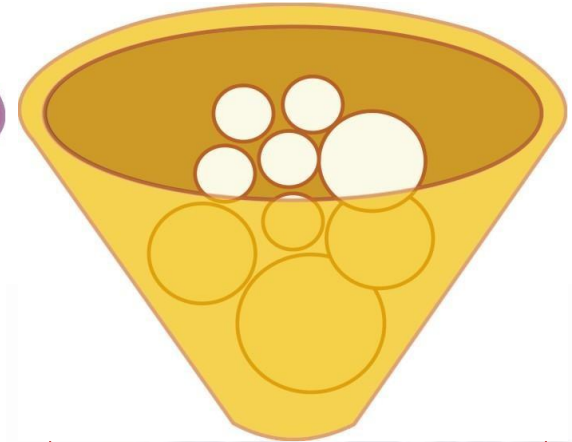
of size



Sprint 2

22 Points

of size



Sprint 3

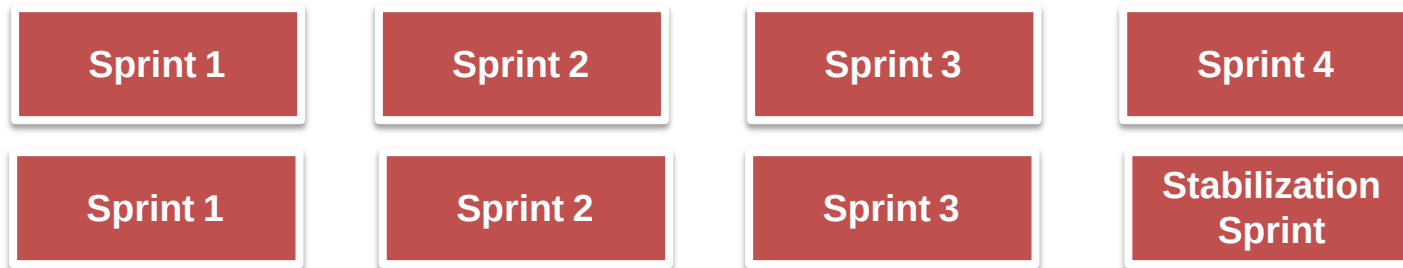
25 Points

of size



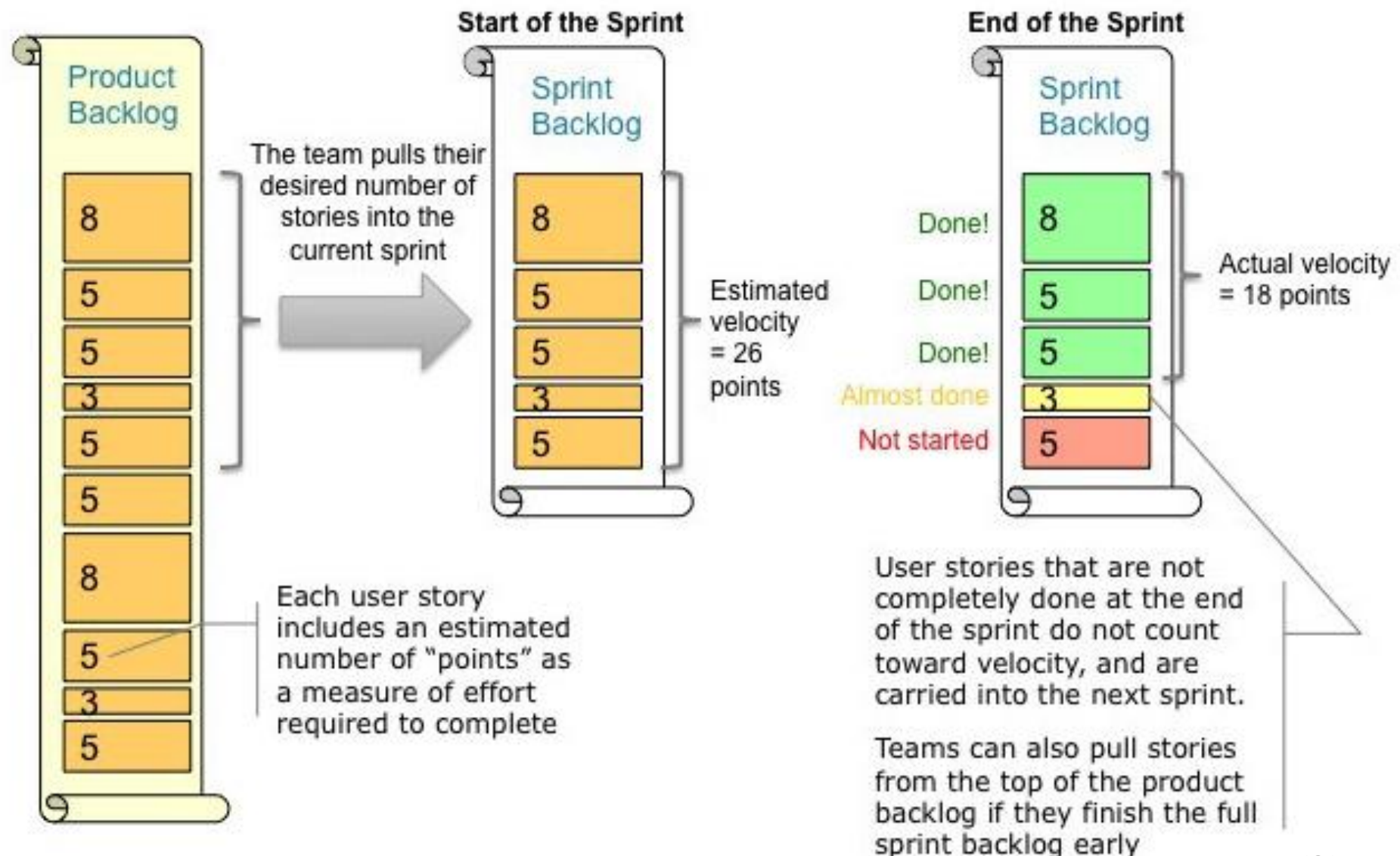
Average of ~ 24 Points per Sprint is the “Velocity”

Estimation: Stabilization Sprints

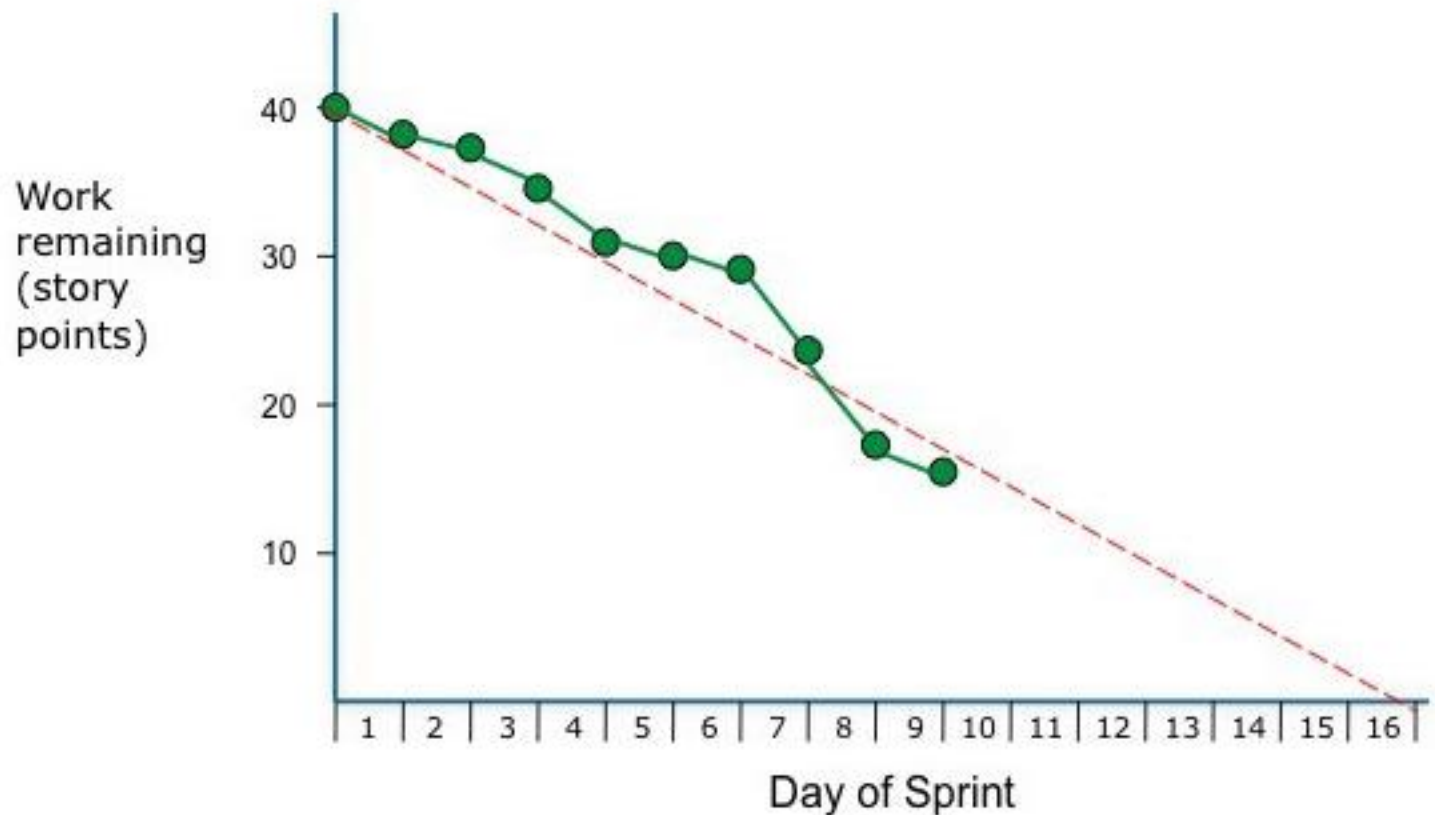


- During “regular” sprints target friendly first use
 - Beta customers and similar can use immediately after sprint
- During “stabilization sprints”
 - **Team prepares a product for release**
 - Useful during
 - active beta periods
 - when transitioning a team to Scrum
 - if quality isn’t quite where it should be on an initial release

"Velocity" is the Key Metric in Scrum



Velocity is Plotted on the "Burndown Chart"



Burndown chart answers the question, "Are we on track to successfully deliver this sprint's output?"

Burndown Strategies

- Burndown stories only in points
 - stories must be small
- Burndown tasks in points
 - use planning poker to estimate tasks in points
- Burndown tasks in hours
 - Scrum Foundation and ScrumInc. no longer view this as best practice

Burn Down Charts

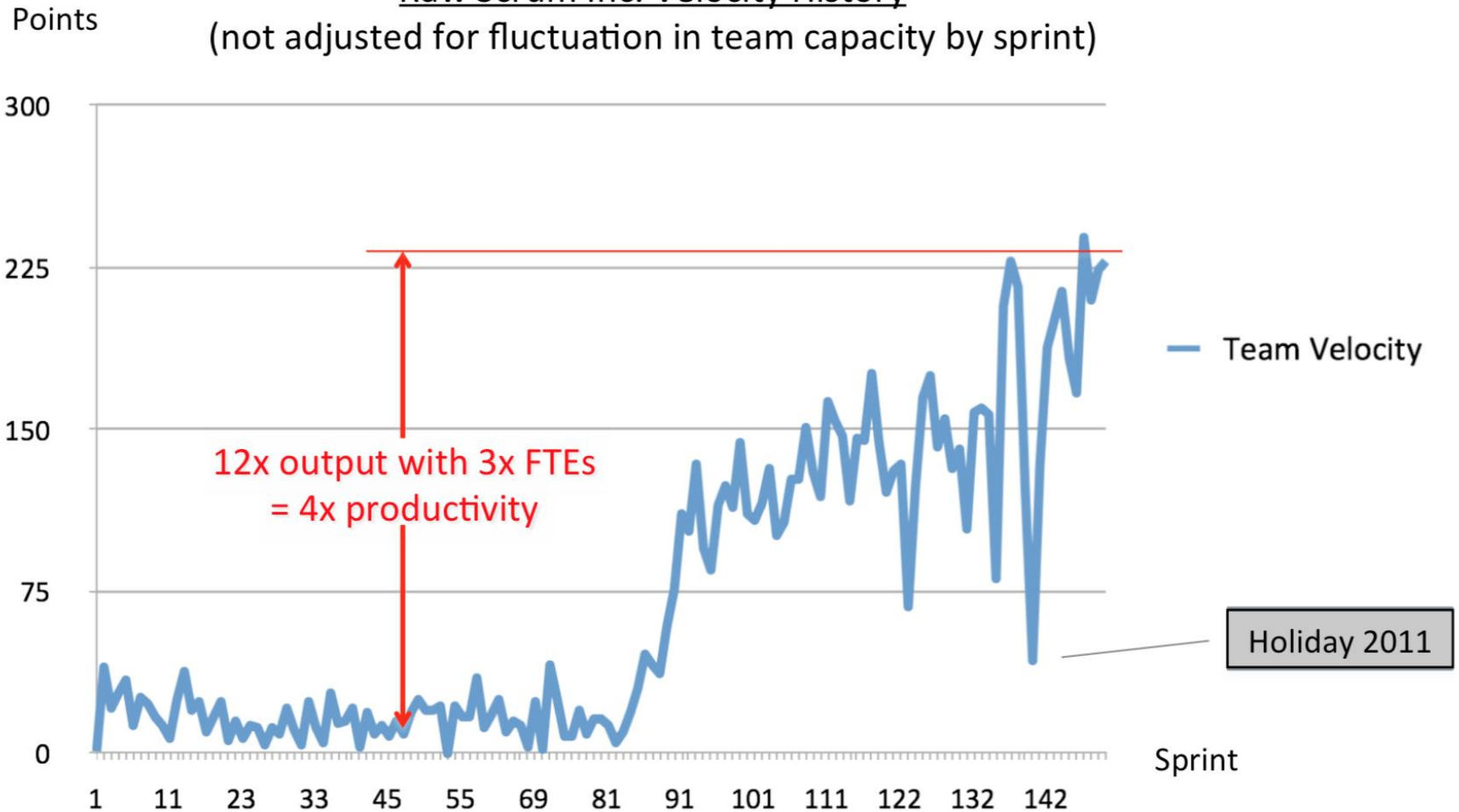
- Are used to represent “work done”.
- Are wonderful Information Radiators
- 3 Types:



- **Sprint** Burn down Chart (progress of the Sprint)
- **Release** Burn down Chart (progress of release)
- **Product** Burn down chart (progress of the Product)

Knowing Output and Taking Time for Process Improvement Enables Hyper-Productivity

Raw Scrum Inc. Velocity History



Source: Scrum Inc. performance data

Release Planning: Example

Product Backlog as made available from the PO

Priority	High-Level Capability	User Story	Initial Est	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5	Sprint 6
1	Home Page	As a Clockazon user, I want to be able to type the Clockazone URL in my browser and load Clockazon's home page	3						
2	Buy a clock	As a Clockazon user, I want to click the "buy" button to put a clock in my shopping cart	5						
3	Buy a clock	As a Clockazon user, I want to complete the purchase of the items in my shopping cart	8						
4	Buy a clock	As a Clockazon user, I want to receive an email with the details of my order, after it's finishing processing	3						
5	Shipping	As the shipping manager, I want to log onto the system and see the orders that need to be shipped	5						
6	Shipping	As the shipping manager, I want to print a shipping label and packing list	5						
7	Buy a clock	As a Clockazon user, I want to click the shopping cart icon and see what's in my shopping cart	3						
8	Buy a clock	As a Clockazon user, I want to remove an item from my shopping cart, or change the quantities	3						
9	Browse	As a Clockazon user, I want to see a page that shows detailed information about a single clock (such as price, specifications, user reviews, etc.), along with thumbnail photos of the clock	5						
10	Browse	As a Clockazon user, I want to click on a thumbnail photo to see a full-size photo of the clock	2						
11	Browse	As a Clockazon user, I want to select a type of clock (wristwatch, alarm clock, etc.), and see the clocks of that type	3						
12	Browse	As a Clockazon user, I want to select a manufacturer, and see all the clocks available from that manufacturer	5						
13	Browse	As a Clockazon user, I want select a number of rating stars, and see all the clocks that were rated at that level or higher	5						
14	Browse	As a Clockazon user, I want to see a list of clocks that have reduced prices or are "on sale"	5						
15	Search for clocks	As a Clockazon user, I want to enter a piece of text and click "search clocks", and then see a list of all clocks that match any part of that text (product search)	8						
16	Ratings	As a Clockazon user, I want to be able to log in and enter a star rating (1-5 stars) for a product which I have purchased, for several different criteria (timekeeping accuracy, value for money, etc.)	3						
17	Acct Mgmt	As a Clockazon user, I want to log onto my account and see whether my order has shipped or not	5						
18	Promos	As the promotions manager, I want to be able to show a list of all the clocks in our inventory, and be able to enter a sale price for any or all of them	13						
19	Promos	As the promotions manager, I want to enter free-form HTML for the content of a promotional box on the home page	5						
20	Mgmt Tools	As the sales manager, I want to upload the specifications, pricing, and photos for a single clock	8						
21	Mgmt Tools	As the sales manager, I want to do a bulk upload of clock specifications, pricing, and photos	13						
22	Shipping	As the shipping manager, I want to look up the status of a particular order	5						
23	Shipping	As the shipping manager, I want to change the status of a particular order	5						
24	Metrics	As the business manager, I want to generate a report showing total sales for a period of time	8						
25	Browse	As a Clockazon user, I want to select a price range, and see all the clocks that are within that price range	3						
26	Browse	As a Clockazon user, I want to see a list of "new arrivals" (clocks that have recently been added to the Clockazon inventory)	3						
27	Browse	As a Clockazon user, I want see a list of the best-selling clocks on the site	5						
TOTAL			144						

Release Planning

Assume that now we should only be planning for **“Must” + “Should”** = 144

$$\underline{144 / 24 = 6 \text{ Sprints}}$$

Estimation Buffer	+15%
Rework Buffer	+10%
Additions Buffer	+10%

$$\underline{= 8.3 \text{ Sprints}}$$

+1

$$\underline{= 9.3 \text{ Sprints}}$$

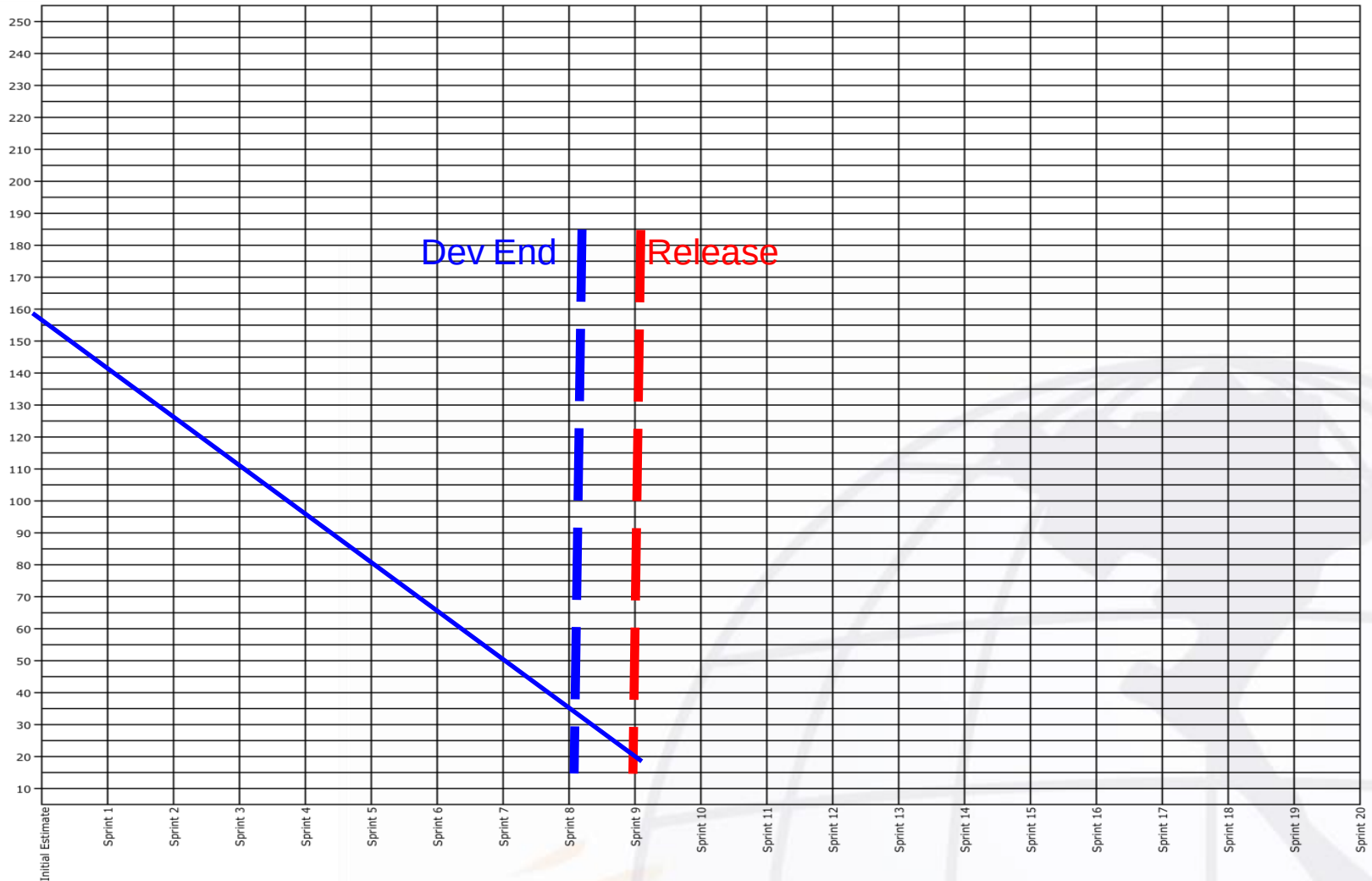
$$\underline{= 10 \text{ Sprints}}$$

**Pre-release
Sprint**



Idea from – <http://www.goodagile.com> – Pete Deemer

Release Burndown Chart

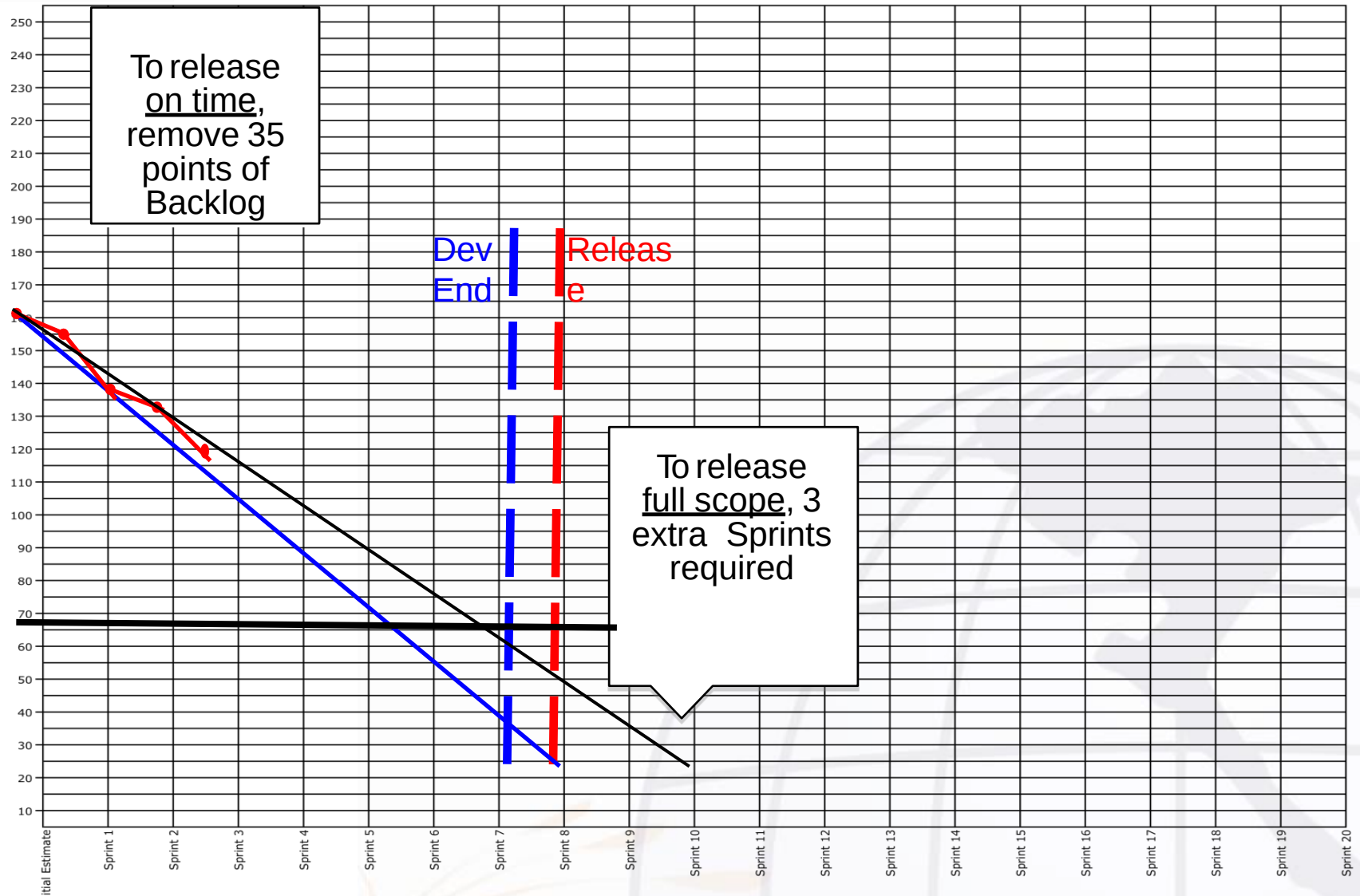


Source – <http://www.goodagile.com> – Pete Deemer

Release Burndown Chart (cont.)

Priority	High-Level Capability	User Story	Initial Est	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5
1	Home Page	As a Clockazon user, I want to be able to type the Clockazone URL in my browser and load Clockazon's home page	3	0	0	0	0	
2	Buy a clock	As a Clockazon user, I want to click the "buy" button to put a clock in my shopping cart	5	1	0	0	0	
3	Buy a clock	As a Clockazon user, I want to complete the purchase of the items in my shopping cart	8	8	0	0	0	
4	Buy a clock	As a Clockazon user, I want to receive an email with the details of my order, after it's finishing processing	3	3	0	0	0	
5	Shipping	As the shipping manager, I want to log onto the system and see the orders that need to be shipped	5	5	0	0	0	
6	Shipping	As the shipping manager, I want to print a shipping label and packing list	5	5	2	0	0	
7	Buy a clock	As a Clockazon user, I want to click the shopping cart icon and see what's in my shopping cart	3	3	3	0	0	
8	Buy a clock	As a Clockazon user, I want to remove an item from my shopping cart, or change the quantities	3	3	3	0	0	
9	Browse	As a Clockazon user, I want to see a page that shows detailed information about a single clock (such as price, specifications, user reviews, etc.), along with thumbnail photos of the clock	5	5	5	2	0	
10		As a Clockazon user, I want to click on a thumbnail photo to see a full-size photo	5	5	5	5	0	
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								
21								
22								
23								
24								
25								
26								
27								
28								
29								
30								
31								
32								
33								
34								
35								
36								
37								
38								
39								
40								
41								
42								
43								
44								
45								
46								
47								
48								
49								
50								
51								
52								
53								
54								
55								
56								
57								
58								
59								
60								
61								
62								
63								
64								
65								
66								
67								
68								
69								
70								
71								
72								
73								
74								
75								
76								
77								
78								
79								
80								
81								
82								
83								
84								
85								
86								
87								
88								
89								
90								
91								
92								
93								
94								
95								
96								
97								
98								
99								
100								
101								
102								
103								
104								
105								
106								
107								
108								
109								
110								
111								
112								
113								
114								
115								
116								
117								
118								
119								
120								
121								
122								
123								
124								
125								
126								
127								
128								
129								
130								
131								
132								
133								
134								
135								
136								
137								
138								
139								
140								
141								
142								
143								
144								
145								
146								
147								
148								
149								
150								
151								
152								
153								
154								
155								
156								
157								
158								
159								
160								
161								
162								
163								
164								
165								
166								
167								
168								
169								
170								
171								
172								
173								
174								
175								
176								
177								
178								
179								
180								
181								
182								
183								
184								
185								
186								
187								
188								
189								
190								
191								
192								
193								
194								
195								
196								
197								
198								
199								
200								
201								
202								
203								
204								
205								
206								
207								
208								
209								
210								
211								
212								
213								
214								
215								
216								
217								
218								
219								
220								
221								
222								
223								
224								
225								
226								
227								
228								
229								
230								
231								
232								
233								
234								
235								
236								
237								
238								
239								
240								
241								
242								
243								
244								
245								
246								
247								
248								
249								
250								
251								
252								
253								
254								
255								
256								
257								
258								
259								
260								
261								

Release Burndown Chart (cont.)



Sprint Planning

Iteration 1

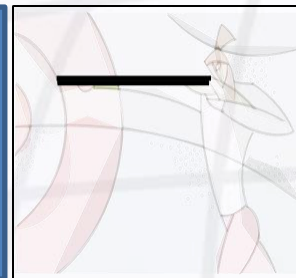
As a BUYFROM ME user, I want to...	3
As a BUY FROM ME user, I want to...	5

Iteration 2

As a Shipping Manager , I want to...	5
As a BUY FROM ME user, I want to...	2

Code the UI	8
Write test fixture	6
Code middle tier	12
Write tests	5

“Yesterday I started on the UI; should finish before the end of today.”



Sprint Planning (cont.)

Iteration 1

As a BUYFROM ME user, I want to...	3
As a BUY FROM ME user, I want to...	5

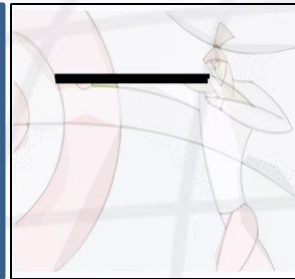
Iteration 2

As a Shipping Manager , I want to...	5
As a BUY FROM ME user, I want to...	2

Code the UI	8
Write test fixture	6
Code middle tier	12
Write tests	5

Creating
this is
Sprint
planning

"Yesterday I started on the UI;
should finish before the
end of today."

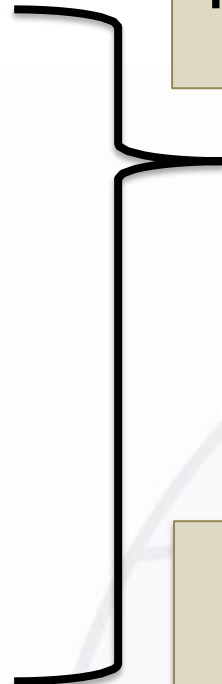


Estimation: Use Historical Data - Where Possible

Use Median



Sorted Velocities
27
34
35
38
39
40
40
41
45



Other concept is to use **Yesterday's Weather** (which means **take reference of the last couple of iterations** and plan for your next one)



90% confidence interval, Use 39 as your velocity in the team and plan your future iterations based on this

Estimation: Estimates are shared

- Estimates are **not created** by a single individual on the team
- Agile team **do not rely** on a single expert to estimate
- Estimates are best **derived collaboratively** by the team, which includes those who will do the work. There are 2 reasons for this:
 - First on an agile project, **one would not tend to know who specifically would be working on a given task**
 - Secondly even though one may not be doing the work (like for examples **specialized testing**), others may have something to say about the estimate
- Additional accuracy in estimation efforts yields very little value beyond a certain point



Estimation: Deriving an Estimate

The 3 most common techniques for estimating are:

➤ Expert Opinion

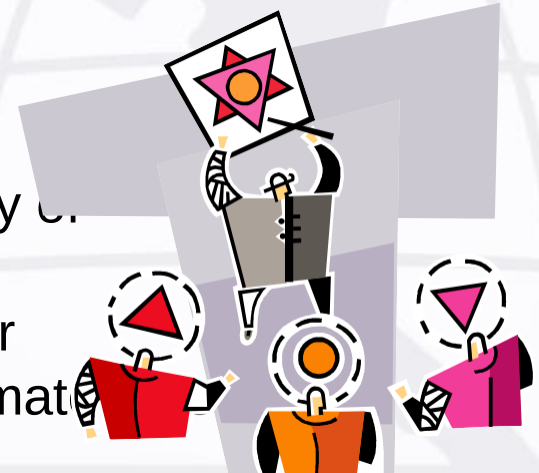
- Ask an expert of the subject, as to how long will it take to do a work.
- The expert relies on their intuition or gut feel and provide an estimate

➤ Analogy

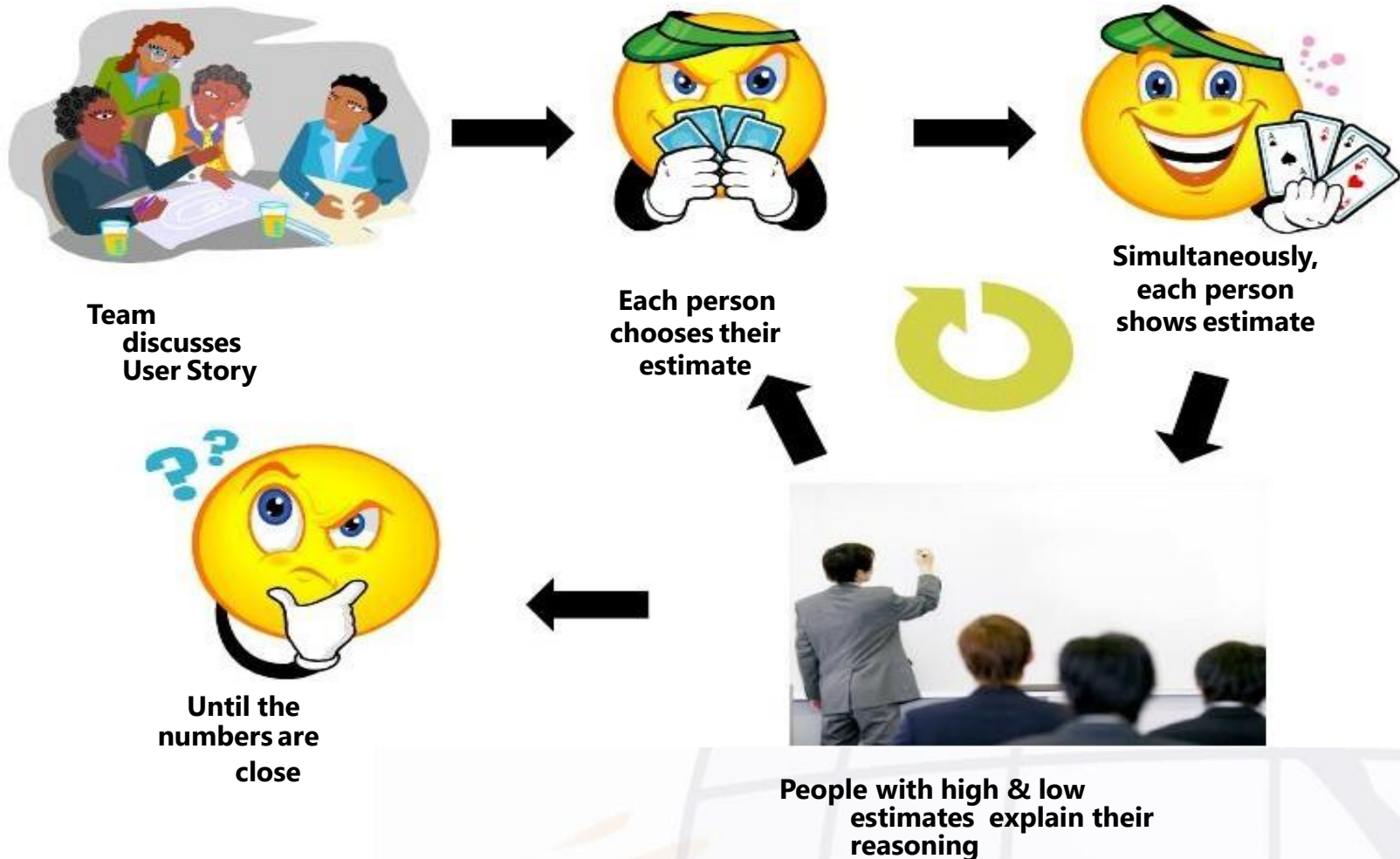
- When estimating by analogy, the estimator compares the story being estimated with one or more other stories. In this technique one compares the new story to the assortment of stories already completed or estimated

➤ Disaggregation

- Refers to splitting a story or a feature into smaller, easier to estimate pieces.
- It would be very difficult to estimate a single story of 100 days.
- The solutions to this is to break the large story or feature into multiple smaller items and then estimate



Story Points Estimation with Planning Poker





Poker Planning

1. Each estimator is given a deck of cards, Each card has a valid estimate written on it
2. Product owner reads a story and it's discussed briefly
3. Each estimator selects a card which is his or her estimate
4. Cards are shown at the same time, so that everyone can see
5. Differences are discussed, especially the very high and low estimates
6. Re-estimate, until an agreement on the estimate is reached

As a PO or SM, I need to know my role in
The Daily Scrum to help the Team
move fast and efficiently



What is the Daily Stand-Up?

- 15 minutes, same time every day
- 3 questions:
 - What did I accomplish yesterday?
 - What do I plan to accomplish today?
 - What are my impediments?
- Not a status meeting
- If further discussion is needed, takes place immediately after Stand-Up
- Distributed team challenges



Daily Standup

Why do the Daily Stand-Up?

- Coordinate work
- Identify impediments
- Clarify questions
- Keep focused

Scrum Masters' Job at Daily Meeting



Do:

- Facilitate meeting
- Keep it time-boxed
- Identify and track impediments
- Keep focus on highest priorities

Don't:

- Assign stories
- Tell the team what to do

Topic: Daily Stand-Up

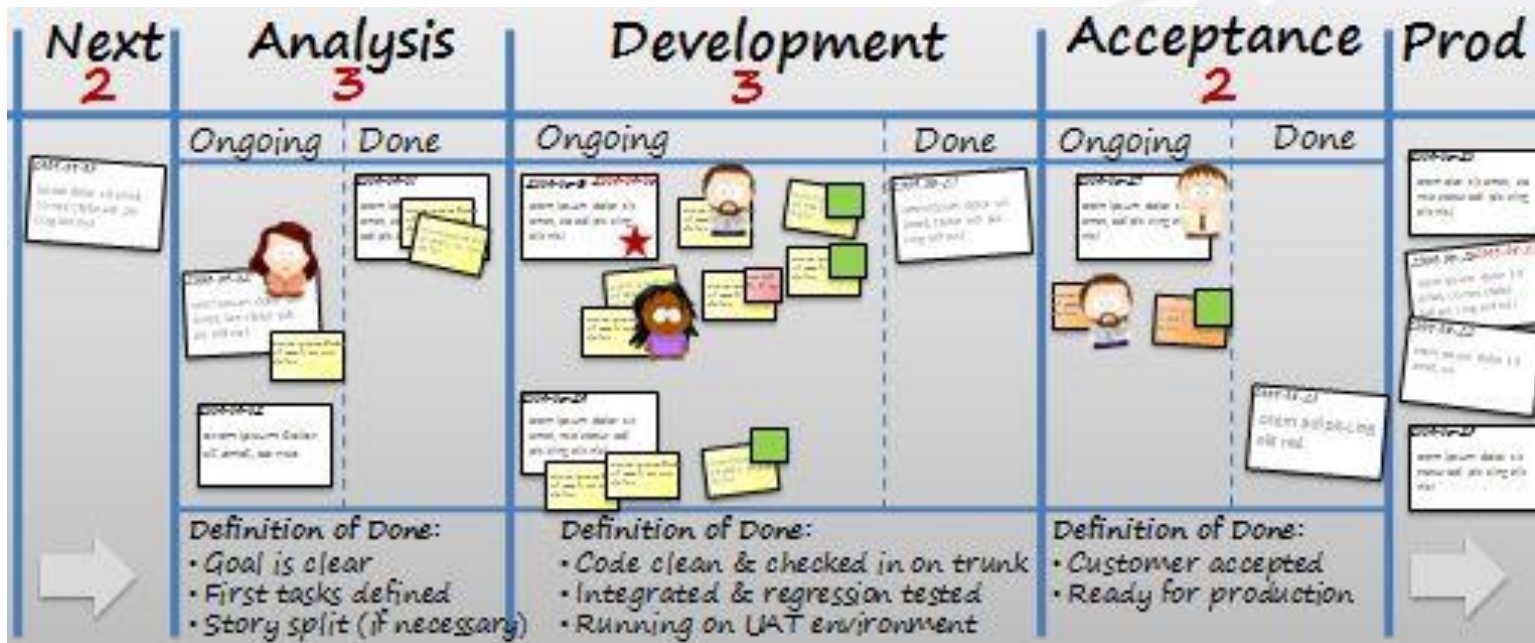


Photo: Jason Yip, Thoughtworks

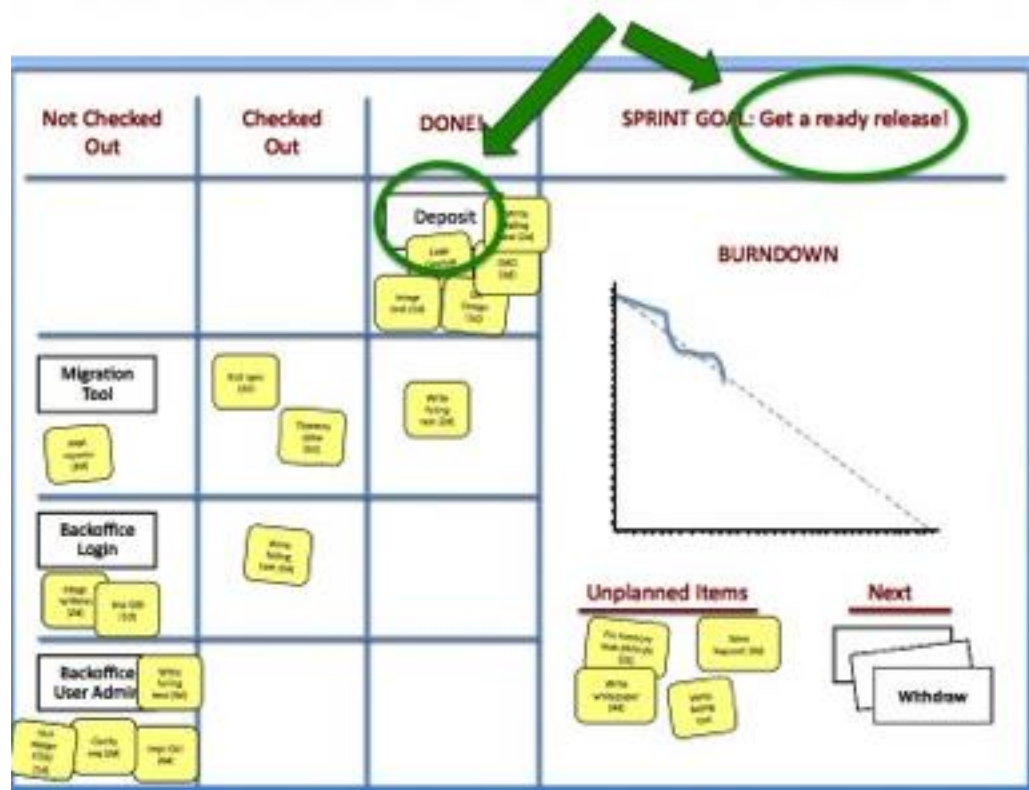
What's right with this picture?
What's wrong with this picture?

Scrum Board with Kanban Board

- Kanban is a flavor of agile that emphasizes the flow and continuous delivery of work
 1. Visualize your work
 2. Limit your work in process
 3. Manage Flow
 4. Make Process Policies Explicit

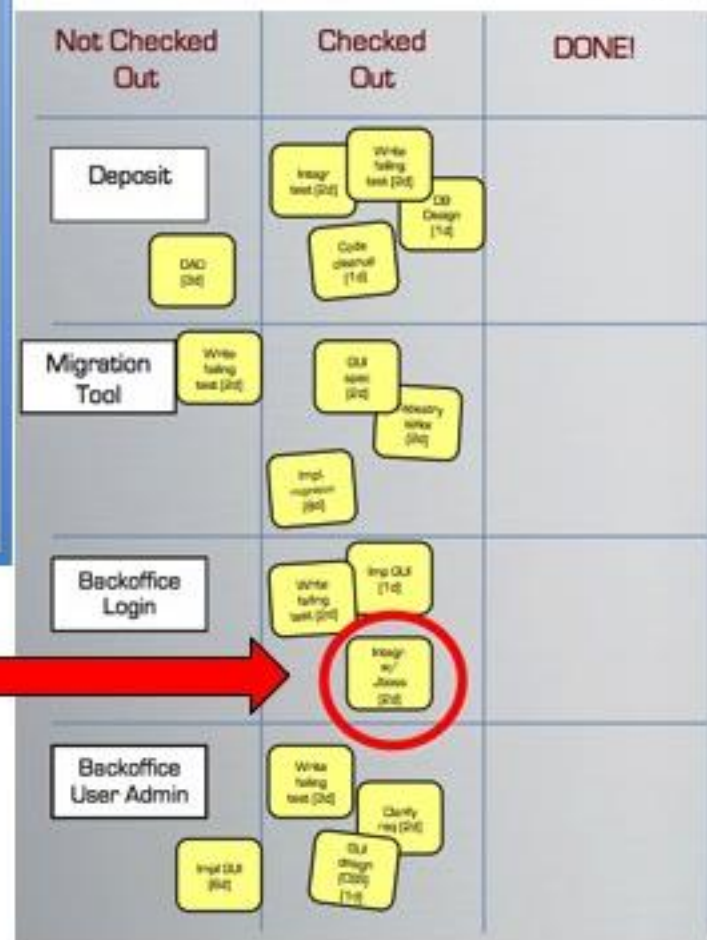


Care about the whole product



This product rocks!

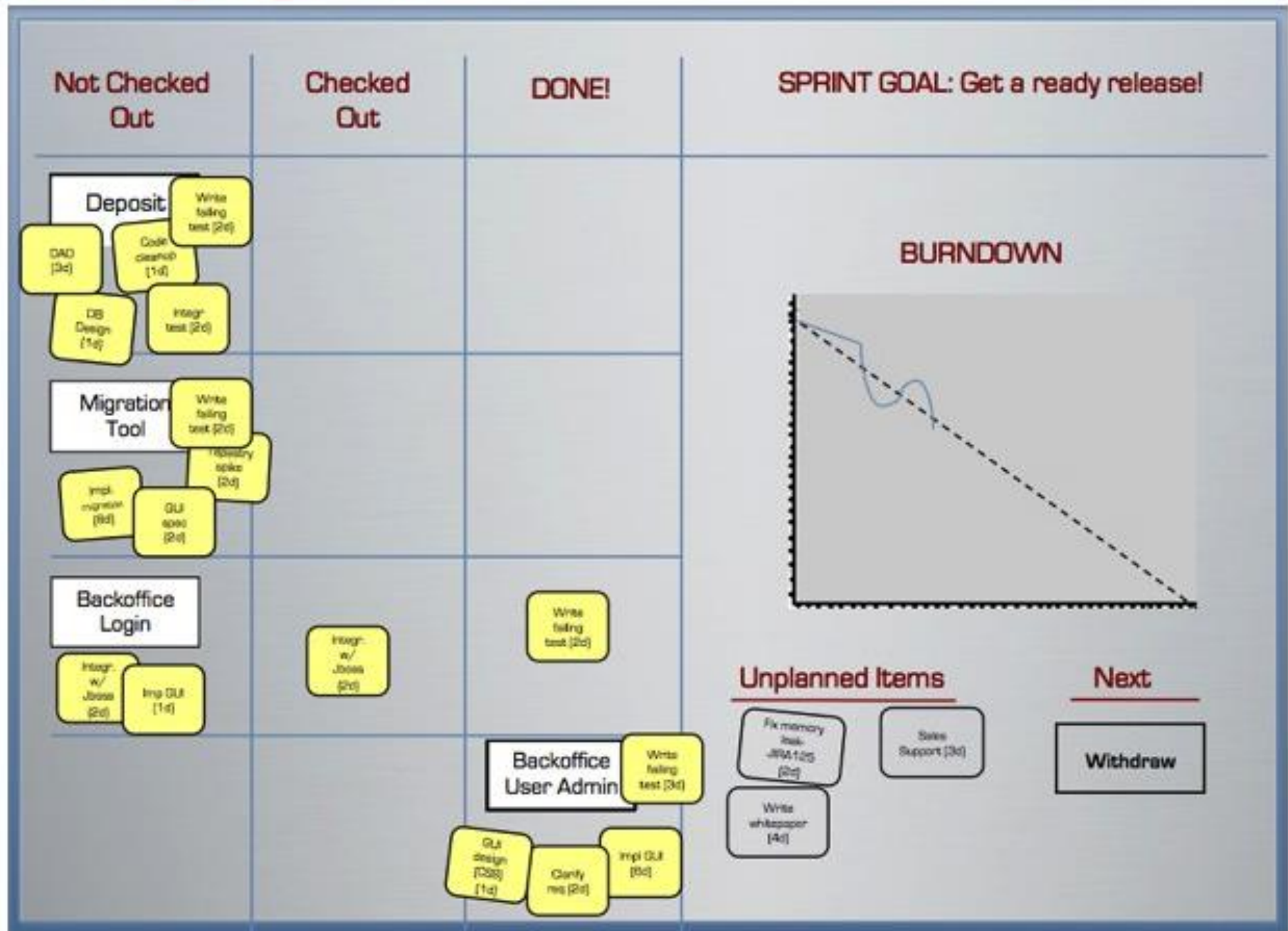
Boy are we effective as a team!



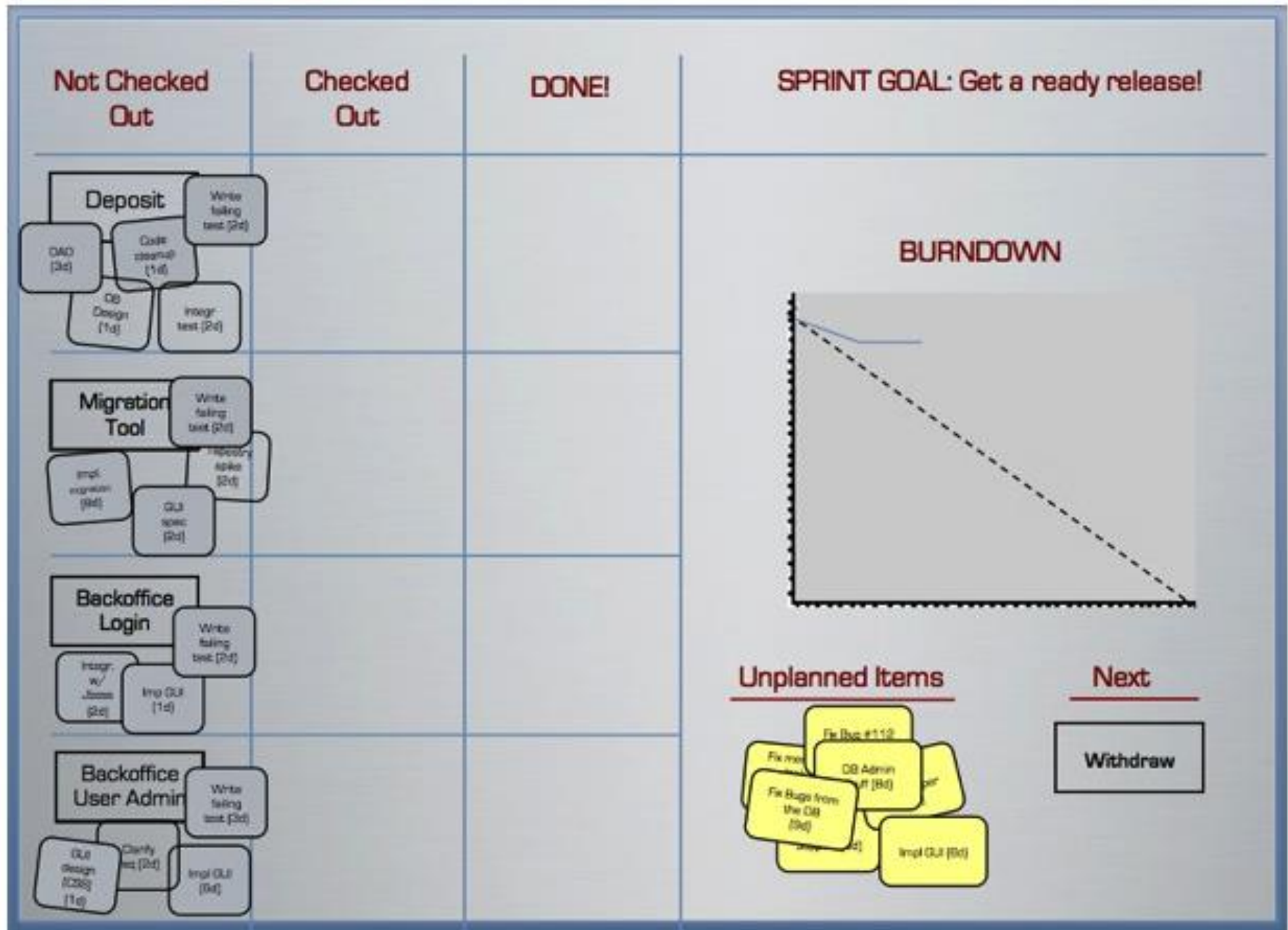
Not just your little task

I'm more efficient if I just do my tasks

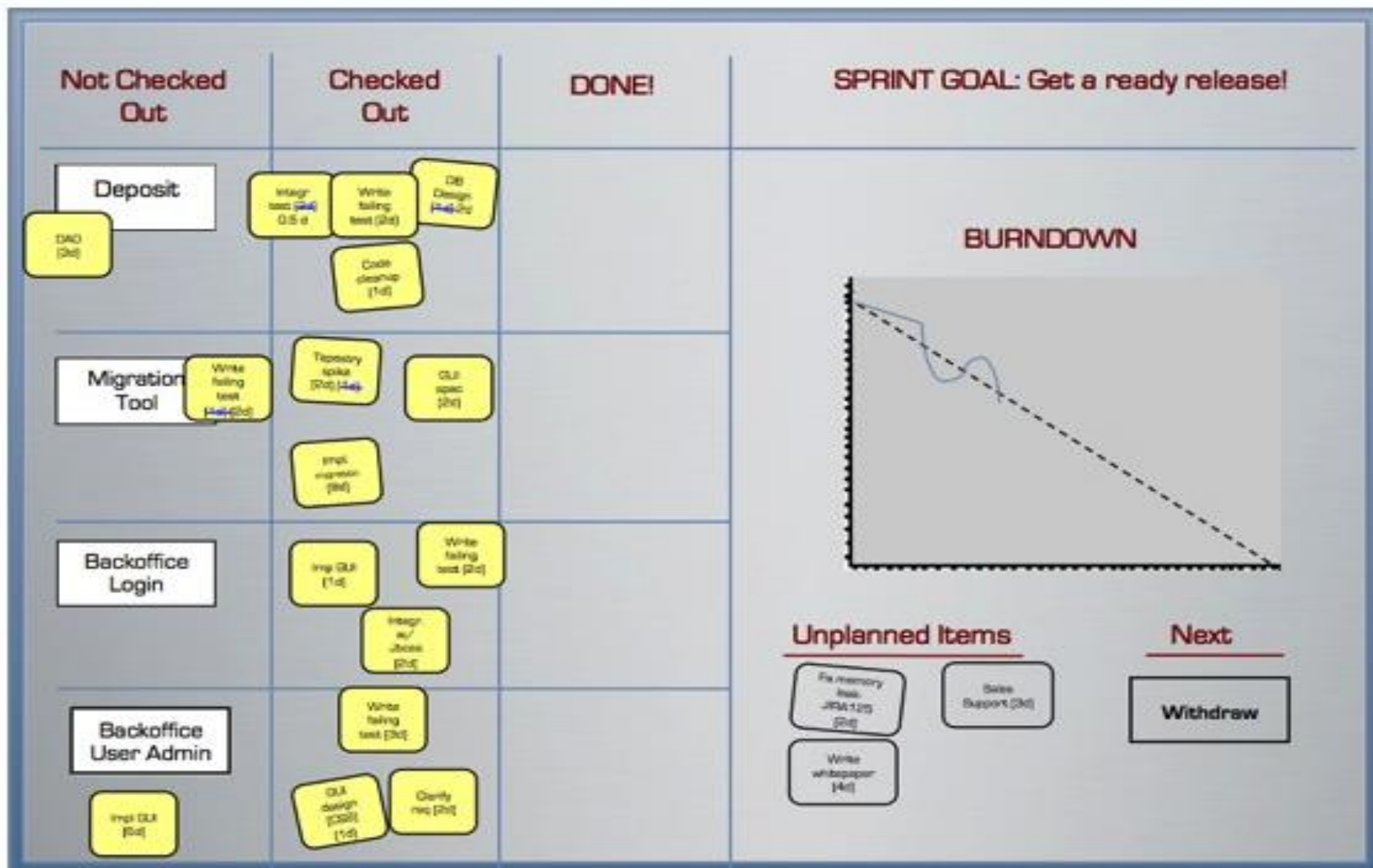
Warning sign #1



Warning sign #2



Warning sign #3



Cost: 50000 Euro/mo/team

Impediments

- Anything that keeps the team from being productive
- Team identifies in the Daily Stand-Up
- Scrum Master is responsible for removing
- Common impediments:
 - My ____ broke and I need a new one today.
 - I still haven't got the software I ordered a month ago.
 - I need help debugging a problem with ____.
 - I'm struggling to learn ____ and would like to pair with someone on it.
 - I can't get the vendor to call me back.
 - Our new contractor can't start because no one is here to sign her contract.
 - I can't get the ____ group to give me any time and I need to meet with them.

What if there are no impediments?

Try this experiment:

- What did you do yesterday to get the top priority story done (tested)?
- What will you do today to get the top priority story done?
- What are impediments that prevent the top priority story getting done today?



Photo: Henrik Kniberg

Common Corporate Impediments

Business analysts often become product owners.

- The product owner owns the product and business analysts have traditionally focused on communicating a customer's intent to the development team. They often lack the credibility and authority to drive business and technical decisions.

Team members work on more than one project.

- Working on many projects makes it hard for team members to find time to collaborate. It requires context switching, which is both an overhead and a risk. Ideas and information are lost as team members switch between projects.

Not everything is done in the sprint.

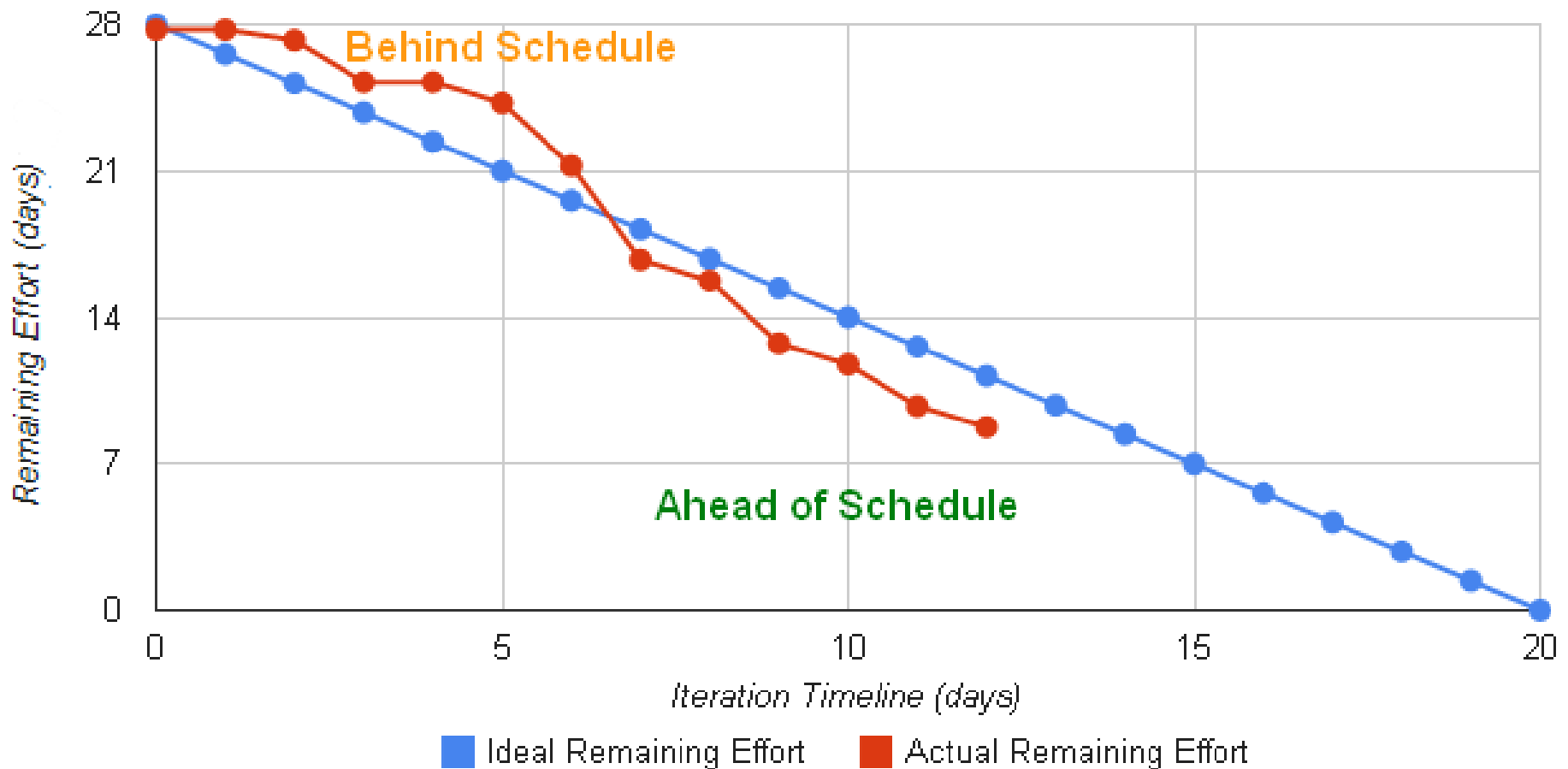
- Many activities that should be included in the sprint are not due to constraints, tradition, or process. Testing is a classic example; in many organizations, there are separate testing teams which results in missed tasks, slower bug resolution, and loss of the rapid feedback.

A project culture causes teams to lose momentum.

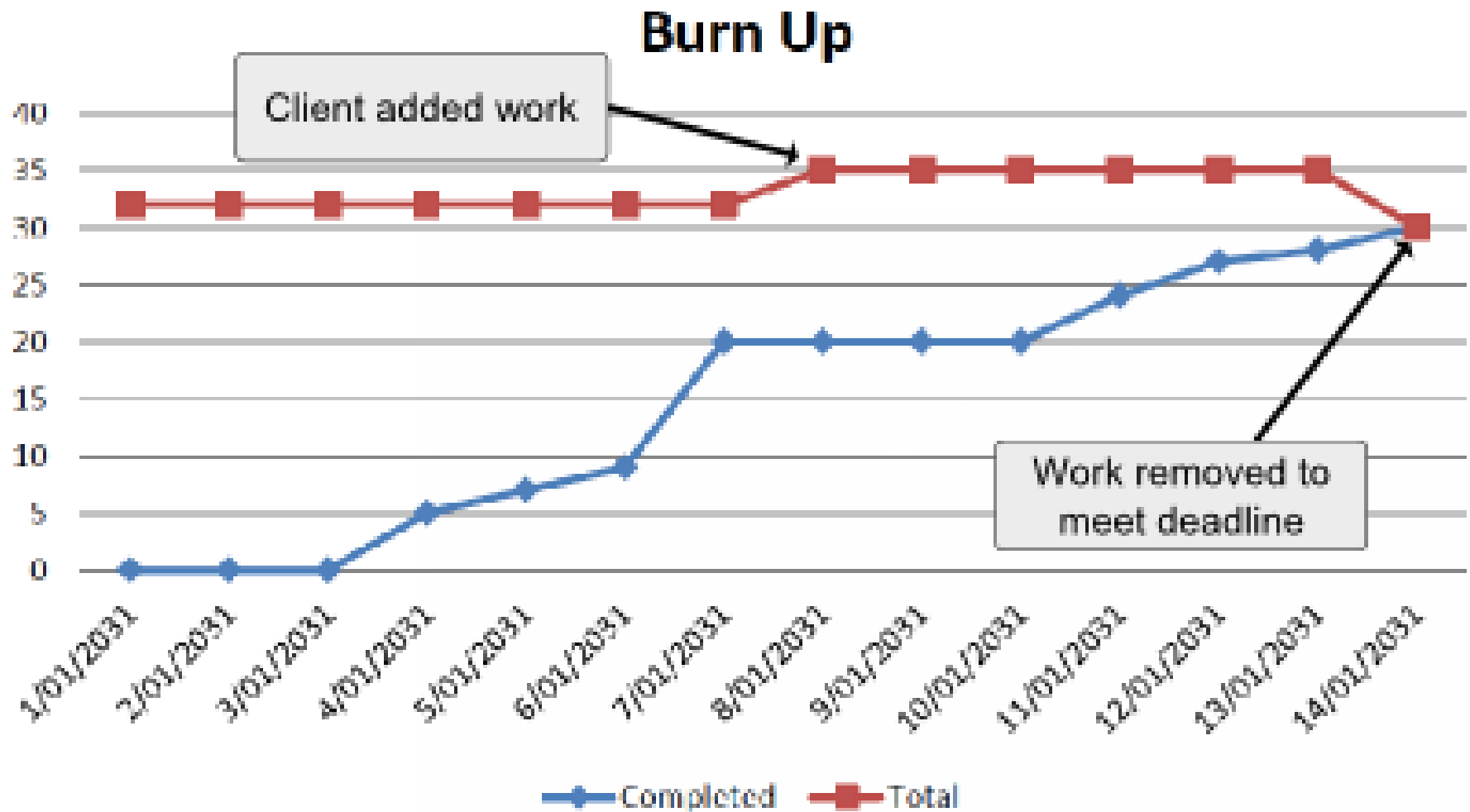
- Teams are often formed for one project. Agile methods encourage teams to stay together across projects. In a project culture, learning suffers and teams lose momentum and flow when they have to socialize working practices and gain agreement on engagement models every time a new product team is formed.

Impediments

Iteration 1 Burn Down



Impediments



Sprint Review

Time Boxed to one hour per week of Sprint length



- The Product Owner identifies what has been Done and what has not been Done
- The Development Team discusses what went well during the Sprint, what problems it ran into, and how those problems were solved
- The Development Team demonstrates the work that it has Done and answers questions about the Increment
- The Product Owner discusses the Product Backlog as it stands. He or she projects likely completion dates based on progress to date; and,
- The entire group collaborates on what to do next, so that the Sprint Review provides valuable input to subsequent Sprint Planning Meetings.

The result of the Sprint Review is a revised Product Backlog that defines the probable Product Backlog items for the next Sprint. The Product Backlog may also be adjusted overall to meet new opportunities.

Sprint Review NOT DEMO

1 Same people as planning meeting plus any other interested parties (e.g. end users)

2 Team demonstrates that they have completed the work as planned in the Sprint Goal.

3 Customers / PO can provide feedback, new ideas, changed requirements, changed priorities.

4 Typically takes the form of a demo of new features or underlying architecture



Good

- Little disturbance
- 80% strong
- Good night sleep
- Good food
- Good service
- Good location
- Good price
- Good staff
- Good atmosphere
- Good facilities
- Good value for money
- Good overall experience

Could have been better

- Planning more extensive
- Product quality & service
- Priority not so high
- Staff not so friendly
- Food not so good
- Service not so good
- Location not so good
- Price not so good
- Atmosphere not so good
- Facilities not so good
- Value for money not so good
- Overall experience not so good

Improvement

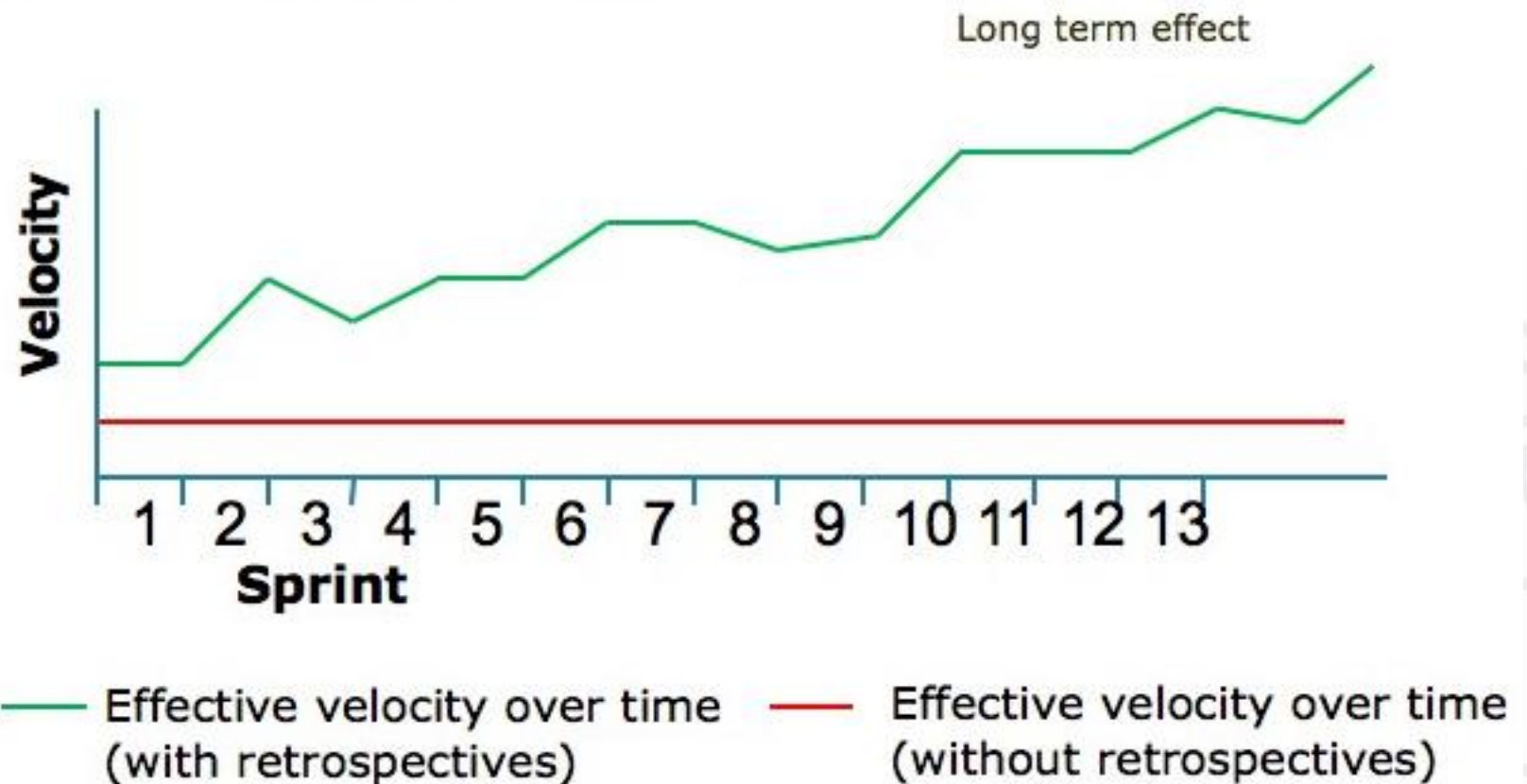
- More staff
- More food & drink
- More service
- More location
- More price
- More atmosphere
- More facilities
- More value for money
- More overall experience

- Product owner
- Scrum master
- Team

- Product backlog
- Sprint backlog
- Sprint burndown

- Sprint planning
- Daily scrum
- **Sprint review**
 - Demo
 - Retrospective

Sprint Retrospective



Sprint Retrospective (cont.)

- Process improvement at end of every Sprint
- All team members reflect on the **past sprint**
- Make **continuous process improvements**
- Two main questions are asked in the sprint retrospective:
 - What **went well** during the sprint?
 - What could **be improved** in the next sprint?
- **Facilitated** by ScrumMaster
- What went **well**, what could be **improved**.
- Scrum Master prioritizes based on **team direction**
- Team devises solution to most **vexing** problems

Sprint Retrospective (cont.)

1. Set the stage
2. Gather Data
3. Generate Insights
4. Decide What to Do
5. Close The Retrospective

Sprint Retrospective (cont.)

- **1. Set the stage**

- Start with a simple welcome and appreciation for people's investment of time
- Restate the purpose of the retrospective and the goal for the session
- **Define the ground rules**
- Goes through the **agenda**
- Define the **goals**

Sprint Retrospective (cont.)

- 2. **Gather Data**

- Create a **shared picture on what happened**
- Different people have different perspectives on the same event
- Include **both facts and feelings**, leads to better thinking and action in the rest of the retrospective.

- 3. **Generate Insights**

- What to do differently / What need to be improved
- What need to be keep

Sprint Retrospective (cont.)

- 4. Decide What To Do
 - Pick **the top items and decide what to do**
 - Choose items that they can commit to and that will have a positive effect.
- 5. Close the Retrospective Meeting:
 - **End the retrospective decisively:** don't let people (and their energy) dribble away
 - Help your team decide **how they'll retain what they've learned from the retrospective**
 - Look at what went well and what you could do differently in the next retrospective

Sprint Retrospective (cont.)

Example: Typical Issues



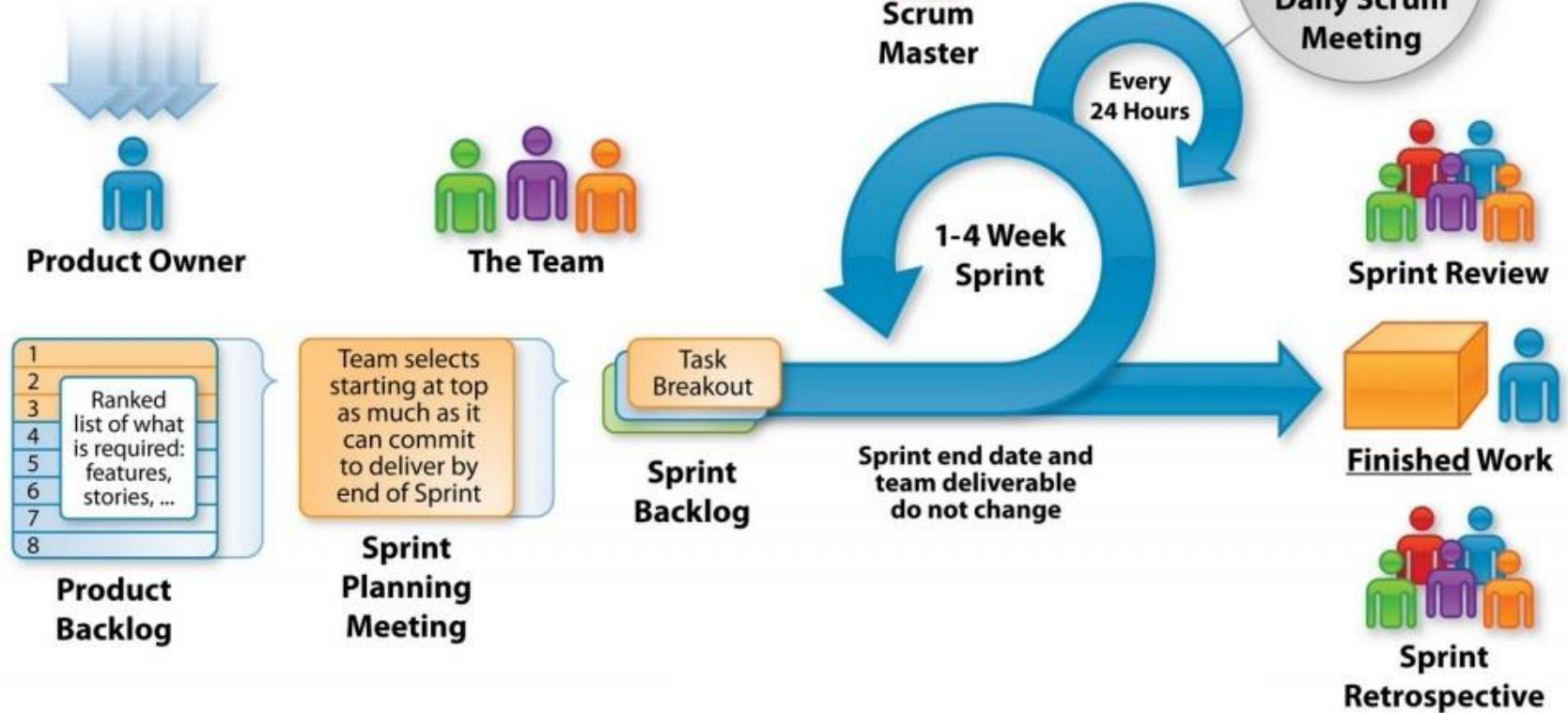
Sprint Retrospective (cont.)

Example: Retrospective Rules

- Only address one impediment
- Put the kaizen in the backlog for the sprint - Scrum the Scrum
- Action should usually yield results quickly
- Communicate actions (and success or not) back to the Team
- And the hardest rule: Use common sense.

The Agile: Scrum Framework at a glance

Inputs from Executives,
Team, Stakeholders,
Customers, Users





**Thank you for
your attention!
& See you in 4.
Pitfall – Best
Practice and
more**

4: Advanced SCRUM

Pitfall – Best Practice

- Handbook

Content

- Good characteristics
- Pitfalls (Common Problem)
- Best practices & Lesson Learnt
- more ...

The Agile: Scrum Framework at a glance

Inputs from Executives,
Team, Stakeholders,
Customers, Users



Product Backlog

Team selects starting at top as much as it can commit to deliver by end of Sprint

Sprint Planning Meeting



Sprint end date and team deliverable do not change



Every 24 Hours



Characteristics - SCRUM Master

- Knowledgeable
- Responsible
- Good team player
- Facilitator
- Good Observer
- Good Listener
- Servant Leader

9 BEHAVIOURS of a servant leader

1. Serve first
2. Build trust
3. Live your values
4. Listen to understand
5. Think about your thinking
6. Add value to others
7. Demonstrate courage
8. Increase your influence
9. Live your transformation

Servant Leadership Institute

Characteristics – Development Team

- Self organizing – Empowered - Collaboration
- Sharing goal and objectives
- Own its decisions & commitment
- **Consensus driven**
- Constructive disagreement
- Constructive feedbacks
- **Trust**
- Motivating team
- Believe they can solve any problem

Pitfalls

- Directive Scrum Master
- Product Owner Specifies Solutions
- Assigning Tasks
- Not Getting Done
- Problem-Solving in the Daily Scrum
- Stretch Goals
- Giving up on Quality

#1: PO Role Not Defined or Under-Resourced

Impediments

Ready



Done

Typical symptoms

- Stories frequently not done at Review due to external dependencies or in-sprint surprises
- Product Owner not available to answer Team questions in a timely fashion
- Many stories “discovered” during the Sprint
- Team feels priorities shifting too frequently
- Team gets conflicting messages from different sources

Root causes

- User stories not clear and READY at start of Sprint
 - Needed information not available in time
 - Poor clarity on who is responsible for providing what information
 - Unclear who leads story creation/refinement
 - **Product Owner role is not well-defined**
 - Single PO creating all backlog for multiple teams or all customer engagement thru to story creation for one accelerating team
 - **Product Owner role under-resourced**
- Conflicting Team goals from multiple sources
 - Unresolved competing stakeholder interests
 - **Product Owner role is not well-defined**

What to do about it

PO role not defined

- Assemble all stakeholders to decide on the single tactical PO to work with team
- All backlog should flow to team through PO
- Set up regular Meta-Scrum meeting for stakeholders to align without impacting team

PO role under-resourced

- Ensure that each team has its own PO
- Designate separate Strategic (epic-level market and ROI) and Tactical (ready backlog) PO to work closely together
- Assign cross-functional PO team

Target end-state

- Stakeholders have an aligned and compelling vision that is maintained regularly
- This vision communicated to Team through the PO and a consistent Product Backlog
- Backlog stories follow regular refinement process to ensure they are ready before Sprint Planning
- Progress communicated back to stakeholders without distracting the Team



Insufficient Cross-Team Coordination #2

Typical symptoms

- Inter-team dependencies impede work
- User stories appear in multiple backlogs, resulting in duplicate work
- Work discovered within Sprint by one team, that is needed by another Team to progress
- Outputs from one team don't integrate well with those of another
- Release dates are missed or hard to estimate

Root causes

- Missed release dates
 - Inter-team dependencies impede work
 - Impediments that span teams not identified
 - Teams do not communicate or coordinate efforts
- **No mechanism for cross-team process coordination**
- Same story appears in different backlogs or key story is missing from all backlogs
 - Teams do not integrate work or coordinate
 - Teams are pulling from separate backlogs
 - POs are not collaborating on backlog
- **No mechanism for central Product Owner coordination**

What to do about it

No mechanism for cross-team process and product coordination

- Establish Scrum of Scrums to coordinate team process and remove impediments as they are identified
- Create a single product backlog that all teams pull from, and hold regular PO meetings to refine unified backlog and plan releases
- Scrum of Scrums for other specialized roles as needed (eg. Architects, UX, QA, etc.)
- Provide Shared access to team metrics and charts

Target end-state

- Saturation of communication between Stakeholders, POs, and Teams
- All teams pull work from a single product backlog, which is curated by all POs
- Scrum Teams have a regular and established method of communication with each other
- Inter-team dependencies are identified ahead of time and addressed efficiently

#3: Team Working Individually Rather than Together

Impediments

Ready



Done

Typical symptoms

- Team thinks of backlog as a shared “to do” list where each PBI is done by only one person: “those are *my* stories”
- Team comprised of Subject Matter Experts
- Bottlenecks created around a single Team member
- One person or group typically working long hours to keep up with demand on their time

Root causes

- High level of Work in Progress (WIP)
 - Each team member pulls a different story
 - Stories requires skill only one Team member possesses
 - Lower priority stories started before higher priority ones completed
 - Next available Team member can’t pull next high priority story
 - High priority story depends on scarce skill
 - **Need for cross-training on skill**
 - Team often relies on one hero to “save the day”
 - This person is only one who can do a task
 - **Team works as a group of individuals**

What to do about it

Pair on Stories

- Encourage collaboration on stories to increase the quality of the end product
- Write stories that provide opportunities to pair
- “Divide and conquer” to get Done on priority stories quicker

Cross-Train to grow Team’s skillset

- Flag scarce skills as a Team impediment
- SME works with one or two Team members to help them learn the unique skill
- Lightweight checklists or notation stored in a Team Wiki for reference for common tasks

Target end-state

- A least two Team members can finish each story and ideally anyone can work on any story
- Work in Progress is low as the Team works together on top priority stories
- Work flows easily from one to member to another
- Team members can enjoy vacation without being needed to deliver work!



#4: Stories Aren't Ready Before Sprint Planning

Typical symptoms

- Sprint Planning Meeting is tedious and takes a long time to complete, maybe even a full day
- Team has many questions during Sprint Planning that PO cannot answer during the meeting
- Stories are difficult to estimate at Sprint Planning
- At the end of each Sprint there are several stories not finished or not even started

Root causes

- Team writing lots of new stories at Planning
 - New stories needed to deliver Sprint priorities
 - Team sees upcoming work for the first time
 - **Team not investing in Refinement**
- Lots of unplanned work emerges during the Sprint
 - Research or clarification often required to begin work planned
 - Team hasn't thought all work needed to deliver the story
 - **Team not investing in Refinement**
- PO needs input from external stakeholders
 - Team needs more information to plan
 - PO hadn't anticipated required lead time
 - **Team not investing in Refinement**

What to do about it

SM encourage Team to look ahead

- Adopt mindset of looking forward to anticipate questions, dependencies and risks
- Coordinate regular Refinement meetings for Team and PO to discuss future sprints
- Coach team to utilize INVEST criteria

PO meet with Team before each Sprint

- Approach specific Team members with questions needed to prepare Sprint Backlog
- Attend Refinement meetings with Team to explain upcoming work, get Team clarification
- Clarify work with stakeholders before Planning

Target end-state

- Shorter and more effective Sprint Planning meetings (1 hour or less per week of Sprint)
- Few "surprises" during Sprint that could have been avoided with better planning
- Team finishes planned work ~80%+ of Sprints
- Team and PO work together to Refine backlog (expect 5-10% of the Team's time)

#5: PBIs Not Broken Down into Small Vertical Slices of Functionality

Impediments

Ready



Done

Typical symptoms

- Stories usually involve only one discipline or team member (function-centered stories)
- Stories difficult for team to act on immediately
- Several stories must be completed before functionality would create value for customers
- Multiple days pass without team completing a story (uneven burndown)
- Actual work often much greater than estimated

Root causes

- Team struggles to work together on PBIs
 - PBI definition includes only one person/ functionality from team
 - **Defined from team not user perspective**
- Multiple stories must be completed before incremental functionality ships
 - **PBIs address only one functional element**
- Actual work often much greater than estimated
 - Not all team members participate in estimation for function-centered PBIs
 - Team members think “it isn’t my work”
 - **PBI not defined as vertical functionality**

What to do about it

PBI's Not Defined As Vertical Slices of Functionality

- Make sure every PBI is in “User Story” form, or at least Team can identify how PBI generates incremental customer value
- Get entire team to agree on clear Definition of Ready for all Backlog items that aligns with target end state
- Have customers participate in Sprint Review to reinforce customer value perspective
- Have PO spend more time with Customer and/or get training on writing better user stories

Target end-state

- Each completed Story delivers a “potentially shippable” increment of value to customer
- Multiple team members can “swarm” together on priority stories
- Every Story is immediately clear and actionable
- Sprint burns down relatively smoothly
- Release Plans are relatively accurate
- Velocity is increasing roughly 10% per Sprint

#6: Team Over-Commits to Work (or is Committed by Someone Else)

Impediments

Ready



Done

Typical symptoms

- At the end of most Sprints, there are unstarted stories or stories not meeting Definition of Done
- Team is working at a unsustainable pace to try and complete each sprint
- The number of stories "in progress" remains high throughout the sprint
- Team feels "behind schedule" or under pressure to finish more output quickly

Root causes

- Team is not completing most Sprints
 - Team over commits during Sprint Planning
 - Team guesses about how much work it can complete each sprint
 - **Team is not tracking velocity**
- Team is working at an unsustainable pace to complete each sprint
 - The team is overcommitted
 - Team following a plan that dictates what must be done by when
 - Team does not control what work is brought into the Sprint
 - **Team is not self-organizing**

What to do about it

Team not tracking Velocity

- Each story brought into the sprint should be estimated in points
- All finished points totaled at end of every Sprint
- Implement Yesterday's Weather Pattern for Sprint Planning

Team is not self-organizing

- Align with leadership on expectations for empowered teams
- Secure buy in that reality on the ground trumps the plan
- Team estimates work and commits to how many stories to bring into the sprint

Target end-state

- Team is tracking Velocity each sprint and all team members know Velocity if asked
- Team pulls in work equal to the average actual points completed in recent sprints
- Team and PO work together to prepare for Sprint Planning
- Team decides, and is not told, how much work to pull into the Sprint Backlog

#7: Found Work Interrupts Sprint Regularly

Impediments

Ready



Done

Typical symptoms

- Team frequently (>20%) fails to complete planned work by end of Sprint
- Team discovers significant unplanned work or receives frequent “surprise” requests from stakeholders that must be addressed right away
- Team feels like priorities are constantly shifting
- Planned stories don’t move to Done
- Burndown chart is flat

Root causes

- Significant amounts of “found work” enters sprint
 - Team not anticipating what is needed to complete work
 - Team is new, or working in unfamiliar area
 - Team hasn’t given room in Sprint for learning
 - **Build in “buffer” for found work**
- Frequent surprise requests from stakeholders
 - Stakeholders asking Team directly for work
 - No formal process for handling “urgent” requests – informal requests add up
 - **Need process for managing, prioritizing and limiting mid-sprint external requests**

What to do about it

Found works interrupts Sprint regularly

- Implement the Interrupt Pattern and include Sprint buffer in categories where found work is expected
- Use Retrospective to identify ways to anticipate found work better

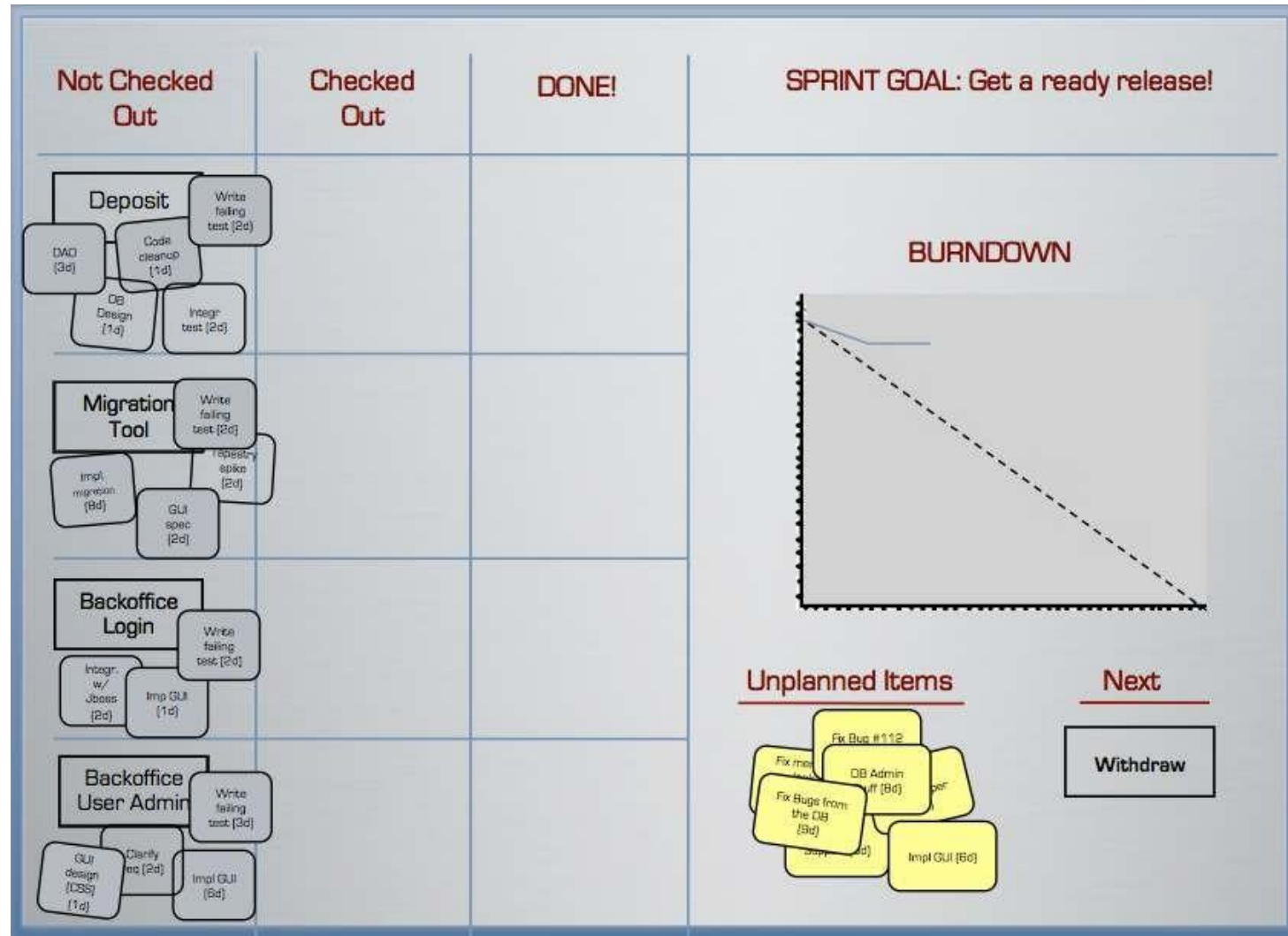
External stakeholder requests displace planned work

- Confront leadership with the effect of interruptions
- Implement the Interrupt Pattern, include limited buffer for surprise requests, and put PO in path to defend team

Target end-state

- Team anticipates some level of unplanned work, and allows for this in Sprint Backlog
- Unplanned work is limited to allow planned work to proceed to completion
- Team finishes all planned work early, and is able to pull additional stories from Product Backlog
- Velocity increases ~10% each Sprint as planning and execution improve

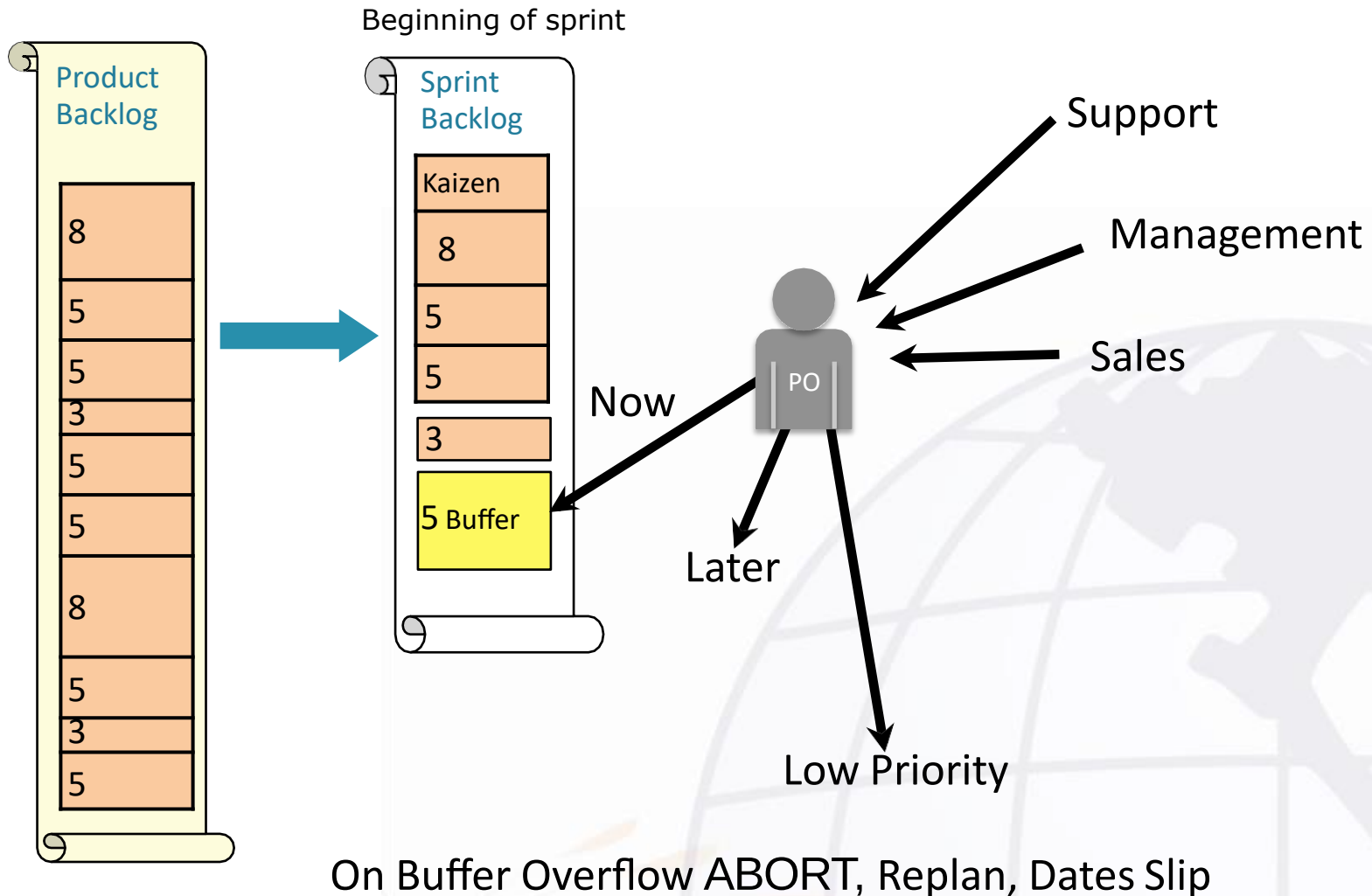
If Your Backlog Looks Like This, You Have a Problem with Interrupts!



Source: Revised after Henrik Kniberg

Pattern: The Interrupt Pattern

Dealing with the unexpected





Velocity is Not Tracked or Measured in Hours Rather than Story Points

Typical symptoms

- Team often fails sprints or needs to abort
- Team misses major product releases
- Team doesn't know its velocity
- Sprint Planning takes longer than an hour
- If known, Velocity is not increasing over time
- User Story estimates are often significantly off

Root causes

- Team brings too much work into each Sprint or sets unrealistic delivery deadlines
 - Teams don't use "Yesterday's Weather" Pattern of only planning as much work as they have completed in the past
 - External deadline pressure applied
 - Team does not track Velocity
 - Team does not fully appreciate its utility
 - Team is "too busy" to estimate
 - Team discovers a lot of work during Sprint
 - Team estimates in hours, not points
 - Team thinks hours are more accurate
 - Team thinks hours are easier
- Already working with hours based on team's hourly billing – inertia

What to do about it

- Estimate all backlog items that need to be completed, using Agile estimation methods
 - Use Story Points estimating relative size
 - Planning Poker rapidly harnesses "wisdom of the team" for more accurate estimates
 - Update estimates as team learns more
- Track the Velocity, or total points completed, at end of each Sprint
 - Useful for planning future work and measuring process improvement over time
- Never plan for the Team to produce more output per Sprint in the future than it has in the past
 - Just be happy when it does

Target end-state

- The Team knows its Velocity
- The Team bring only as much work into each Sprint as it has actually completed on average over previous three Sprints
- Short and effective Sprint Planning meetings (1 hour or less per week of Sprint)
- Team Velocity increases at ~10% per Sprint
- Team ultimately becomes Hyper-Productive

Daily Standups and Retrospectives are Ineffective #9

Impediments

Ready



Done

Typical symptoms

- Velocity is not increasing
- The Daily Scrum feels like a “status meeting”
- Team members never have any impediments
- Team does not know its top impediment
- The Retrospective meeting is skipped because it “wasn’t useful”
- Team processes have been static for several Sprints

Root causes

- Team members never have impediments
 - Working hard to finish work and not investing enough time in continuous improvement
 - **SM not facilitating needed introspection**
- Team members are hesitant to name issues
 - There is a fear of conflict
 - **Team lacks needed trust to face underlying issues**
- Retrospective skipped
 - Team doesn’t perceive value to retrospective
 - Retro does not drive to actionable results
 - Identified impediments are not removed
 - **SM not following up on impediments**

What to do about it

SM owns identifying/removing Impediments

- Additional training or coaching for SM
- Surface team blockers at the daily meeting and push actively to resolution
- Track impediments and show impact of removal

Implement the Happiness Metric

- Great tool to probe for impediments and underlying root causes at Retrospective
- Entire team discusses the top root cause and identifies counteractions

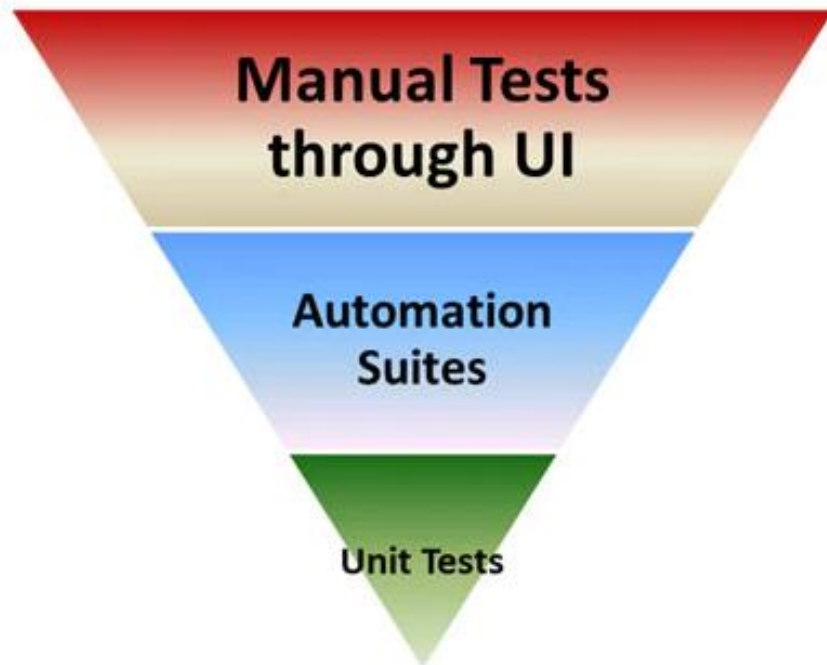
Confront Interpersonal Issues on Team

- Make meetings a “safe” environment to clear the air and voice future issues
- Accept that some team members may not fit

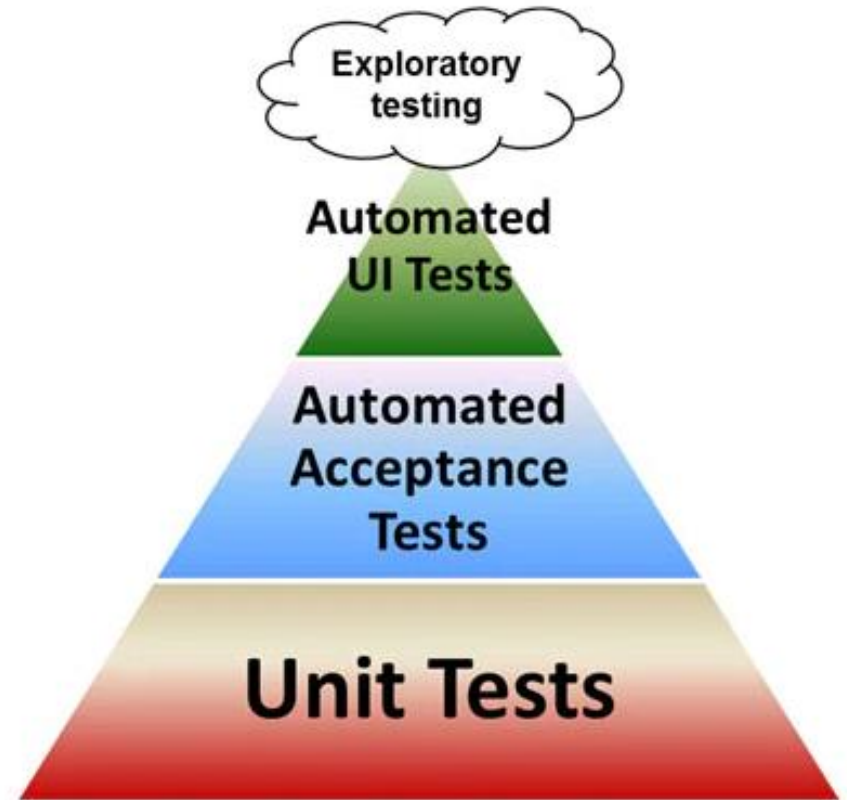
Target end-state

- Velocity grows ~10% each Sprint & team has fun
- Team leaves each Retro with one actionable process improvement and acceptance criteria
- SM maintains visible impediment backlog
- Daily Scrum has rich and interactive discussion around making process easier
- The Team can bring up impediments without fear of inciting an emotional backlash

Agile Testing



Traditional
(find bugs)

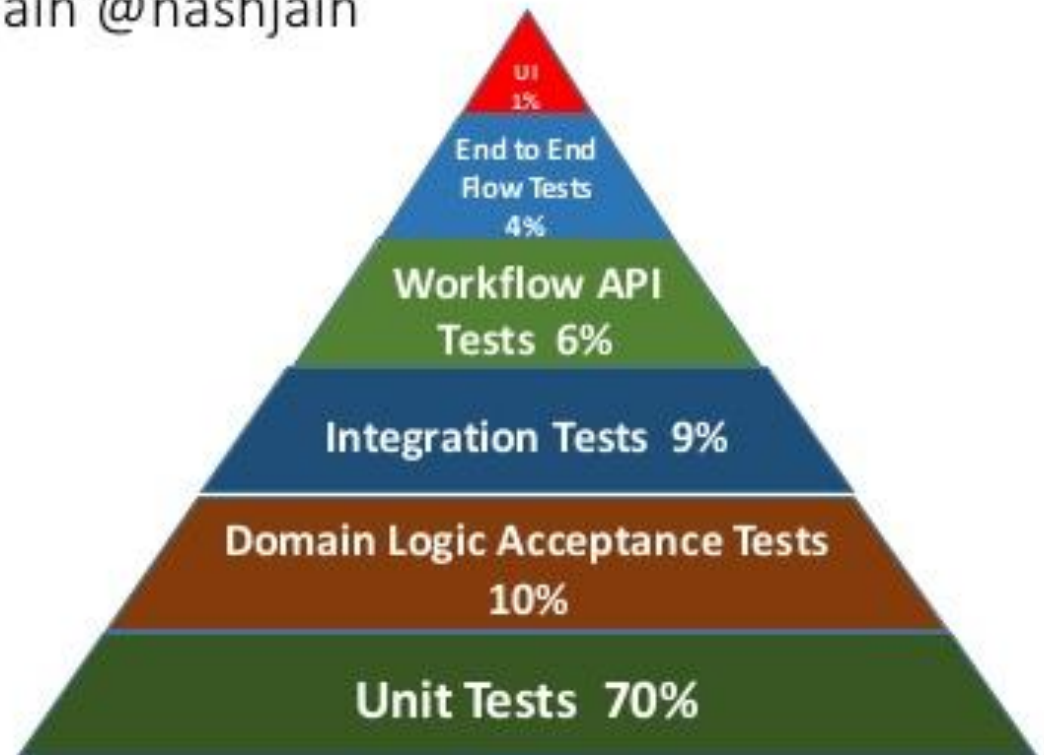
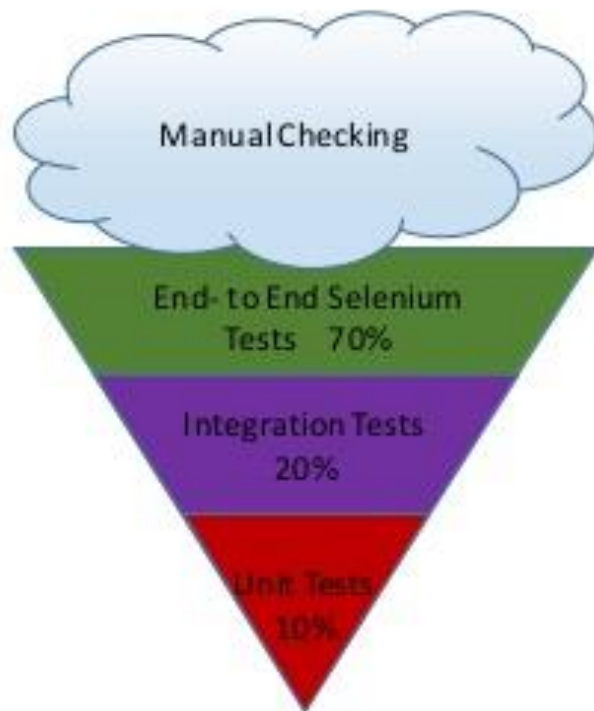


Agile
(prevent bugs)

Agile Testing (cont.)

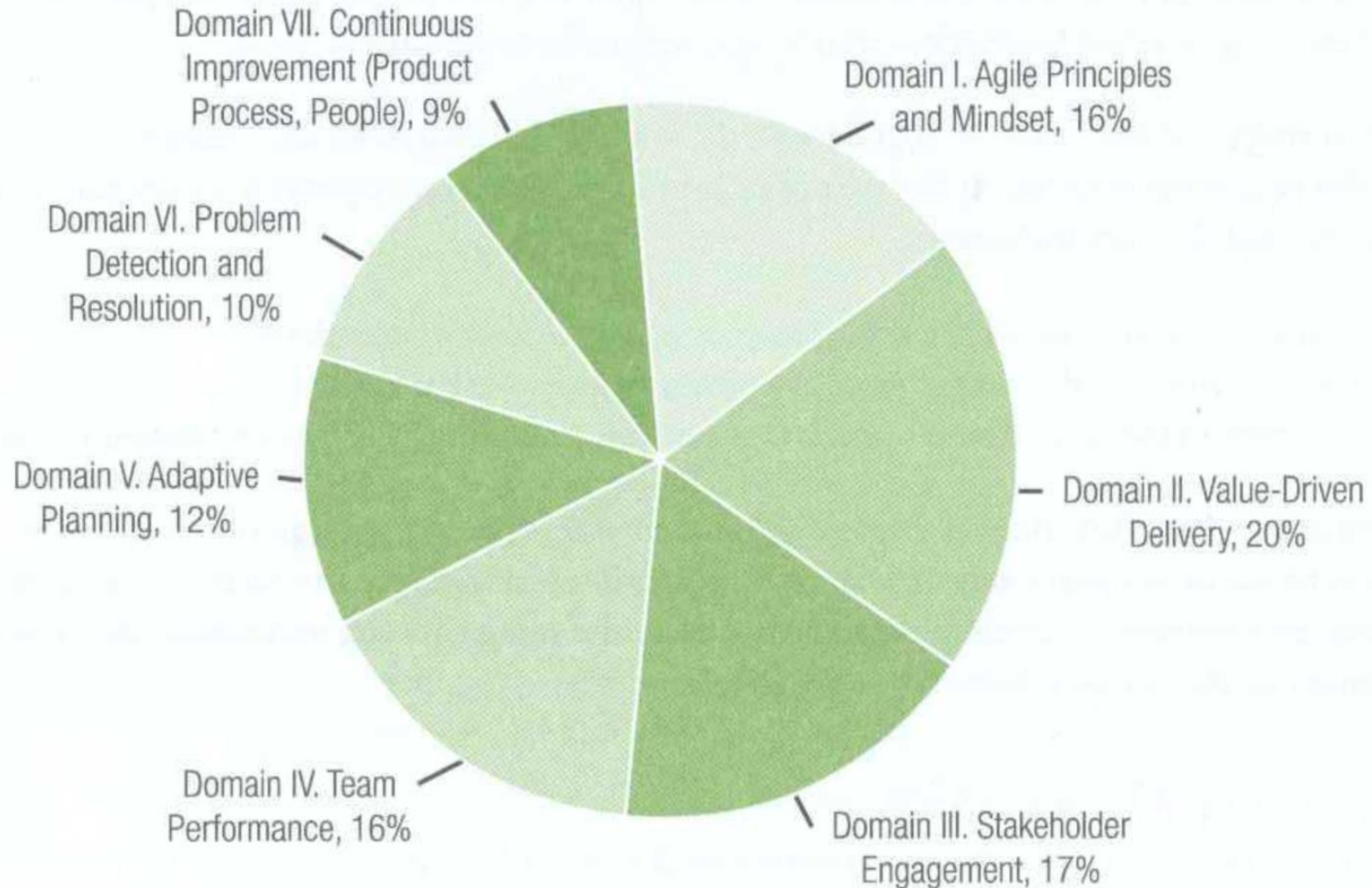
Selenium Detox for Achieving the Right Test Pyramid

Naresh Jain @nashjain

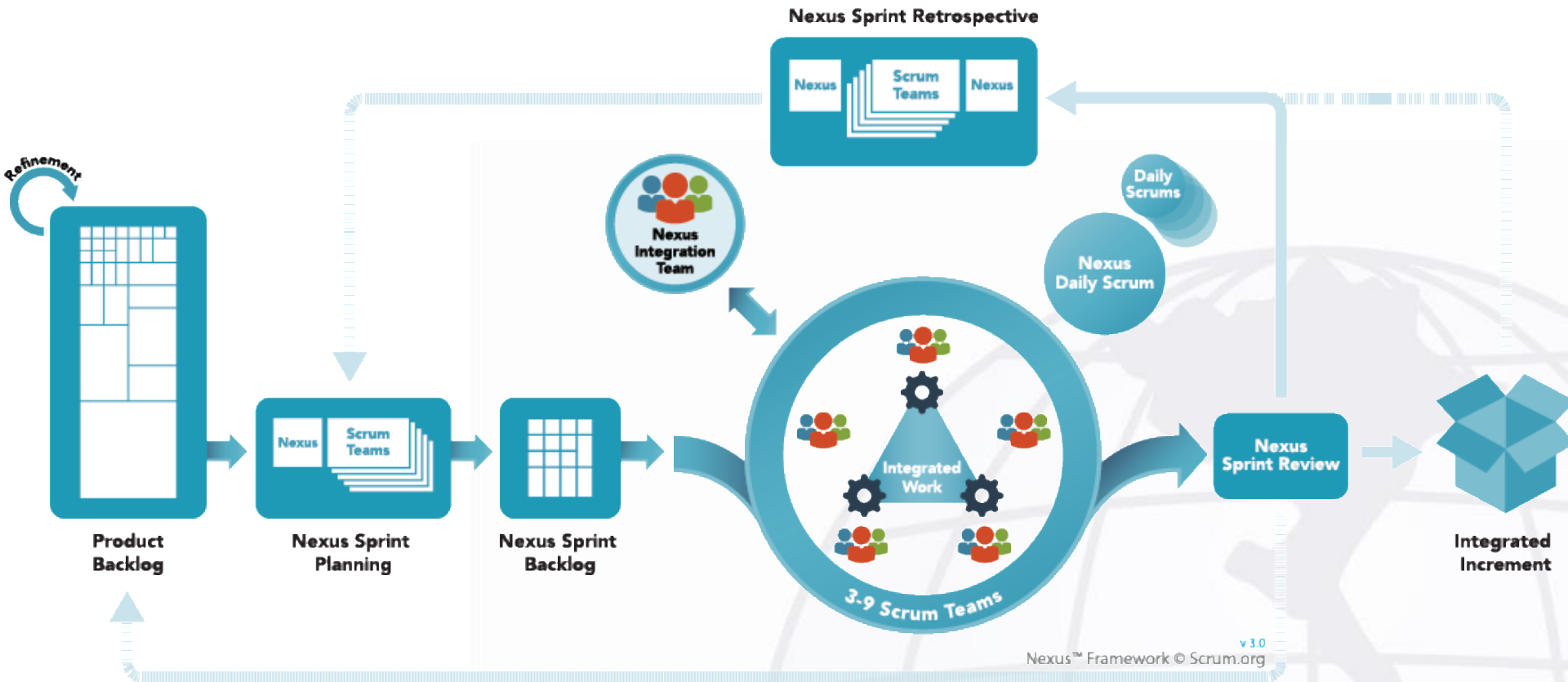


Agile in PMI-ACP

Weighting of Domains for PMI-ACP Exam

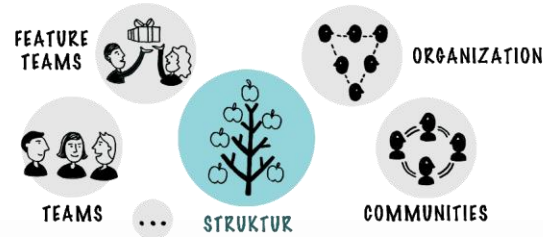


Nexus Framework



Less Framework

LeSS BOOK CHAPTER 2:
INTRODUCTION



SCRUM HANDBOOK

Content

- Transform Traditional to Agile
- **GSOFT SCRUM** Handbook

The Agile: Scrum Framework at a glance

Inputs from Executives,
Team, Stakeholders,
Customers, Users



Transform to Agile (Working Space)



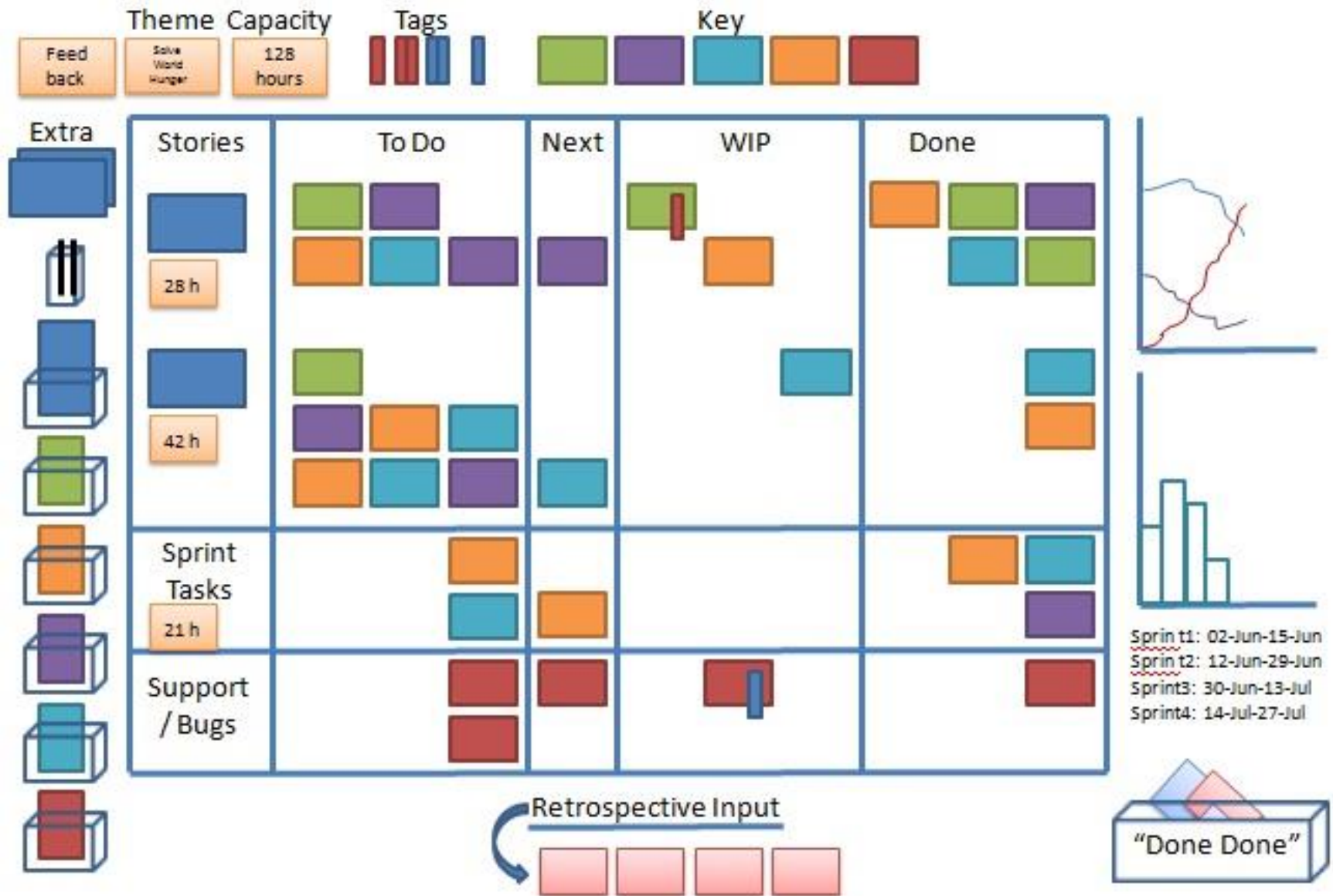
Transform to Agile (Working Space)



Transform to Agile (information radiator)



Transform to Agile (information radiator)



Transform to Agile (information radiator)



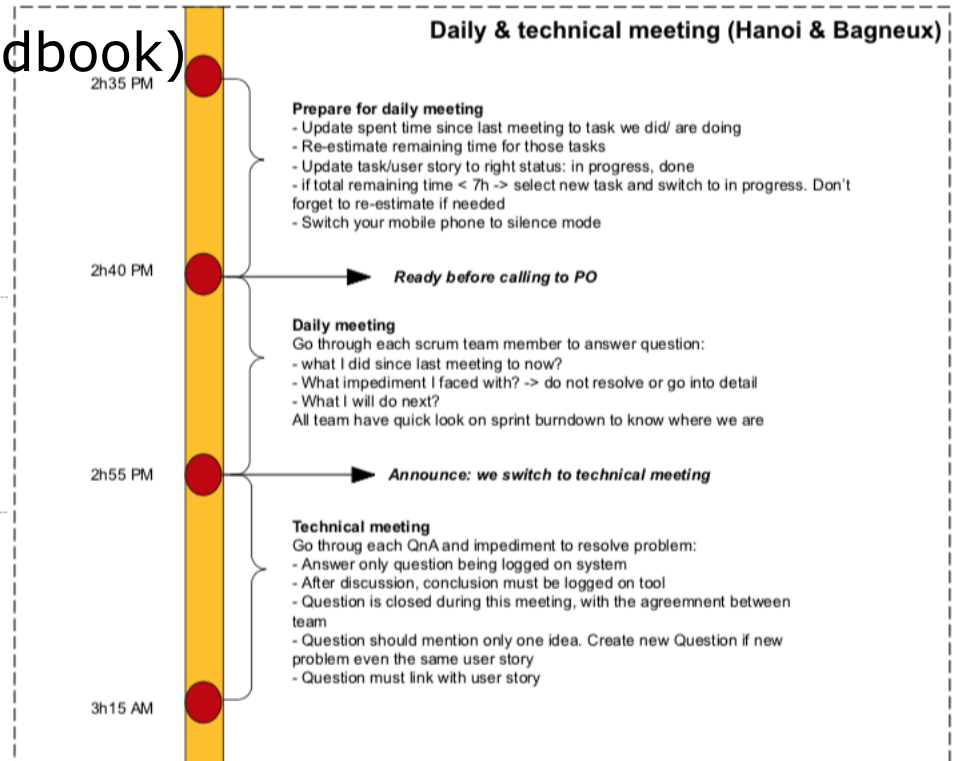
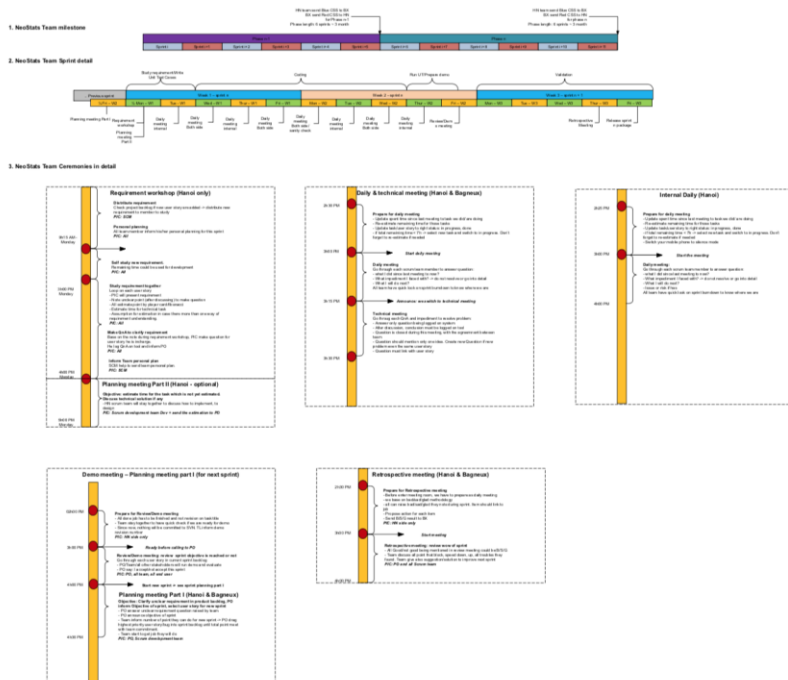
Transform to Agile (CI/CD)

- Versioning Server → Subversion and Rule?
- Analysis Code → Coding Process?
- CI/CD in some commands
- Team/PO/Stakeholder
- “Solution” Server → “CI Env” for Tester, “Feedback Env” for PO, Users and Stateholders

... to be continued or TBD

Transform to Agile (For Team)

• “GSOFT” Scrum Process (Handbook)



- DOR – Definition of Ready
- DOD – Definition of Done
- Team Rules
- Impediment Management

• Agile Tooling

- prefer **low-tech**, high-touch tool over sophisticated computerized models

Transform to Agile (Team Rules)

- Don't start on Monday
- Test Driven Development
- Pair Programming
- Collaboration Games
- Generally Accepted Scrum Practices (GASPs)
 - GASPs can be domain specifics

Establish a Team Working Agreement

Deliverables:

Team Name,
Team members
Roles,
Scrum Room,
Sprint Cadence
Ceremonies,
FOT

Facilitation:

Single Flip
Chart, Sharpie

Team Scrumptious Working Agreement

Product Owner: Janet
Scrum Master: Steven
Team Members: Sasha, Mike, Erick
Teresa

Friends of the
Team: Joel - X1234
Joe - X9876
Pat - X5678

Sprint Length: 1 week, Wednesday to Wednesday
Sprint Planning: Wednesday, 1pm @ Team Conf.
Daily Scrum: Everyday, 10am @ Team Conf.
Refinement: Fri & Mon, 10:30am @ Team Conf.
Demo: Fri & Mon, 10:30am @ Team Conf.
Retro: Fri & Mon, 10:30am @ Team Conf.

Transform to Agile (For Team)



Apprentice

- Contribute to simple tasks
- Learn from journeyman
- Situated learning
- Review work of the master
- Developing through demonstrated progress



Journeyman

- Has worked for several years with a master
- Coaches apprentices
- Learns from masters
- Spreads knowledge between masters
- Focused on delivering applications
- Some journeyman will become a master one day, but many will remain journeymen the rest of their careers



Master Craftsman

- Passionate about the craft
- Guides a team of journeymen and apprentices
- Infects the team with enthusiasm and passion for the craft
- Has delivered many successful, robust, high-quality applications
- Is recognized by users, customers and developers
- Is responsible for passing on the craft

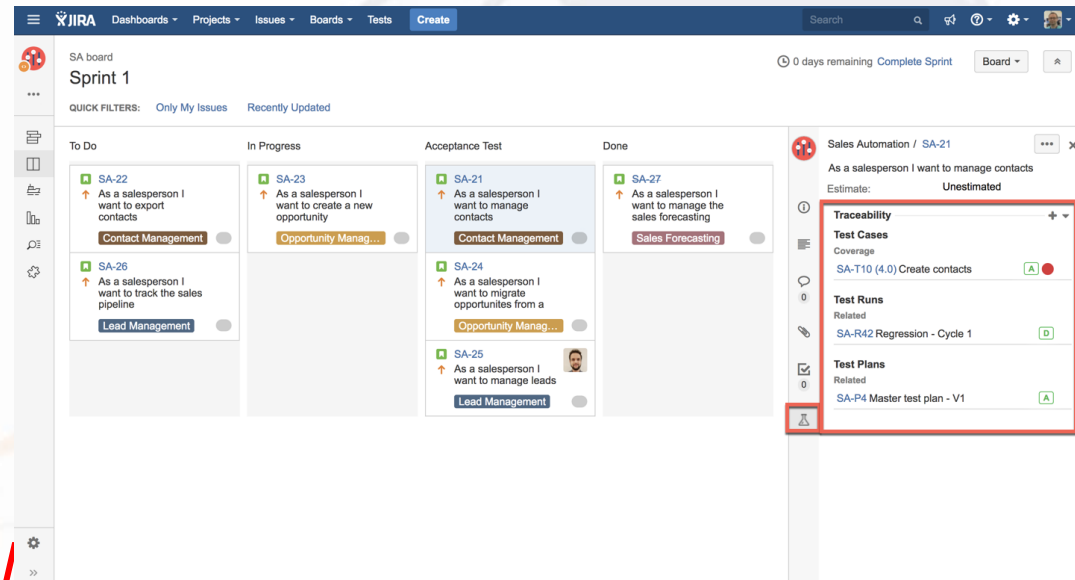
Transform to Agile (For Team)

- Standard User Story and Story Point
- Estimation Methods:
 - Planning poker: www.planningpoker.com or <https://www.amazon.com/Agile-Sizing-Cards-Planning-Estimating/dp/B00IM4ETNK>

• Product Backlog and Sprint Backlog use JIRA
Trello

- Velocity (points)
- Capacity (hours)

... to be continued or TBL



Transform to Agile (For PO)

- Product Vision (Story Map)
- Strategy for specifying Value
- Release Planning
- SRS

... to be continued or TBD

Transform to Agile (Scrum Framework)

JIRA Software

Teams in Space
Scrum: Teams in Space ▾

Backlog
Agile board
Releases
Reports
All issues
Components
Add-ons

PROJECT SHORTCUTS
Mars Team HipChat Room
Space Station Dev Roadmap
Teams in Space Org Chart
Orbital Spotify Playlist
Hyperspeed Bitbucket Repo
+ Add shortcut

TIS-70 Scrum Board

QUICK FILTERS: [Critical partners](#) [Only my partners](#) [Recently updated](#)

12 To do

2 In progress

3 Done

▼ TIS Developer Love 3 issues

TIS-37
↑ Service should return prior trip details and info
[SeeSpaceEZ plus](#) 2

TIS-10
↑ Bad JSON data coming back from hotel API
[SeeSpaceEZ plus](#)

TIS-8
↑ Requesting flights is now taking > 5 seconds
[SeeSpaceEZ plus](#)

▼ Everything Else 21 issues

TIS-68
↑ Homepage footer uses an inline style-should use class
[Large Team Support](#)

TIS-20
↑ Engage Saturn Shuttle lines for group tours
[Space Travel Partners](#) 3

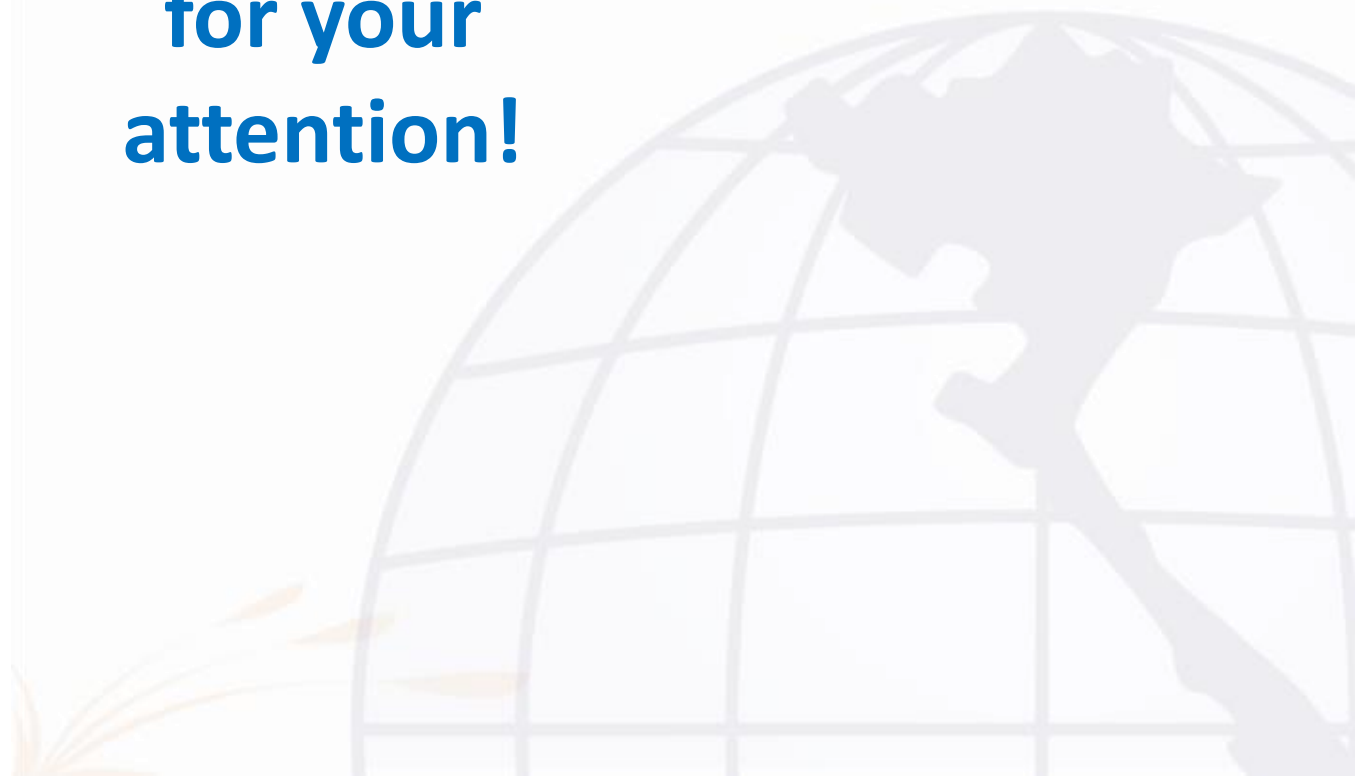
TIS-12
⊘ Create 90 day plans for all departments in Mars office

TIS-17
↑ Engage Saturn's Rings Resort as preferred
[Space Travel Partners](#)

TIS-56
↑ Add pointer to main css file to create child themes
[Large Team Support](#)

TIS-45
↑ Email non registered users to sign up with TIS
[SeeSpaceEZ plus](#)

**Thank you
for your
attention!**



SELF-CHECK QUIZ



- Which statement best describes the Sprint Review?

It is a review of the team's activities during the Sprint.

It is when the Scrum Team and stakeholders inspect the outcome of the Sprint and figure out what to do in the upcoming Sprint.

It is a demo at the end of the Sprint for everyone in the organization to provide feedback on the work done.

It is used to congratulate the Development Team if it did what it committed to doing, or to punish the Development Team if it failed to meet its commitments.

- The Scrum Master is responsible for what? (multichoice)

Ensuring that the Development Team IS managed

Running the Daily Scrum

Tracking Development Team Status

Ensuring that Scrum is enacted

Ensuring that Scrum is understood

SELF-CHECK QUIZ



- A Scrum Master is keeping a list of open impediments, but it is growing and he/she has been able to resolve only a small portion of the impediments. Which three techniques would be most helpful in this situation?

Prioritize the list and work on them in order.

Alert management to the impediments and their impact.

Arrange a triage meeting with all other project managers.

Discuss the absence of management support with the Development Team.

Tell the Product Owner that Scrum isn't working.

Consult with the Development Team.

SELF-CHECK QUIZ



- What is the result of Sprint Review?

A completed Sprint Backlog

Critique on the development product Increment

Critique on the Development Team

The Scrum Master's documentation on the Sprint statistics

A revised Product Backlog

- The Sprint Goal is devised before the sprint-backlog

True

False

SELF-CHECK QUIZ



- Who is responsible for updating the work estimates during a Sprint?

The Development Team.

The Scrum Master.

The Product Owner.

The most junior member of the Team.

- An abnormal termination of a Sprint is called when?

When it is clear at the end of a Sprint that everything won't be finished.

When the Team feels that the work is too hard.

When Sales has an important opportunity.

When the Product Owner determines that it makes no sense to finish it.

SELF-CHECK QUIZ



- Who has the last say on the order of the Product Backlog?

The Stakeholders

The Development Team

The Scrum Master

The Product Owner

- The Product Backlog is ordered by:

Small items at the top to large items at the bottom.

Safer items at the top to riskier items at the bottom.

Least valuable items at the top to most valuable at the bottom.

Whatever is deemed most appropriate by the Product Owner.

- Development Team membership should change:

Every Sprint to promote shared learning.

Never, because it reduces productivity.

As needed, while taking into account the short term reduction in productivity.

Just as it would on any development team, with no special allowance for changes in productivity.

SELF-CHECK QUIZ



- Which two (2) things does the Development Team NOT do during the first Sprint?
 - Deliver an increment of potentially shippable functionality.*
 - Nail down the complete architecture and infrastructure.*
 - Develop and deliver at least one piece of functionality.*
 - Develop a plan for the rest of the project.*

- What is the primary way a Scrum Master keeps a Development Team working at its highest level of productivity?
 - By facilitating Development Team decisions and removing impediments.*
 - By ensuring the meetings start and end at the proper time.*
 - By preventing changes to the Backlog once the Sprint begins.*
 - By keeping high value features high in the Product Backlog.*

SELF-CHECK QUIZ



- It is important that the product increment be released to production or shipped to customers at the end of each Sprint?

True

False

- When is a Sprint over?

When all Product Backlog items meet their definition of done.

When the Product Owner says it is done.

When all the tasks are completed.

When the timebox expires

- The CEO asks the Development Team to add a "very important" item to the current Sprint. What should the Development Team do?

Add the item to the current Sprint without any adjustments.

Add the item to the current Sprint and drop an item of equal size.

Add the item to the next Sprint.

Inform the Product Owner so he/she can work with the CEO

SELF-CHECK QUIZ



- Which statement best describes the Sprint Review?

It is a review of the team's activities during the Sprint.

✔ *It is when the Scrum Team and stakeholders inspect the outcome of the Sprint and figure out what to do in the upcoming Sprint.*

It is a demo at the end of the Sprint for everyone in the organization to provide feedback on the work done.

It is used to congratulate the Development Team if it did what it committed to doing, or to punish the Development Team if it failed to meet its commitments.

- The Scrum Master is responsible for what? (multichoice)

Ensuring that the Development Team IS managed

Running the Daily Scrum

Tracking Development Team Status

✔ *Ensuring that Scrum is enacted*

✔ *Ensuring that Scrum is understood*

SELF-CHECK QUIZ



- A Scrum Master is keeping a list of open impediments, but it is growing and he/she has been able to resolve only a small portion of the impediments. Which three techniques would be most helpful in this situation?
 - ✔ *Prioritize the list and work on them in order.*
 - ✔ *Alert management to the impediments and their impact.*
 - Arrange a triage meeting with all other project managers.*
 - Discuss the absence of management support with the Development Team.*
 - Tell the Product Owner that Scrum isn't working.*
 - ✔ *Consult with the Development Team.*

SELF-CHECK QUIZ



- What is the result of Sprint Review?

- A completed Sprint Backlog*

- Critique on the development product Increment*

- Critique on the Development Team*

- The Scrum Master's documentation on the Sprint statistics*

- ✓ *A revised Product Backlog*

- The Sprint Goal is devised before the sprint-backlog

- ✓ *True*

- False*

SELF-CHECK QUIZ



- Who is responsible for updating the work estimates during a Sprint?

✓ *The Development Team.*
The Scrum Master.
The Product Owner.
The most junior member of the Team.

- An abnormal termination of a Sprint is called when?

When it is clear at the end of a Sprint that everything won't be finished.
When the Team feels that the work is too hard.
When Sales has an important opportunity.
✓ *When the Product Owner determines that it makes no sense to finish it.*

SELF-CHECK QUIZ



- Who has the last say on the order of the Product Backlog?

The Stakeholders

The Development Team

✓ *The Scrum Master*

The Product Owner

- The Product Backlog is ordered by:

Small items at the top to large items at the bottom.

Safer items at the top to riskier items at the bottom.

Least valuable items at the top to most valuable at the bottom.

✓ *Whatever is deemed most appropriate by the Product Owner.*

- Development Team membership should change:

Every Sprint to promote shared learning.

Never, because it reduces productivity.

✓ *As needed, while taking into account the short term reduction in productivity.*

Just as it would on any development team, with no special allowance for changes in productivity.

SELF-CHECK QUIZ



- Which two (2) things does the Development Team NOT do during the first Sprint?
 - Deliver an increment of potentially shippable functionality.*
 - ✔ *Nail down the complete architecture and infrastructure.*
 - Develop and deliver at least one piece of functionality.*
 - ✔ *Develop a plan for the rest of the project.*

- What is the primary way a Scrum Master keeps a Development Team working at its highest level of productivity?
 - ✔ *By facilitating Development Team decisions and removing impediments.*
 - By ensuring the meetings start and end at the proper time.*
 - By preventing changes to the Backlog once the Sprint begins.*
 - By keeping high value features high in the Product Backlog.*

SELF-CHECK QUIZ



- It is important that the product increment be released to production or shipped to customers at the end of each Sprint?

True

✓ *False*

- When is a Sprint over?

When all Product Backlog items meet their definition of done.

When the Product Owner says it is done.

When all the tasks are completed.

✓ *When the timebox expires*

- The CEO asks the Development Team to add a "very important" item to the current Sprint. What should the Development Team do?

Add the item to the current Sprint without any adjustments.

Add the item to the current Sprint and drop an item of equal size.

Add the item to the next Sprint.

✓ *Inform the Product Owner so he/she can work with the CEO*